

Multimedia

Vorlesung

Hannes Kaufmann

Research Unit Virtual and Augmented Reality
Institute of Visual Computing & Human-Centered Technology

Based on material by Prof. Christian Breiteneder, a few slides by Prof. Robert Sablatnig (EVC)
Material on HDR by Scott Janus, Principal Engineer at Intel; Thuong Nguyen tnguyen@humaxdigital.com; Prof. Marc Pollefeys and Dr. Gabriel Brostow at ETHZ
A few slides on JPEG used by courtesy of Dr. Dirk Thißen from RWTH Aachen

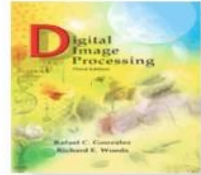
Media Types

- History, representation, formats and operations of media types
- Text
- Audio
- Image
- Video
- No established digital standards for other media types:
smell, taste, haptics

Media Type: Image

- Human Visual System & Color Models
- High Dynamic Range
- Image Compression

Human Visual System



Digital Image Processing, 3rd ed.
Gonzalez & Woods
www.ImageProcessingPlace.com



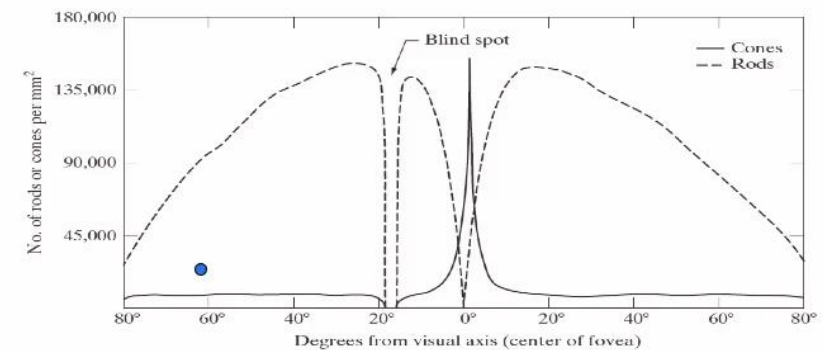
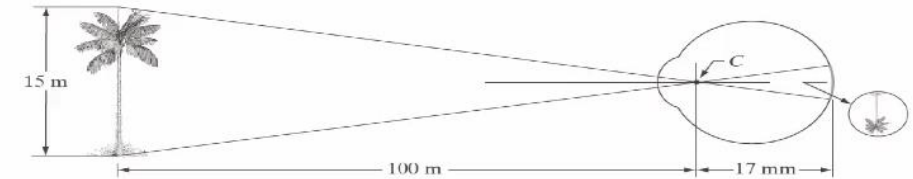
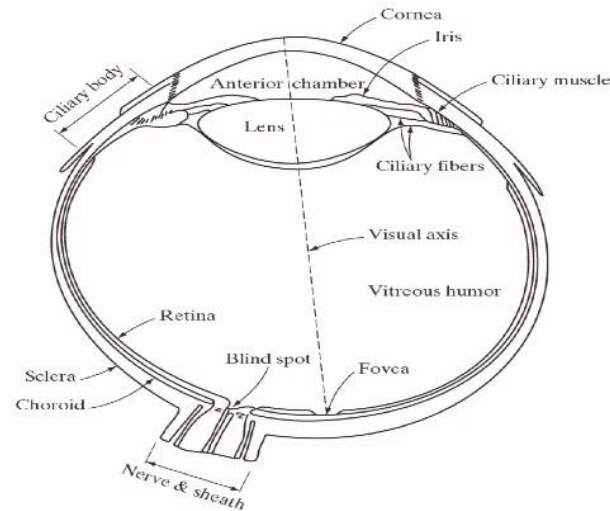
Chapter 2 Digital Image Fundamentals

Helligkeitssehen:

~120 Mill. "Stäbchen"/Rods

Farbsehen:

~7 Mill. "Zapfen"/Cones



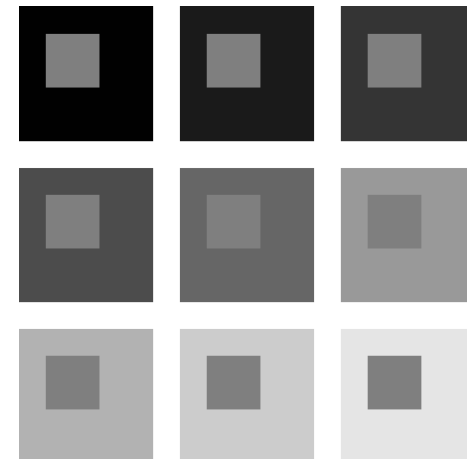
© 1992–2008 R. C. Gonzalez & R. E. Woods

Light

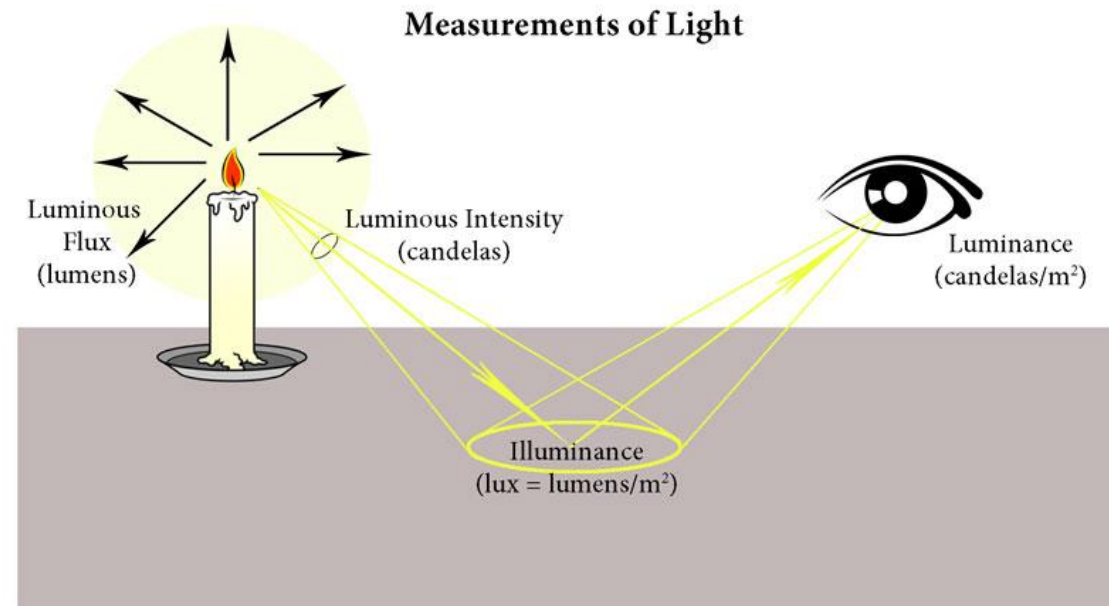
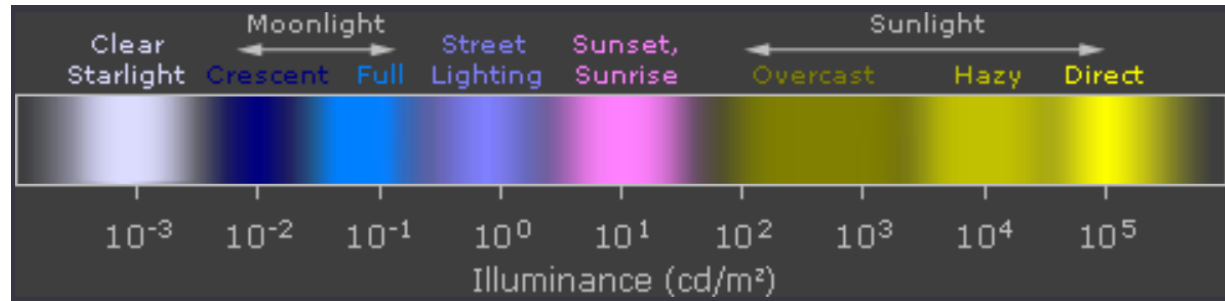
- Chromatic: of color
- Achromatic: without color (i.e. B&W)
- Color of an object depends upon:
 - object surface: reflectivity, physical properties, composition
 - light source(s) illuminating it
 - color of surrounding environment
 - visual system (eyes, brain, photoelectric cell, ...) of perceiving entity

Achromatic Light

- black, white, usually greys (rare to see pure black or pure white)
- quantity or energy of light is the only measurement used for achromatic light
 - Intensity, luminance, brightness:
 - **Luminance** describes the amount of light that is emitted from, passed through, or reflected off an object. It is measured using candela per square meter (cd/m^2), which is the concentration of luminous intensity per unit area.
 - **Brightness** is subjective and perceived by the human eye. It cannot be measured by any formula
- eye is sensitive to ratios of intensity changes, not the changes in intensity values themselves
- small changes of low intensity seem to have the same effect as larger changes of high intensity
- **Dynamic range**: ratio between min. and max. intensities



Illuminance

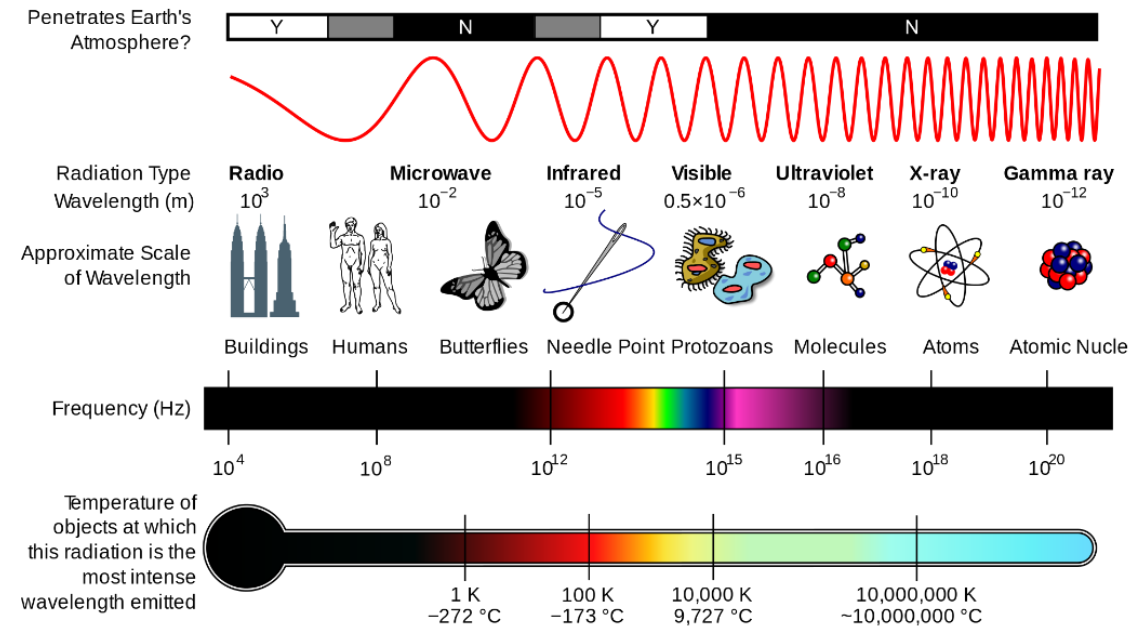
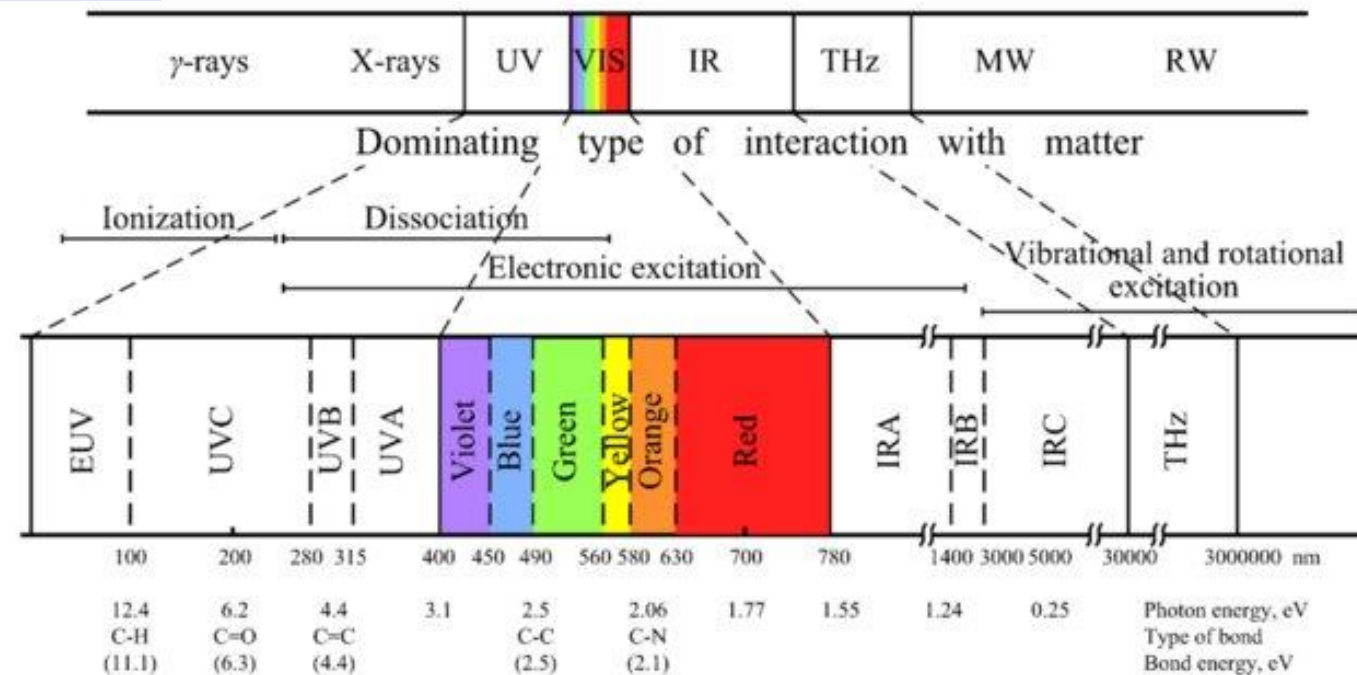


Illuminance = only incident light

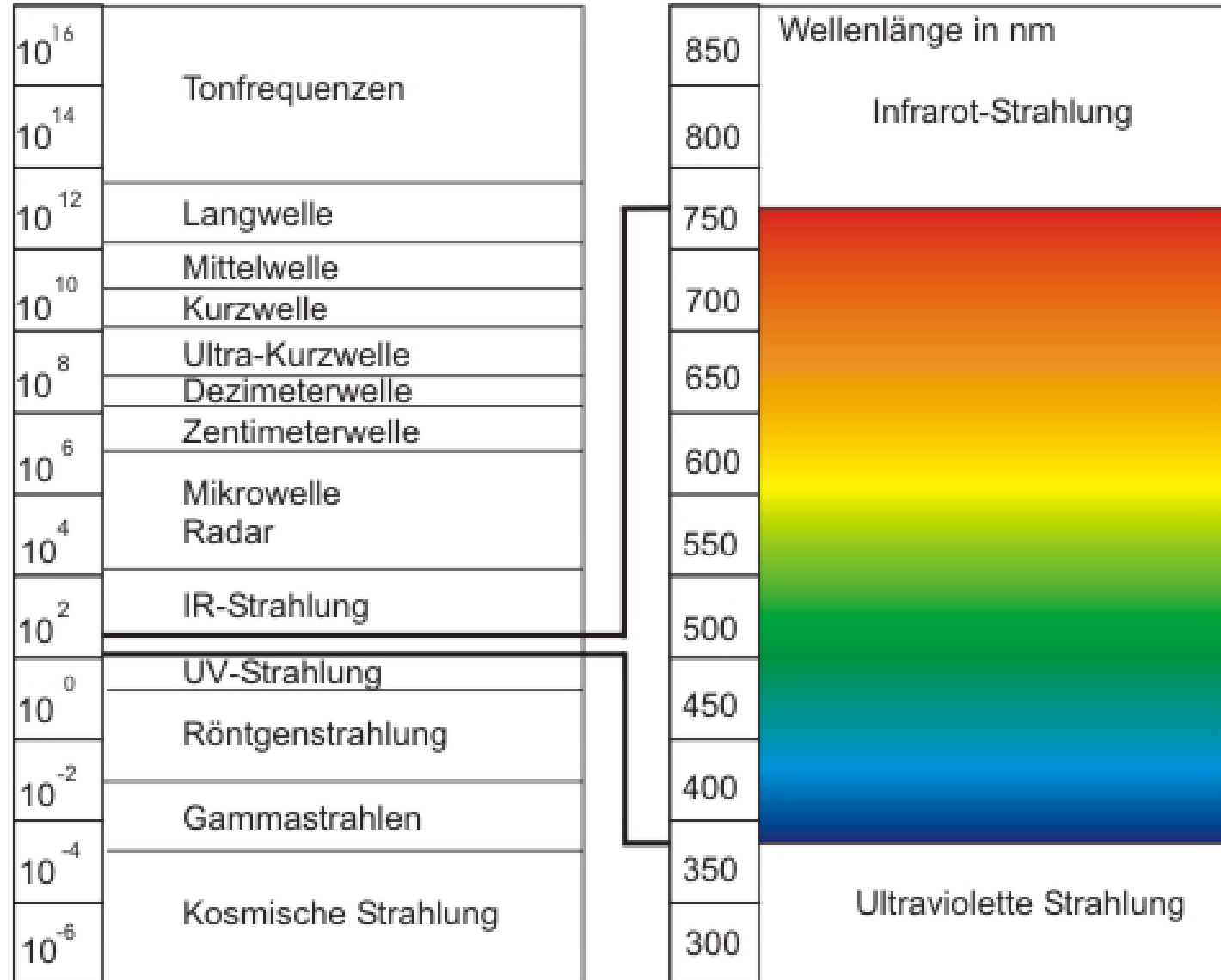
- Illuminance is light we can't see directly, so-called ambient light;
- Light passing through air which is actually invisible until it reflects off an object (which we then see as luminance).
- It is measured in lux - **luminous flux** per unit area (lx or lm/m^2).

Color, Light - Wavelengths

- Color is how we perceive beams of electromagnetic energy that fall on our retina
- We see light between 400 and 700 nanometers in wavelength (10^{-9} meters)



Light Spectrum – Wavelengths

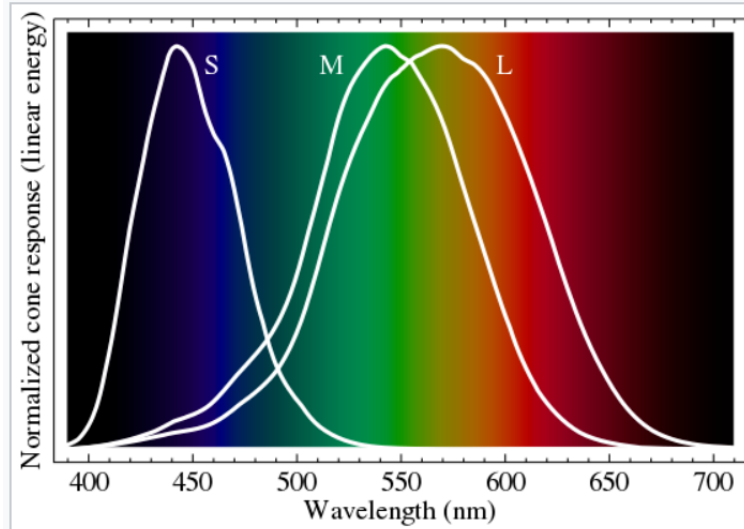


Color

- Various models of color used:
 - Printing & graphics industry: color sample books and codes
 - Artists: tint (adding white to pure pigment), shade (adding black), tone (adding both)
 - Displays: use hardware capabilities (RGB)
 - Physicists: use optical models (wavelengths, energy measures)
- Human perception of color depends on how brain reacts to the whole visual environment, as well as brain & eye biological factors

Tristimulus Model

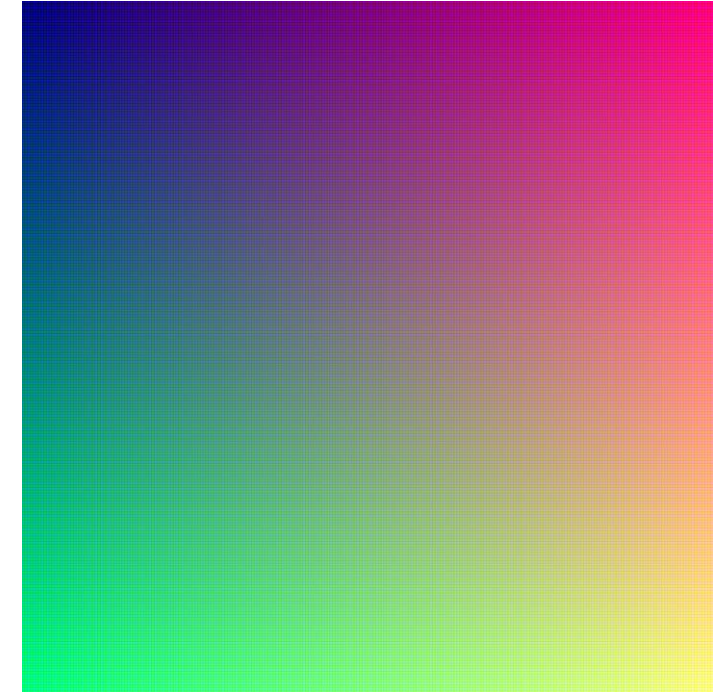
- The retina contains cones that sense either red, green, or blue light
- 6-7 million cones per eye, concentrated in central area called "fovea"
- Rods: sensors that surround fovea, and perceive weak "night" images (>100 million per eye)



Normalized response spectra of human cones, 
to monochromatic spectral stimuli, with
wavelength given in nanometers.

Color vision in Nonhumans

The human eye can distinguish about 10 million different colors (up to debate)



16.7 million colors

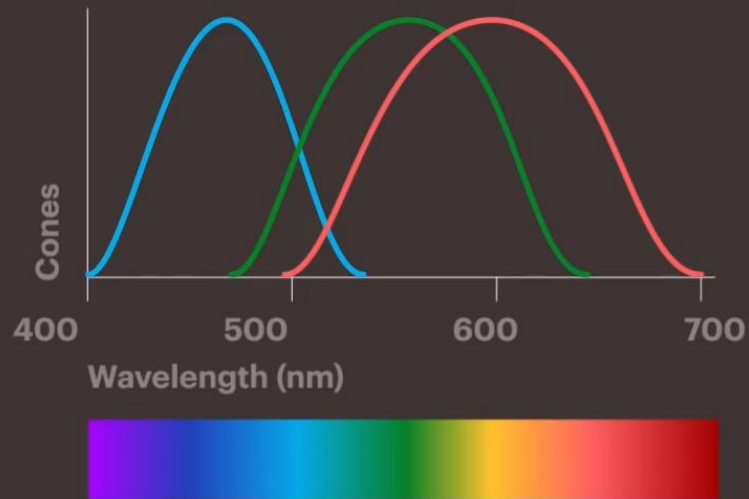
Color vision table

State	Types of cone cells	Approx. number of colors perceived	Carriers
Monochromacy	1	200	Marine mammals, owl monkey, Australian sea lion, achromat primates
Dichromacy	2	40,000	Most terrestrial non-primate mammals, color blind primates
Trichromacy	3	10 million ^[54]	Most primates, especially great apes (such as humans), marsupials, some insects (such as honeybees)
Tetrachromacy	4	100 million	Most reptiles, amphibians, birds and insects, rarely humans
Pentachromacy	5	10 billion	Some insects (specific species of butterflies), some birds (pigeons for instance)

The Amazing Ways Animals See the World

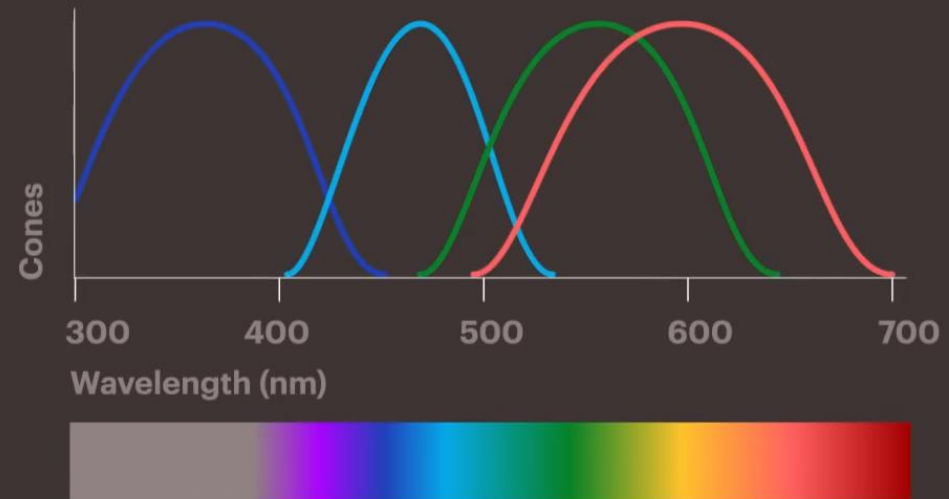
<https://www.youtube.com/watch?v=LlfKk37bkyk>

Humans



3 types of cones

Birds



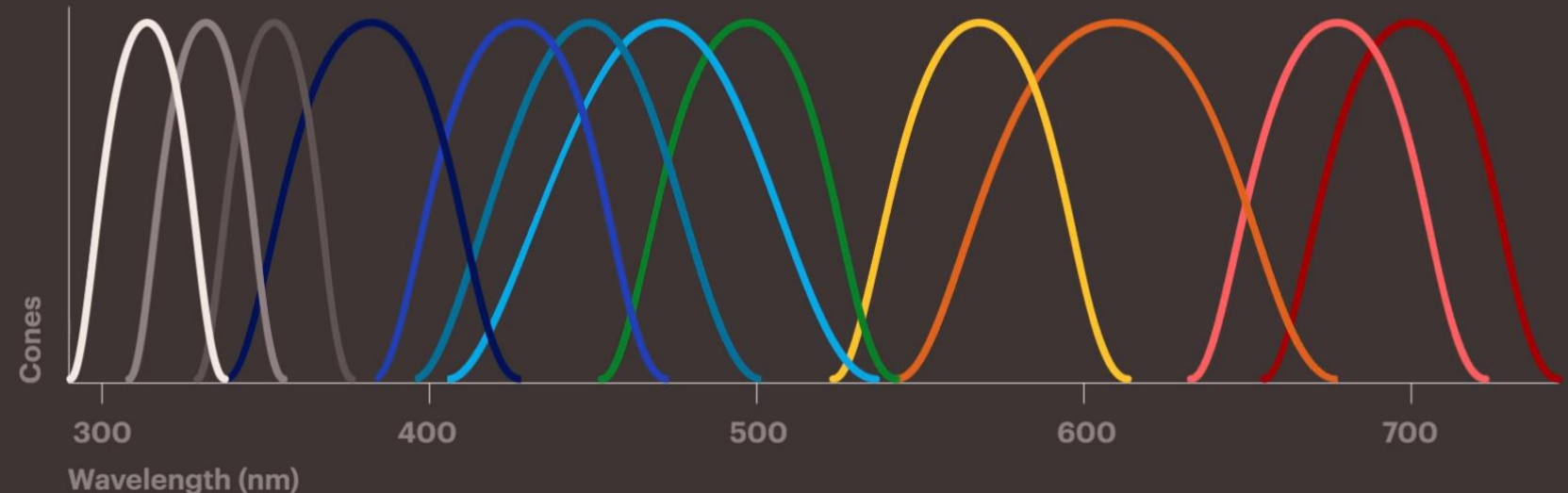
4 types of cones

Mantis Shrimp:

- 12-16 types of cones
- independent eye movement
- Sees ultraviolet, infrared and 6 types of polarization

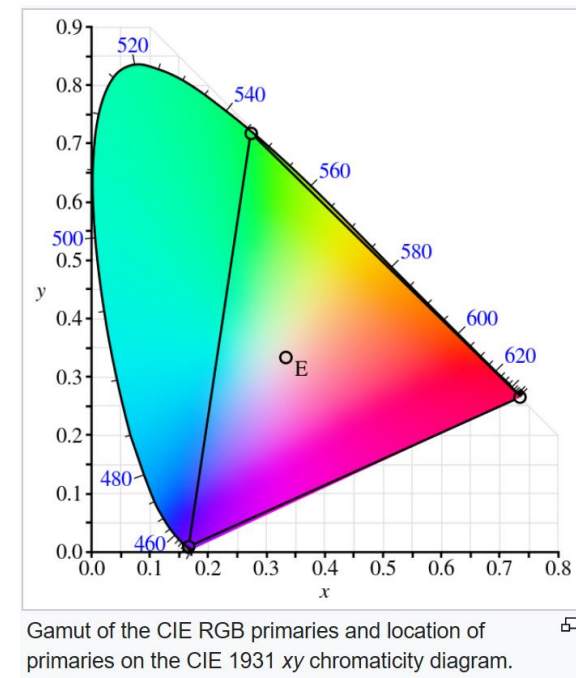


Mantis Shrimp



CIE XYZ Color Space (1931)

- CIE color space (CIE = Commission Internationale de l'Eclairage, International Commission on Illumination) used to calibrate other color models;
1931 CIE XYZ
- Device-invariant representation of color
- Serves as a standard reference against which many other color spaces are defined
- Many combinations of dominant wavelength, luminance, saturation appear the same to us



CIE Chromaticity Diagram

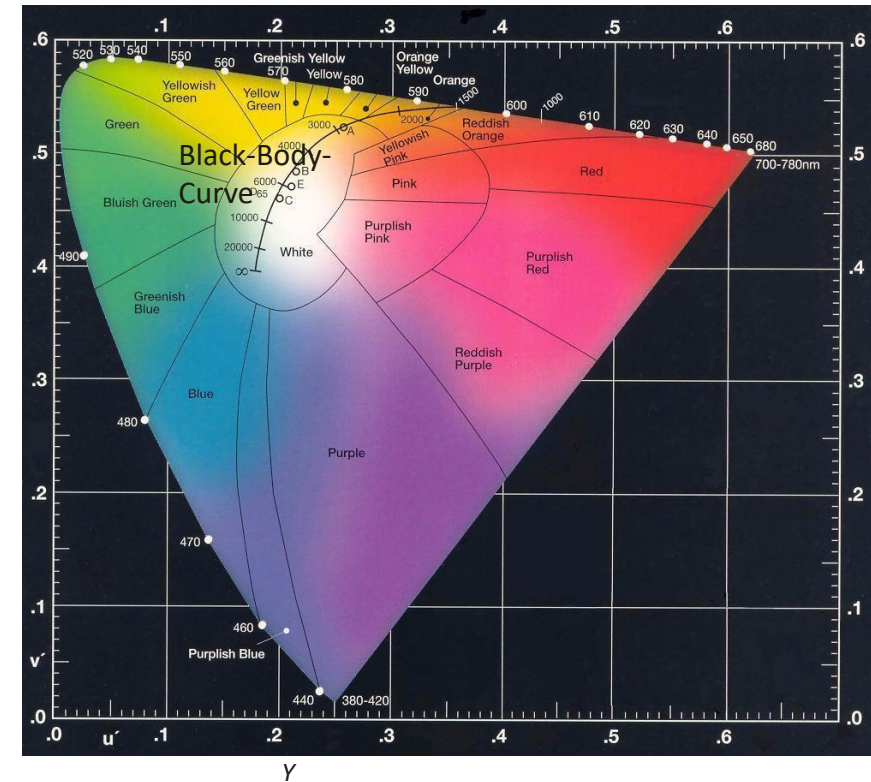
- A full plot of all human visible colors is a three-dimensional figure (cone).
- The concept of color can be divided into two parts: **luminance** and **chromaticity**
- CIE Diagram shows a **planar cut** through that cone at a specific luminance
- Color described in x, y (with $x+y+z=1$)
- Luminance = Y
- Full color: (x, y, Y)

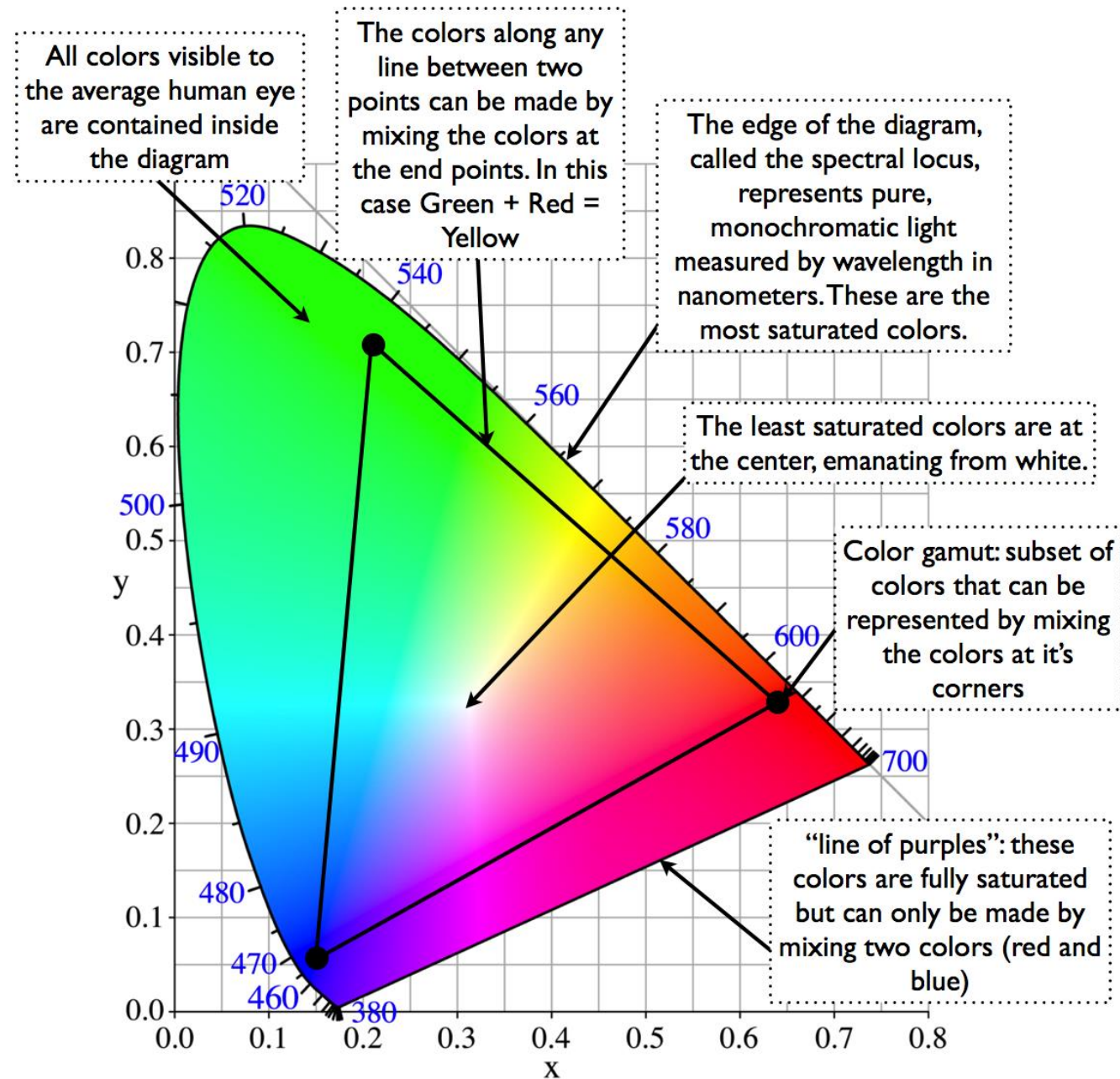
Helligkeitssehen:

~120 Mill. "Stäbchen"/Rods

Farbsehen:

~7 Mill. "Zapfen"/Cones





Anatomy of a CIE Chromaticity Diagram

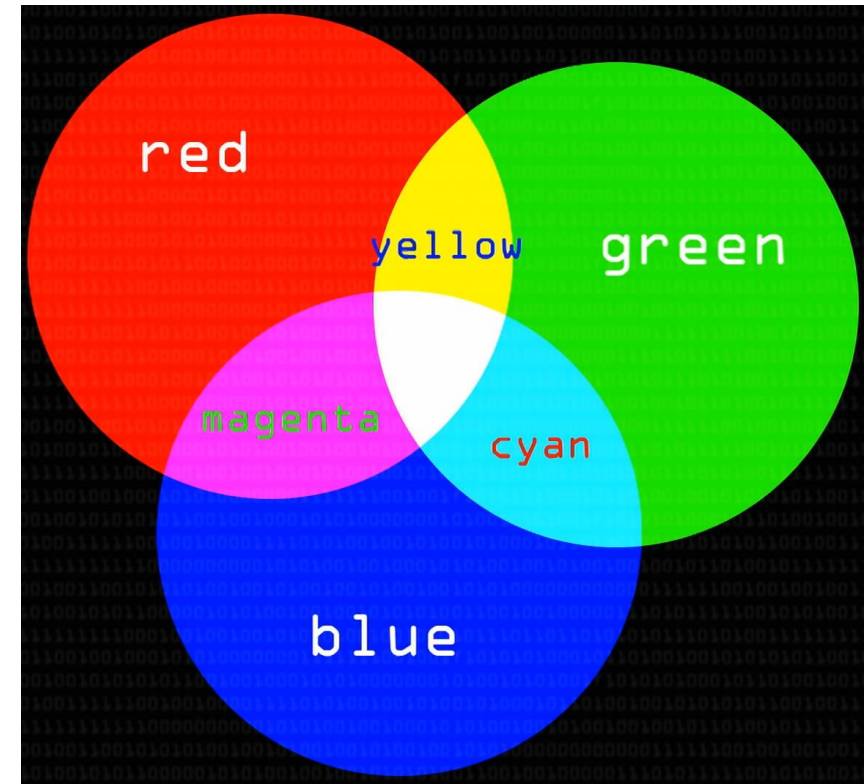
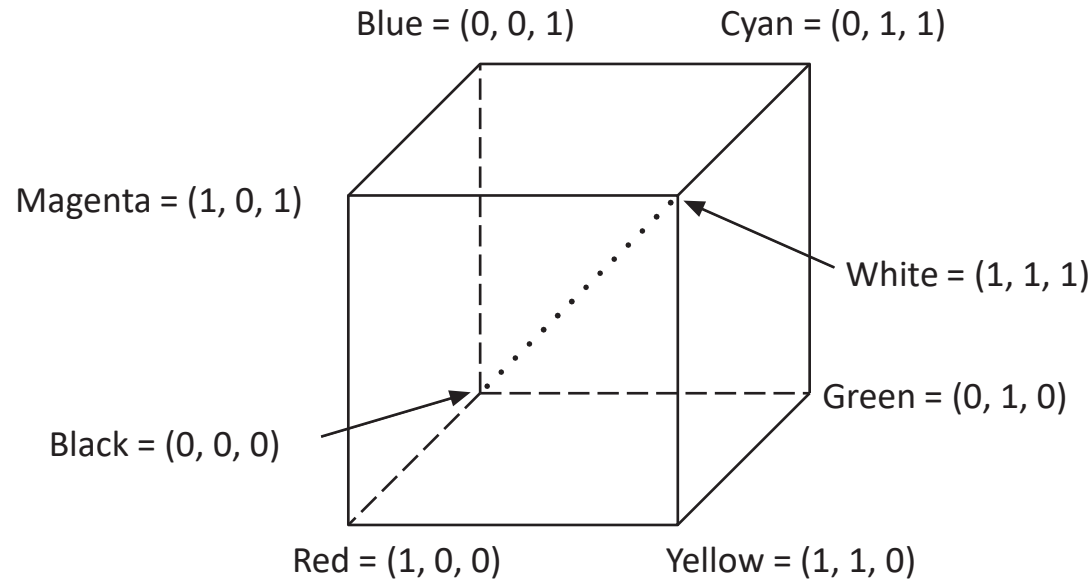
Quick Overview of Color Spaces



Color Models (Spaces) Overview

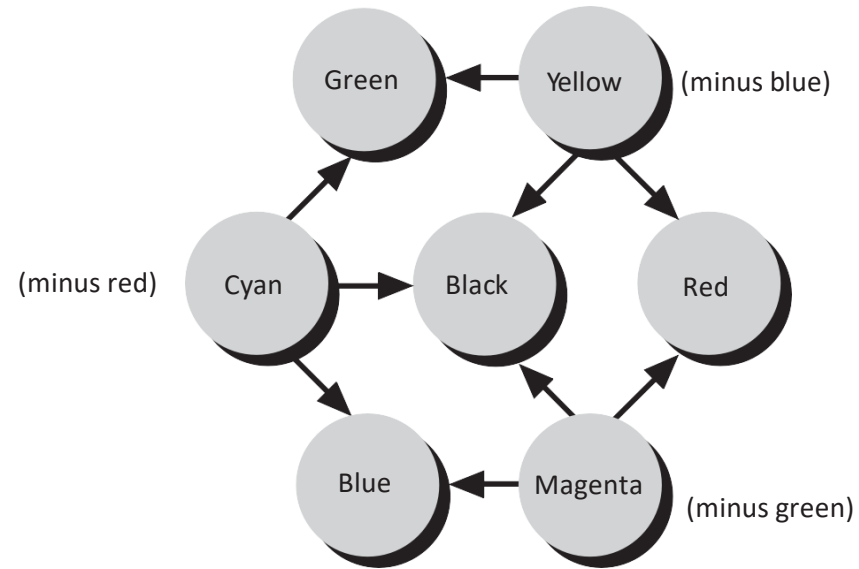
- CIE color space (Commission Internationale de l'Eclairage) used to calibrate other color models 1931 CIE XYZ, tristimulus theory
- RGB — for video display drivers
- CMYK — cyan, magenta, yellow, black, (subtractive primaries)
- HSB — hue (dominant color), saturation (amount of gray), brightness (intensity)
- YUV — used in television industry, Y (luminance), UV (color difference signals) = analog
- YCbCr = digital; Y (luminance), C_B and C_R are the blue-difference and red-difference chroma components

The RGB Color Model

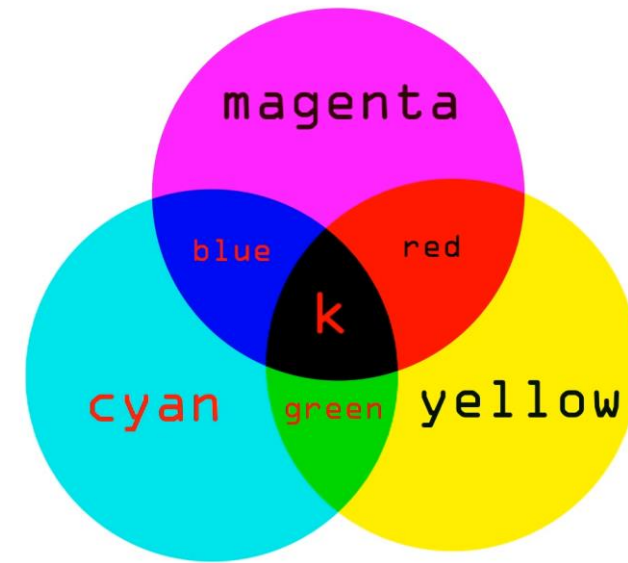


- Hardware-oriented model that turns on red, green, and blue pixels at varying intensities
- Additive: add red, green, blue together to get overall color
- Not all colors are exactly duplicable, and gamut (color range) covered by a display depends on performance of its display technology

The CMY Color Model



$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} C \\ M \\ Y \end{bmatrix}$$

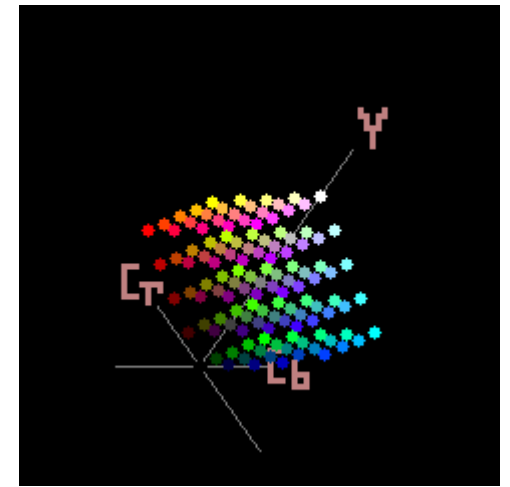


- Another related model, CMYK, uses black as 4th color:

$$K = \min(C, M, Y), C = C - K, M = M - K, Y = Y - K$$

YCbCr / YUV

- Splits **luminance** (**Y**) from color (**chromacity**)
Cb (Blue-Yellow Chrominance) und **Cr** (Red-Green Chrominance)
- Historically YUV (analog version) was used to transmit color (UV) to PAL/SECAM standard color TV sets, while black/white TVs only received a Y luminance signal and would still work. The same signal was transmitted for analog b/w and color TVs.
- Humans are very sensitive to changes in luminance, less to chromacity.
- Reducing color resolution allows great compression of images & videos (is used in JPEG)



Conversion YCbCr <-> RGB

- Necessary in preparation for JPEG compression
- YCbCr (256 levels) can be computed directly from 8-bit RGB as follows:

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

- During decoding YCbCr color space needs to be converted back to RGB:

$$R = Y + 1.402 (Cr-128)$$

$$G = Y - 0.34414 (Cb-128) - 0.71414 (Cr-128)$$

$$B = Y + 1.772 (Cb-128)$$

Color Display – Color Depth

image descriptor = {
 image width = 640
 image height = 480
 image depth = 24 bit
 color model = RGB
 encoding = YUV 8:2:2, JPEG}

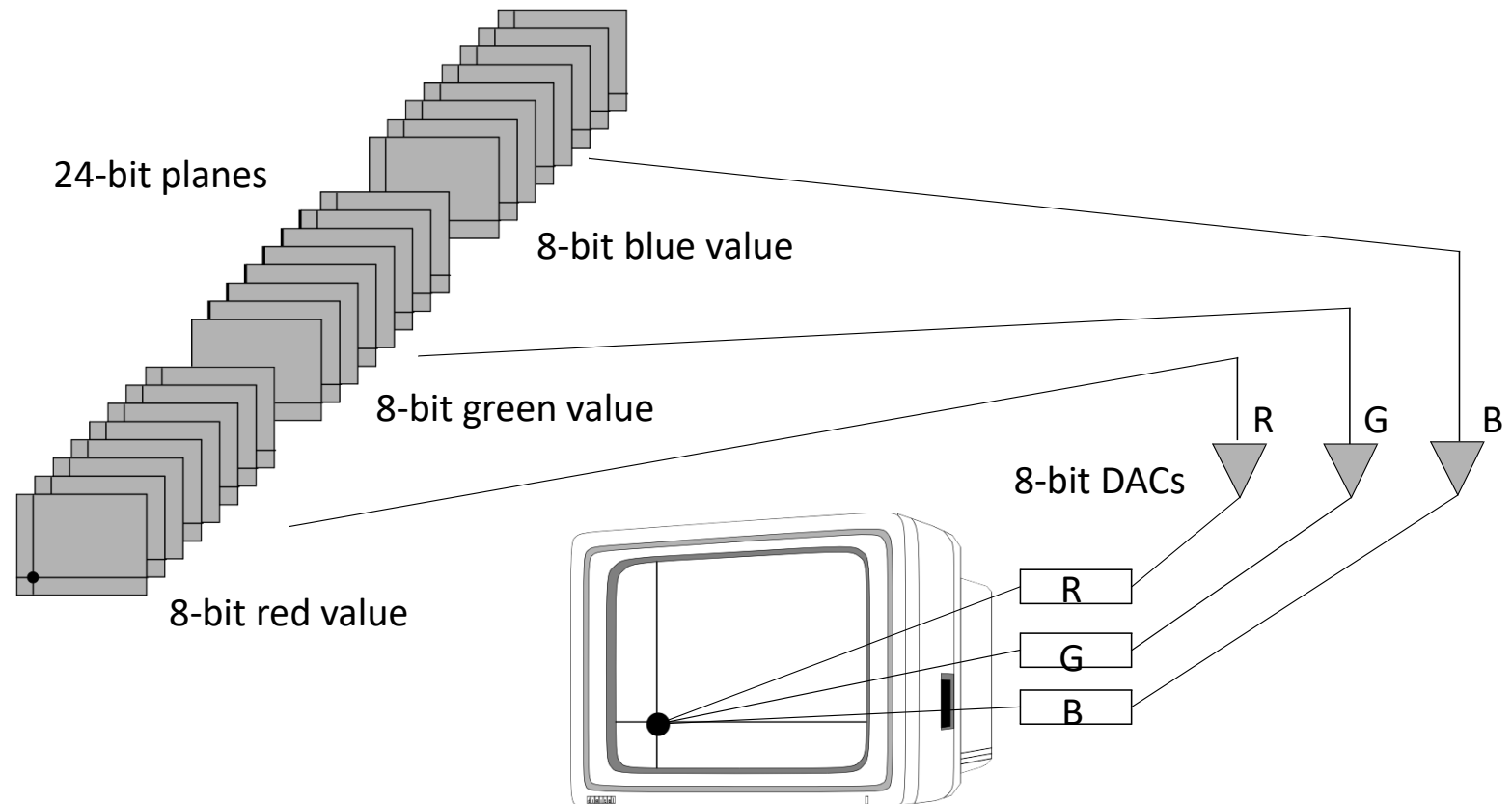
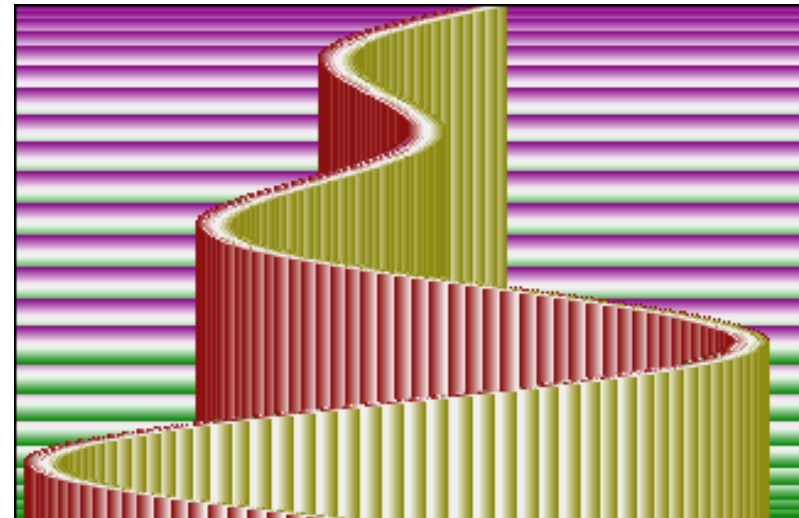


Image Representations

- Number of channels — dim. of color model plus **alpha channels**
 - e.g. 32 bit color depth = 24 bit color (8/8/8) + 8 bit alpha channel for transparency
- Channel depth — number of bits-per-pixel
- Alpha — controls transparency; masks and blends
- Pixel aspect ratio (usually square pixels nowadays)
- Interlacing (important for video)
- Compression — lossless vs. lossy
- Indexing — colors are represented by index in color maps or color lookup tables [CLUT]; map predefined or part of image





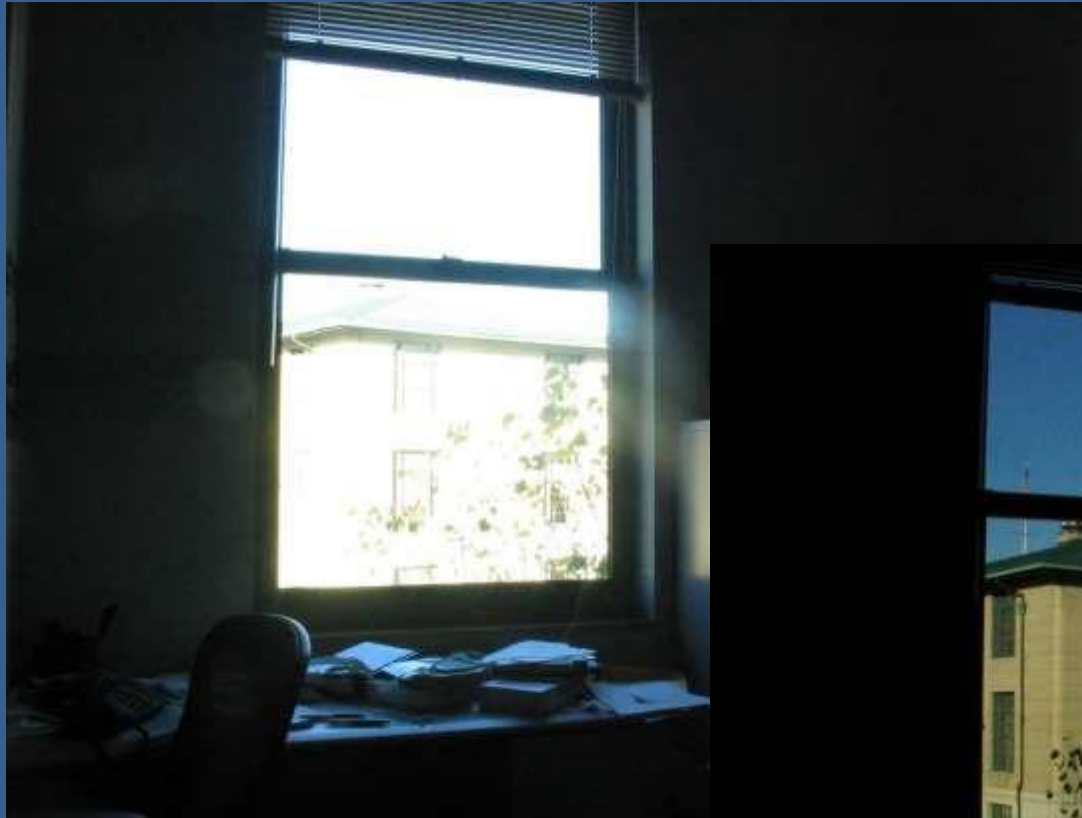
HIGH DYNAMIC RANGE DEMYSTIFIED

Scott Janus, Principal Engineer

Slides adapted from Scott Janus, Principal Engineer at Intel; Thuong Nguyen tnguyen@humaxdigital.com; Prof. Marc Pollefeys and Dr. Gabriel Brostow at ETHZ

Problem of Dynamic Range

The Grandma Problem

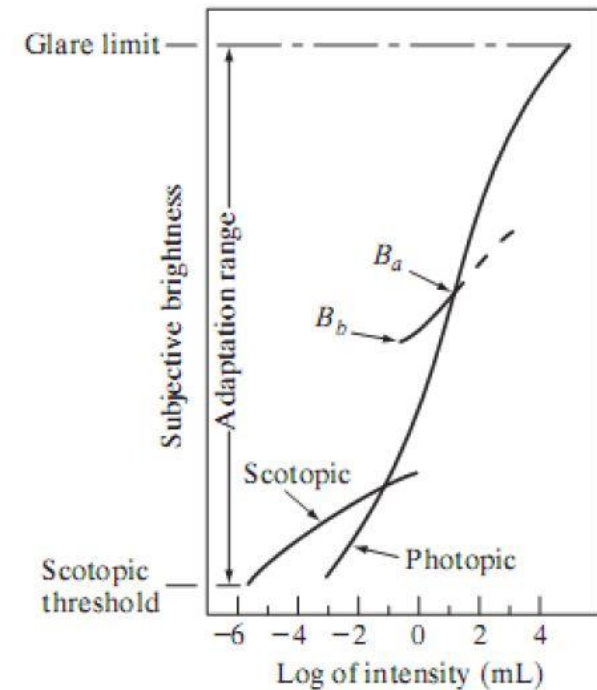


Dynamic Range: Ratio of largest to smallest signal in a system



What is High Dynamic Range?

- Display does not have enough brightness to show the whole range.
- High Dynamic Range:
Content with a **wider range of brightness and color**
 - Also an increased number of brightness and color levels
- Experiencing HDR content requires new displays!



Color is complicated



“The Dress”

https://en.wikipedia.org/wiki/The_dress

White and gold

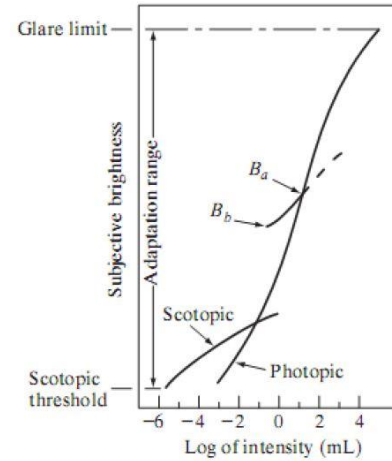
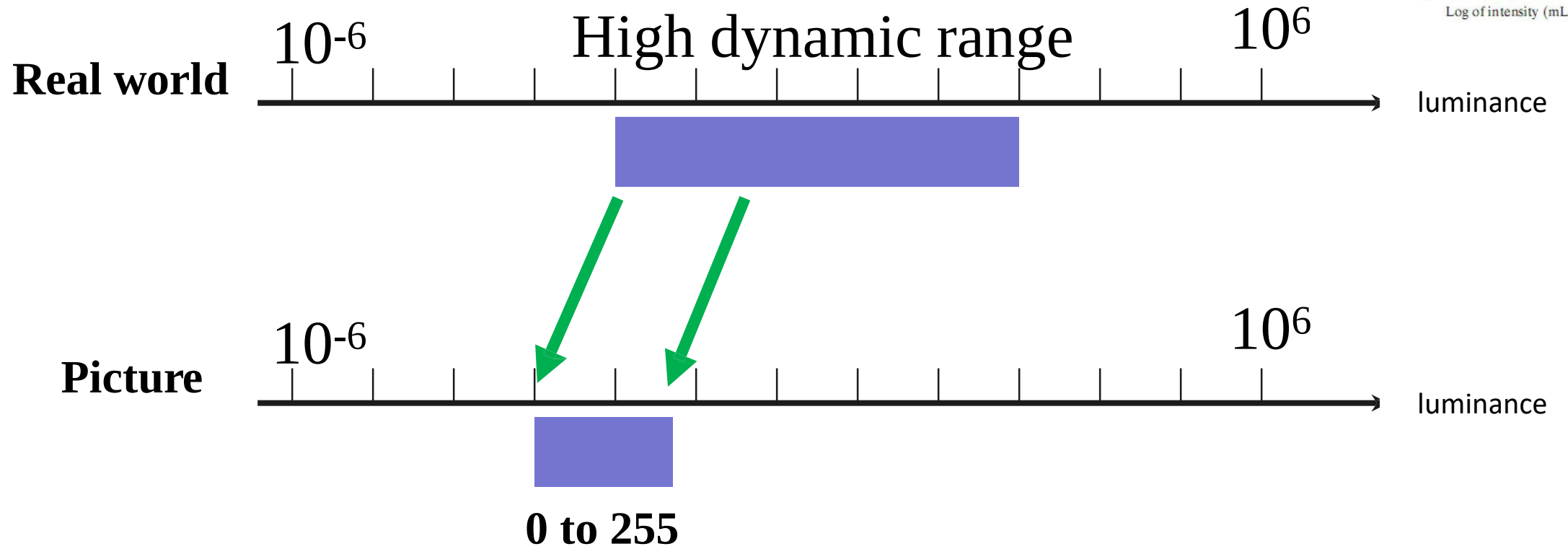
Or

Black and blue?

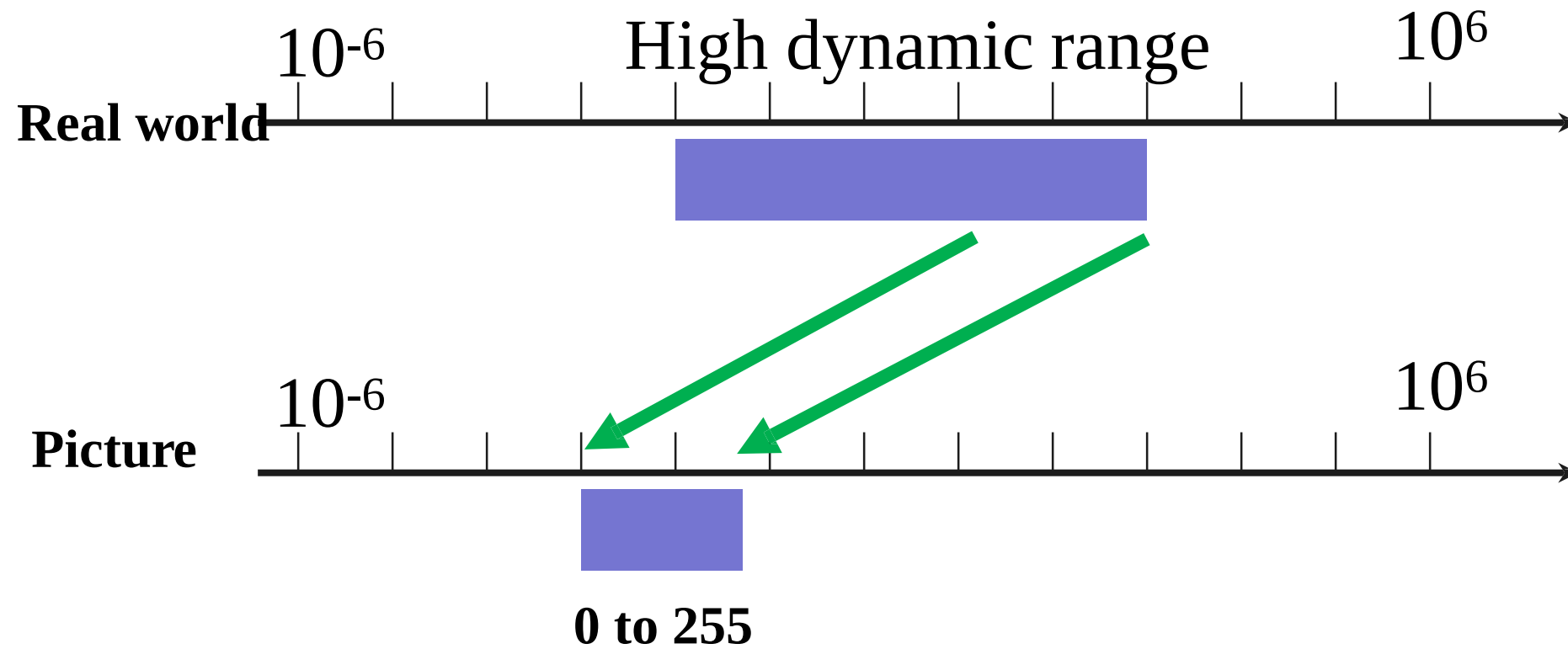


The real dress is
Blue with black stripes

Long Exposure

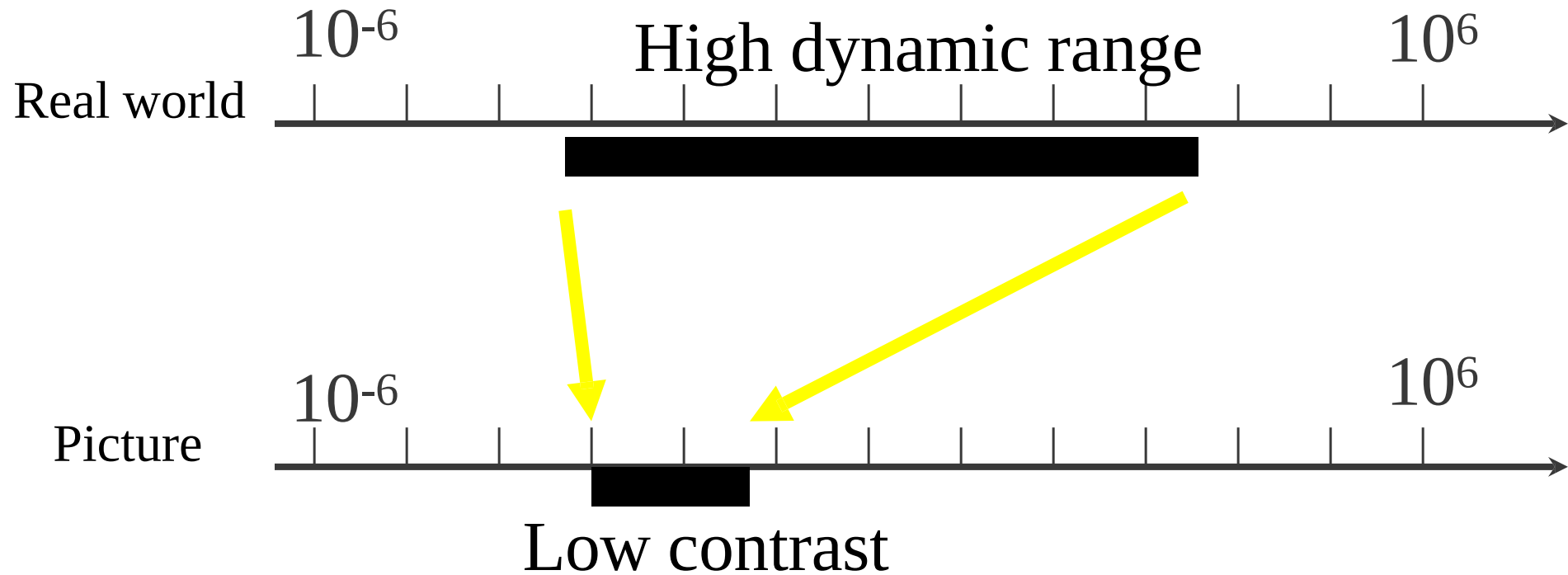


Short Exposure



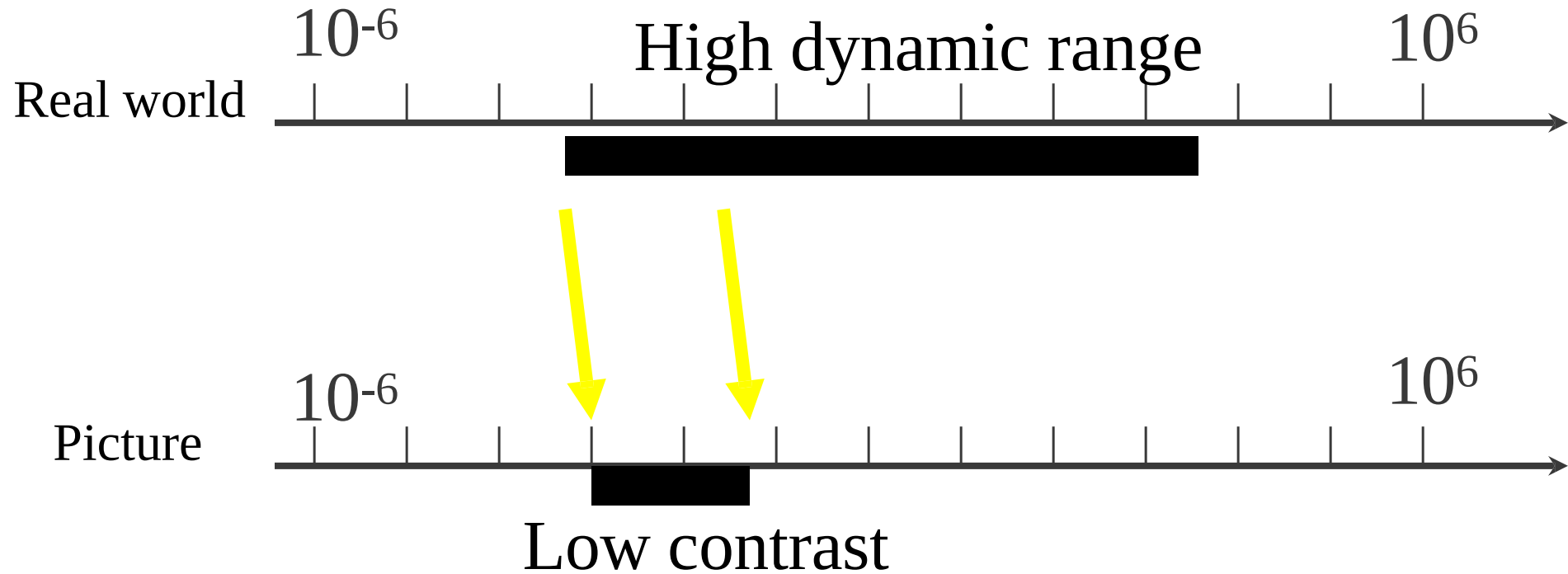
Multiple exposure photography

- **Sequentially measure all segments of the range**



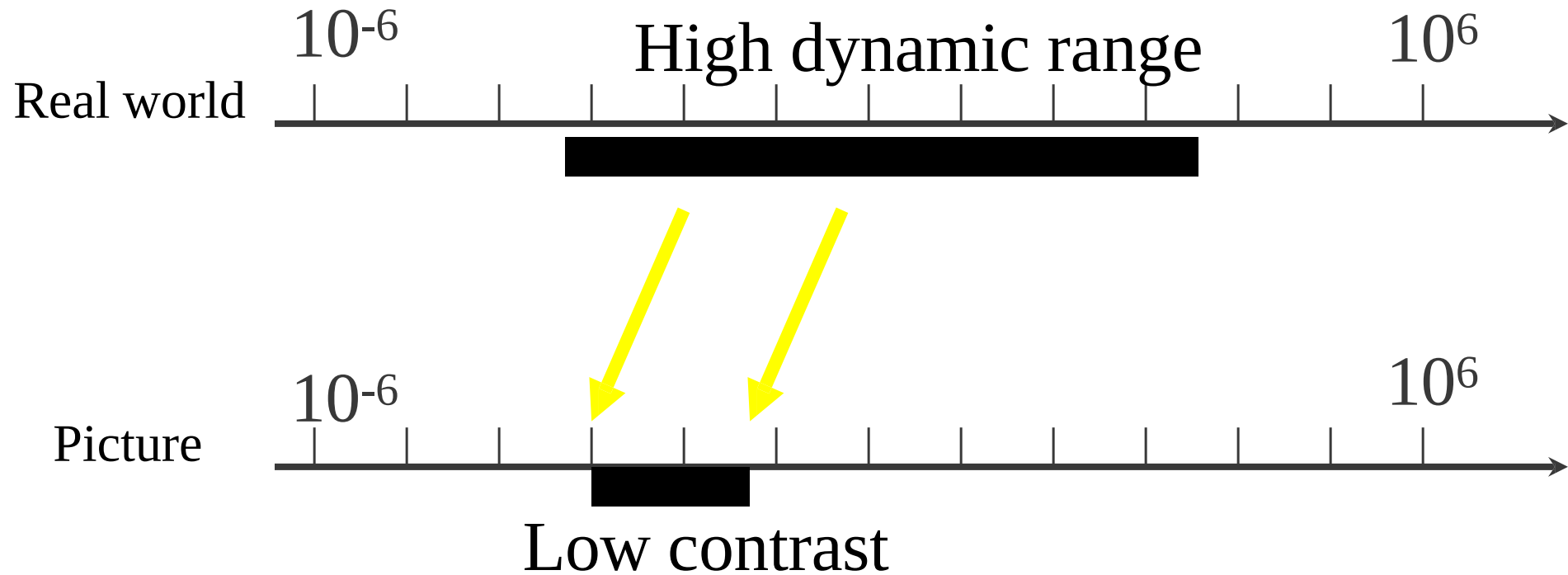
Multiple exposure photography

- **Sequentially measure all segments of the range**



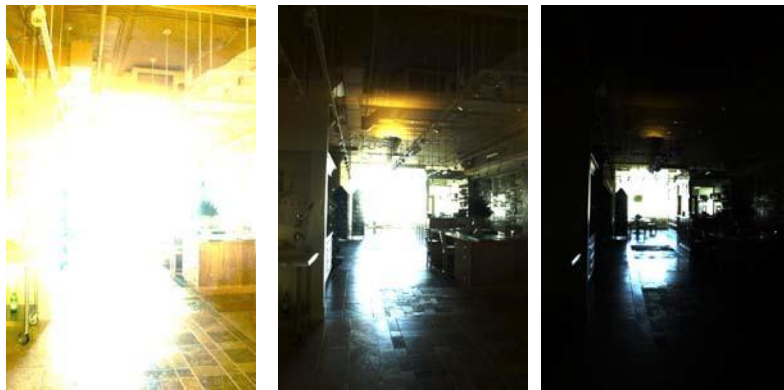
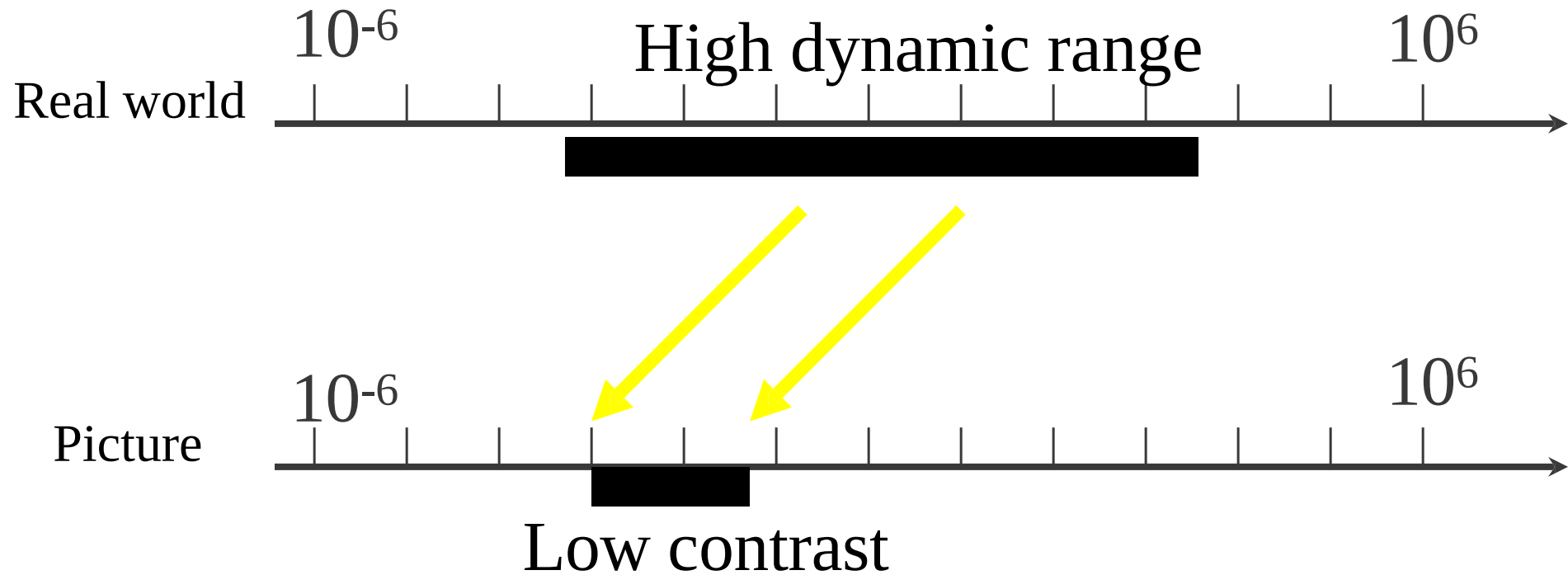
Multiple exposure photography

- **Sequentially measure all segments of the range**



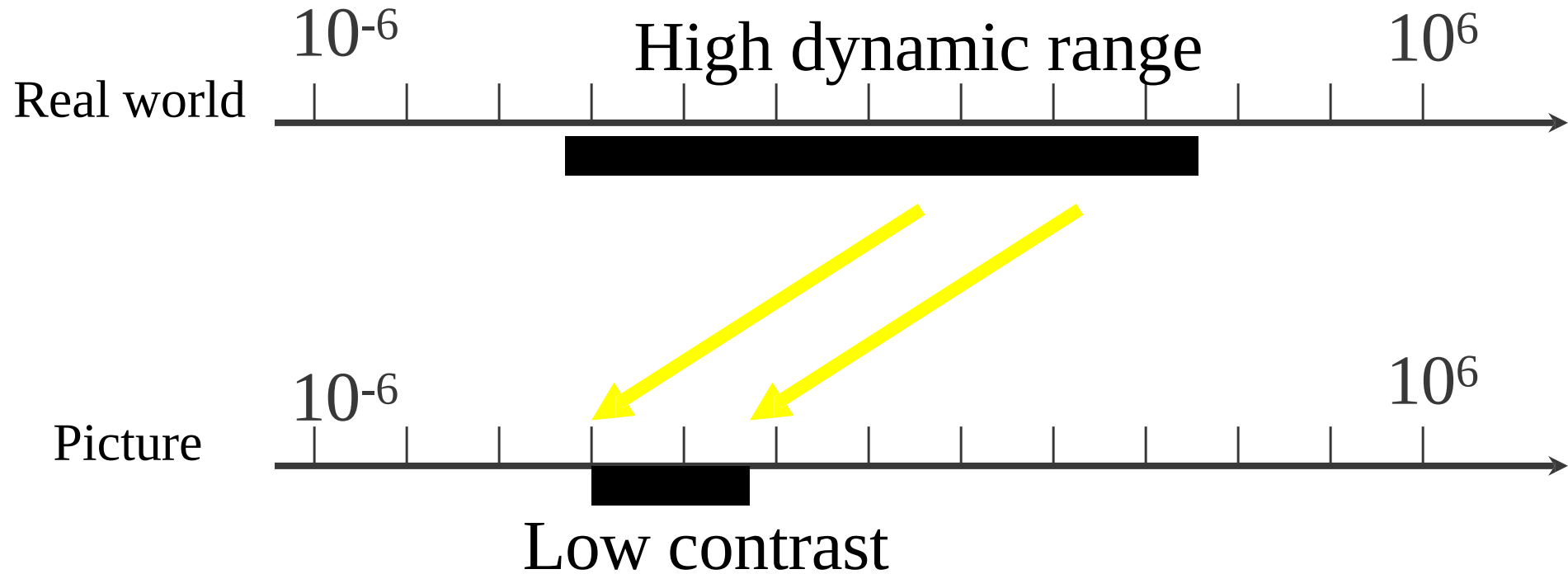
Multiple exposure photography

- **Sequentially measure all segments of the range**



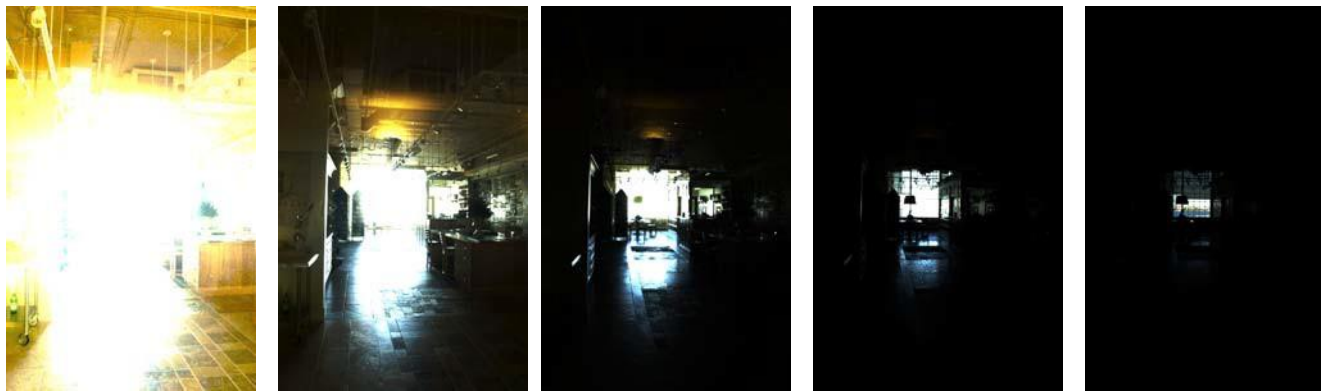
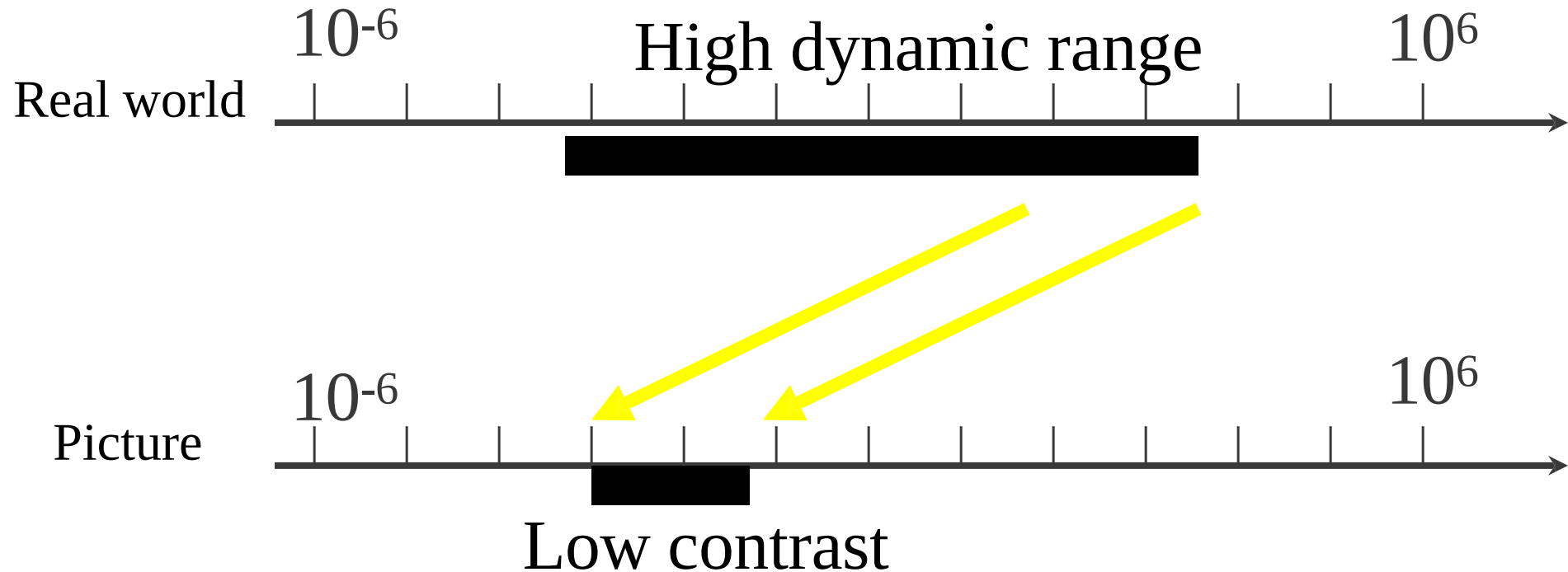
Multiple exposure photography

- **Sequentially measure all segments of the range**



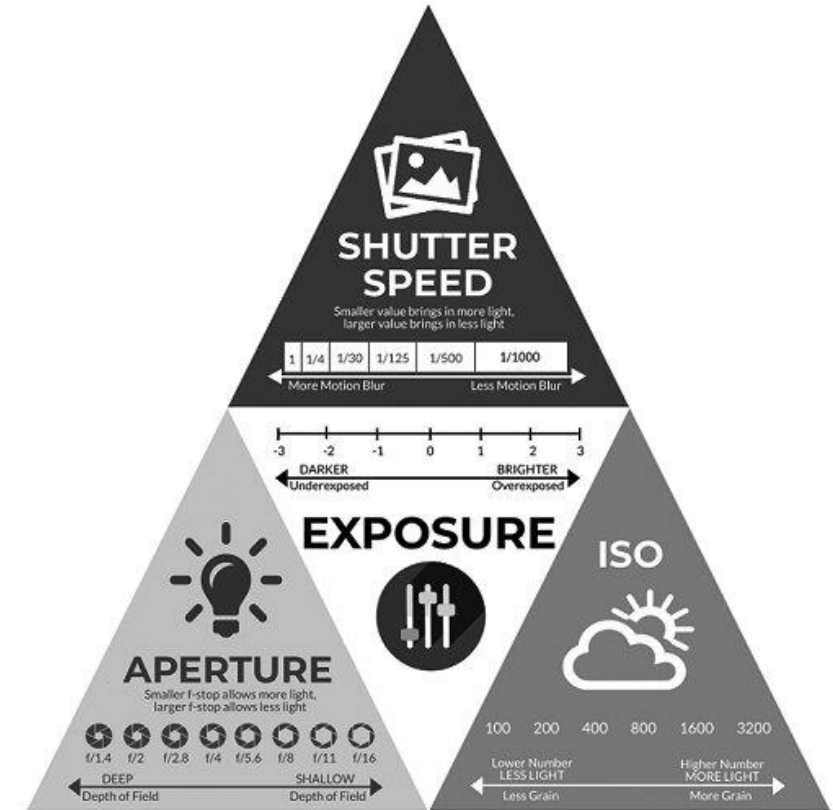
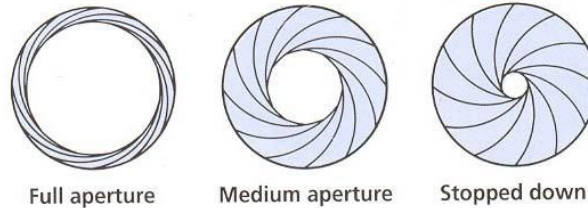
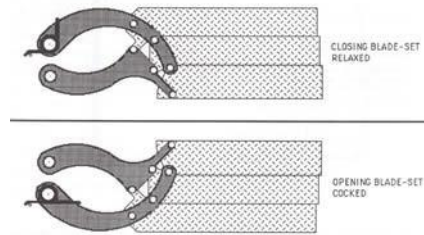
Multiple exposure photography

- **Sequentially measure all segments of the range**



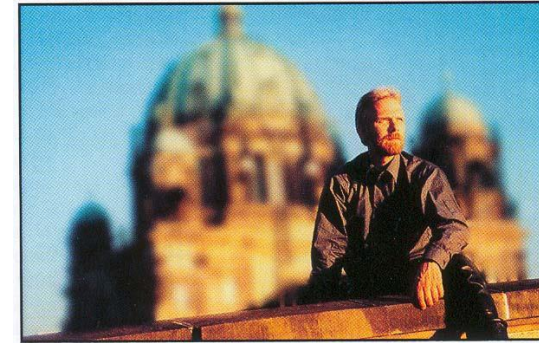
How do we vary exposure?

- Options:
 - Shutter speed
 - Aperture
 - Light Sensitivity (ISO)
 - [Neutral density filter]

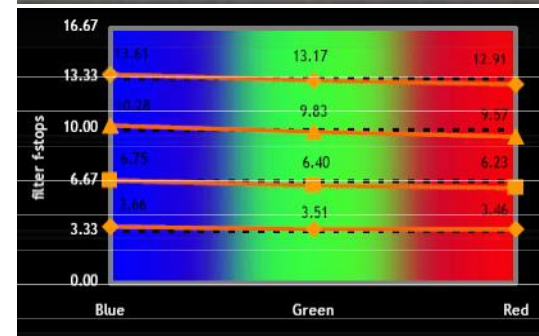


Tradeoffs

- **Shutter speed**
 - Range: ~ 30 sec to $1/4000$ sec (6 orders of magnitude)
 - Pros: reliable, linear
 - Cons: sometimes noise for long exposure
- **Aperture**
 - Range: $\sim f/1.4$ to $f/22$ (2.5 orders of magnitude)
 - Cons: changes depth of field
- **Light Sensitivity (ISO)**
 - Range: ~ 100 to 1600 (1.5 orders of magnitude)
 - Cons: noise
- **Neutral density filter**
 - Range: up to 4 densities (4 orders of magnitude) & can be stacked
 - Cons: not perfectly neutral (color shift), not very precise, need to touch camera (shake)
 - Pros: works with strobe/flash, good complement when desperate

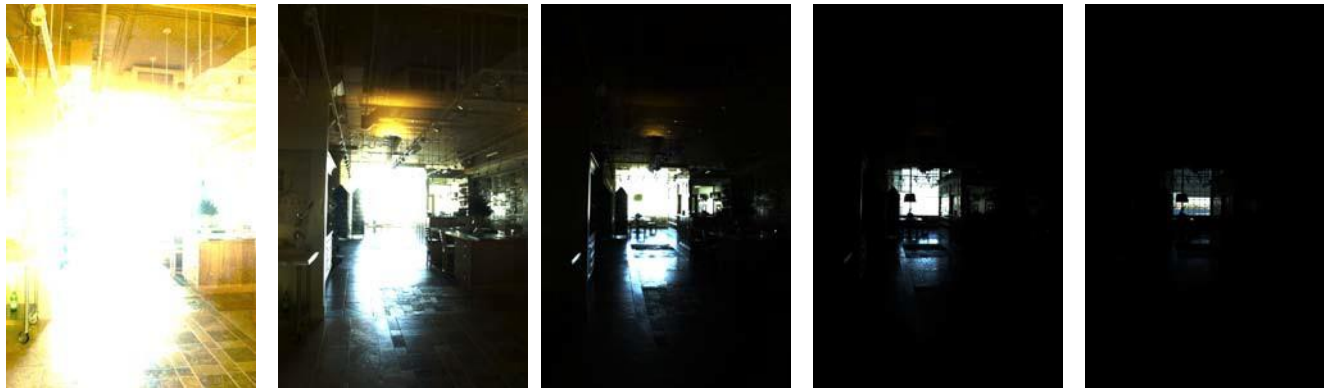


Nikon D2X
ISO 3200



HDR image using multiple exposure

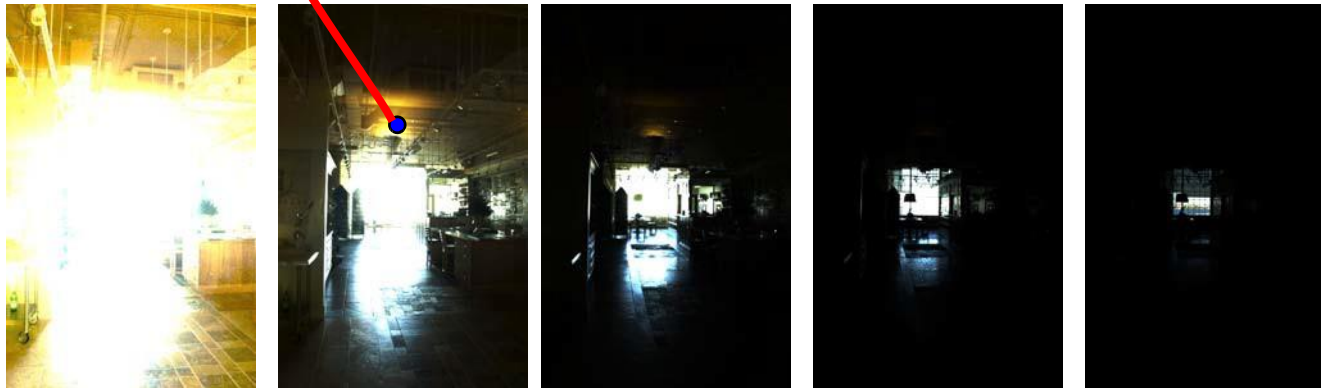
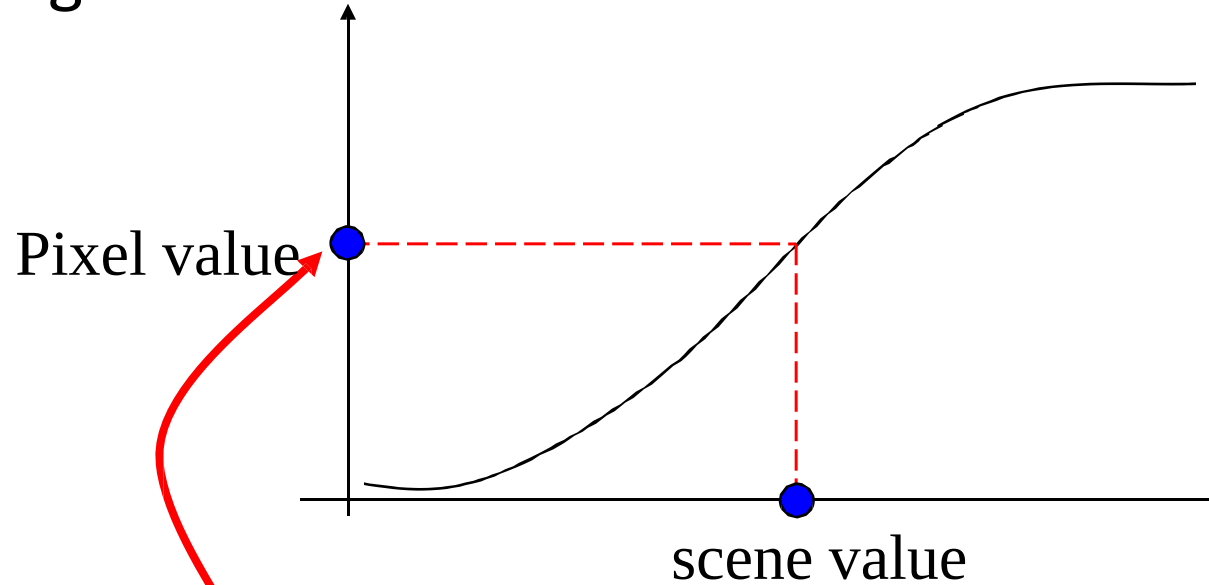
- Given N photos at different exposure
- Recover a HDR color for each pixel



If we know the response curve

- Just look up the inverse of the response curve
- But how do we get the curve?

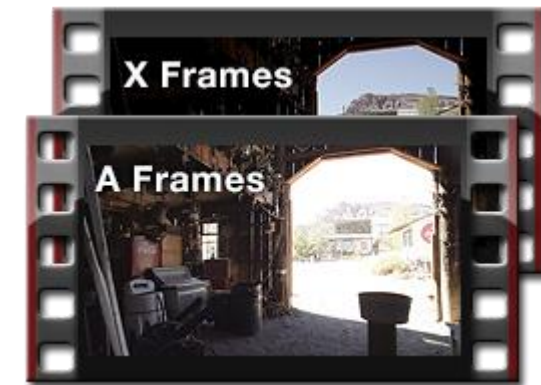
A response curve is the relation between the input and output of a sensor.



Calibrating the response curve

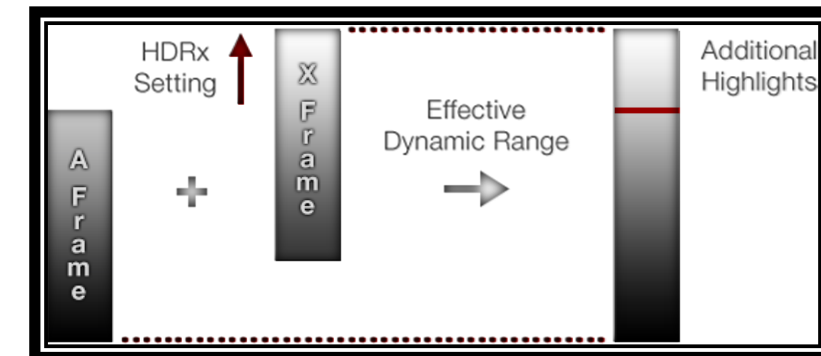
- **Two basic solutions**

- Vary scene luminance and see pixel values
 - Assumes we control and know scene luminance
- ***Vary exposure and see pixel value for one scene luminance***



- **Best of both:**

- **Vary exposure**
- Exploit the large number of pixels

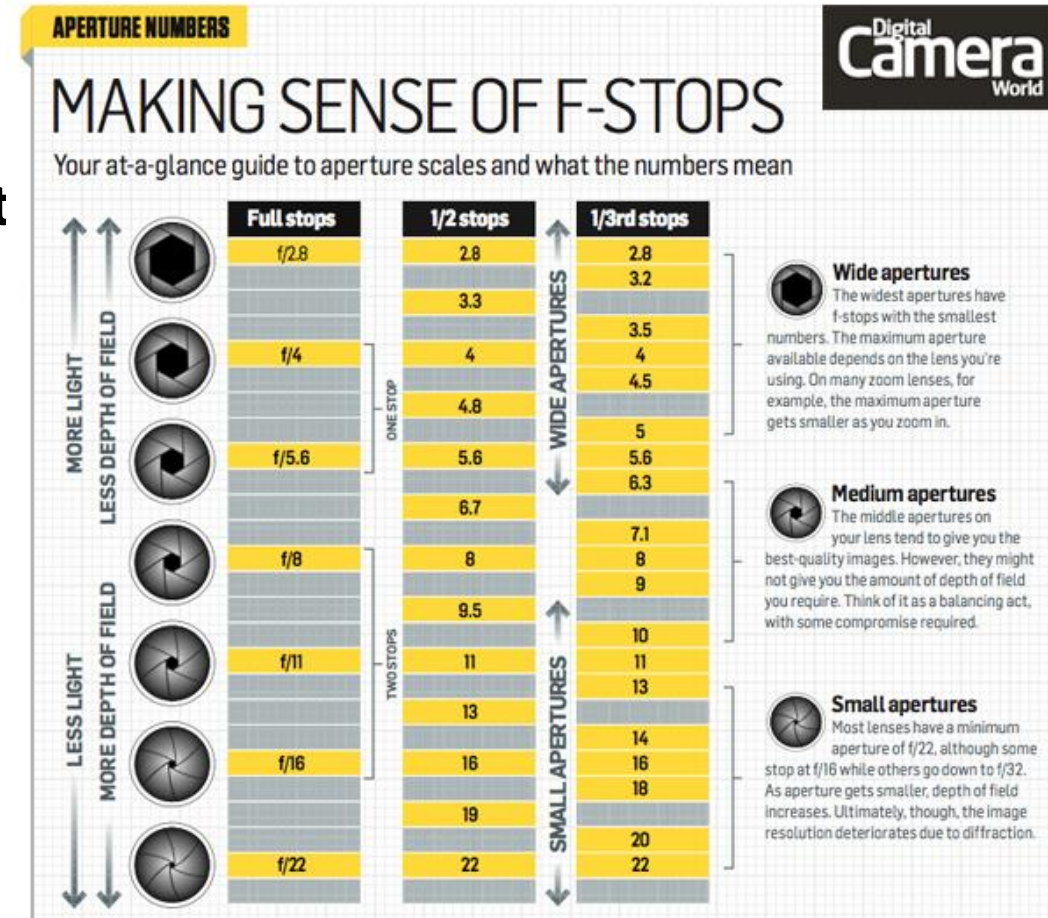


Explanation of the HDRx mode (RED film cameras) – 2 exposures captured:
<https://www.red.com/red-101/hdrx-high-dynamic-range-video>

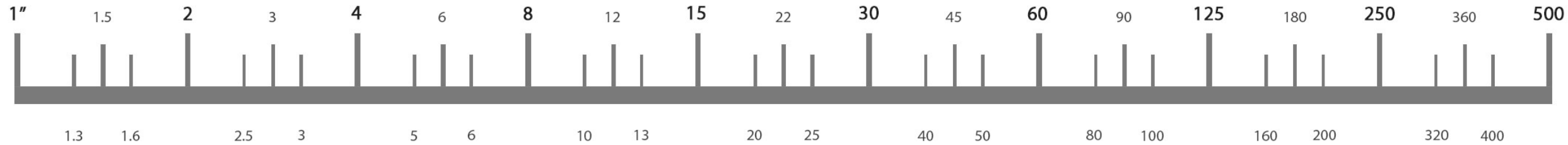
F-stop number (Aperture Stop)

- Each stop is **twice as bright as the previous one**
- Here's the aperture scale. Each step down lets in **half** as much light:
- These are the main aperture “stops,” but most cameras and lenses today let you set some values in between, such as f/1.8 or f/3.5.
- When a camera is said to have **12 stops of dynamic range**, it means that the **brightest part of a scene can be 2^{12} times brighter than the darkest part.**

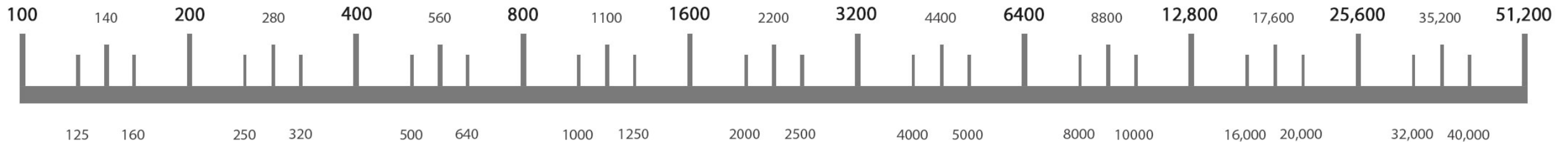
f/1.4	f/2.0	f/2.8	f/4.0	f/5.6	f/8.0	f/11.0	f/16.0	f/22.0
Very large aperture	Large aperture	Large aperture	Moderate aperture	Moderate aperture	Moderate aperture	Small aperture	Small aperture	Very small aperture
Lets in a huge amount of light	Half as much light	Half as much light	Half as much light	Half as much light (a very “medium” aperture)	Half as much light	Half as much light	Half as much light	Half as much light (by which point your photos are very dark)
Very thin depth of field	Thin depth of field	Thin depth of field	Moderately thin depth of field	Moderate depth of field	Moderately large depth of field	Large depth of field	Large depth of field	Very large depth of field



Shutter Speed & ISO Stops



A shutter speed range of 10 stops with values from one second all the way to 1/500th of a second



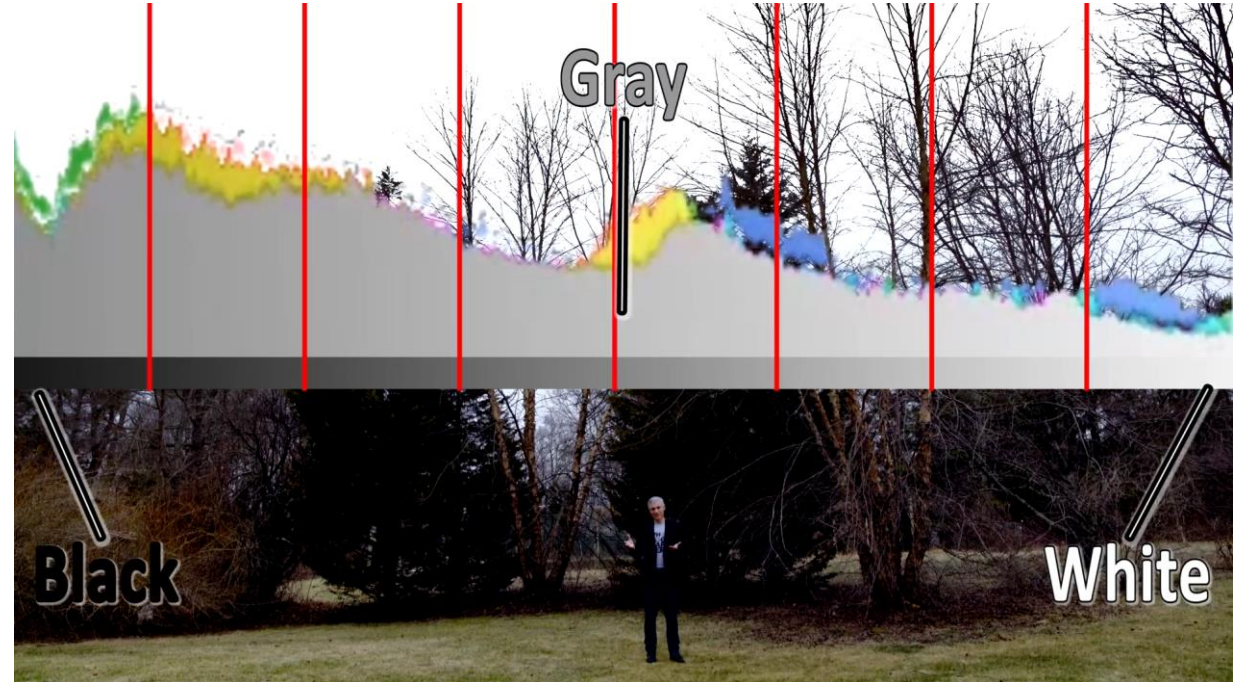
Calculating ISO sensitivity stops is very easy – one just needs to divide or multiply the value by 2 to add a stop or go one stop down.

PLEASE NOTE: A digital sensor does not and cannot change its sensitivity to light. There is no setting whatsoever on a digital camera, including ISO, that can modify a sensor's sensitivity to light. ISO in digital is a level of post-capture analog amplification that occurs.

Dynamic Range Explanations & Examples



<https://www.youtube.com/watch?v=SrXwiPH-fCg>



<https://www.youtube.com/watch?v=MfvwqcmM6FQ>

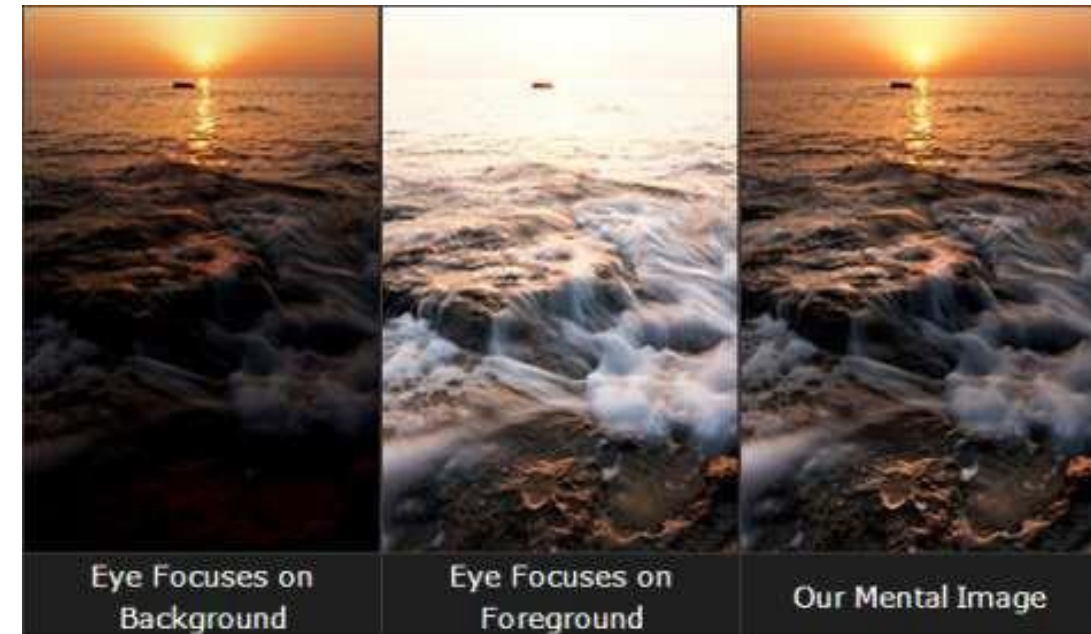
Very good detailed explanation of a film maker about Dynamic Range:

<https://www.youtube.com/watch?v=2sshGdMgJxQ>

The Human Eye



- Human eye uses the pupil size to see darker or brighter images
 - Turn smaller to limit incoming light to see brighter scenes
 - But cannot see darker & brighter region at same time
 - Can detect a luminance range of 10^{14} , or one hundred trillion (100,000,000,000,000) (**about 46.5 f-stops**), from 10^{-6} cd/m², or one millionth (0.000001 cd/m²) to 10^8 cd/m² or one hundred million (100,000,000 cd/m²)



Stops - Contrast

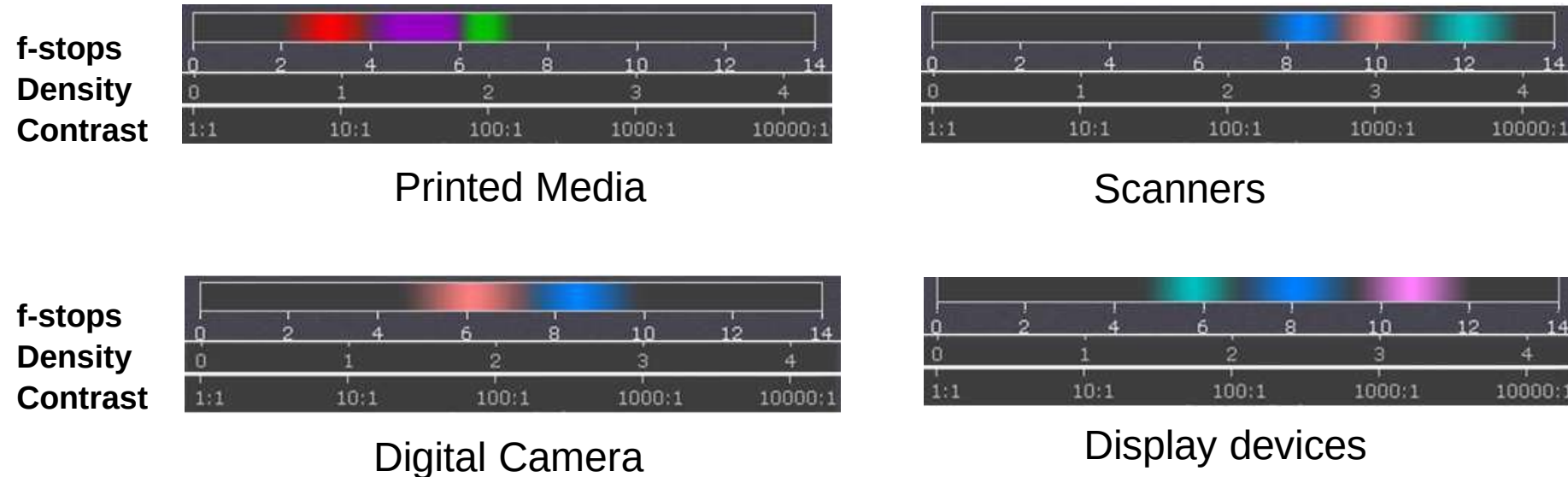
Dynamic ranges of common devices

Device	Stops	Contrast ratio
Glossy photograph paper	7 (7 - 7 2/3) ^[58]	128:1
LCD	9.5 (8 – 10.8) ^[citation needed]	700:1 (250:1 – 1750:1)
Negative film (Kodak VISION3)	13 ^[59]	8000:1
Human eye	10–14 ^[54]	1000:1 – 16000:1
High-end DSLR camera (Nikon D850)	14.8 ^[60]	28500:1
Digital cinema camera (Red Weapon 8k)	16.5+ ^[61]	92000:1

Without adaptation

Contrast Comparison

- Dynamic range is commonly measured on a logarithmic scale



Optische Dichte d (Density): dekadischer Logarithmus der Lichtleistung

Bit Depth & Dynamic Range

- (bit per pixel) Quantifies how many unique colors are available
 - How many unique shades are available in grayscale image

Bit Precision of Analog/Digital Converter	Contrast Ratio	Dynamic Range	
		f-stops	Density
8	256:1	8	2.4
10	1024:1	10	3.0
12	4096:1	12	3.6
14	16384:1	14	4.2
16	65536:1	16	4.8

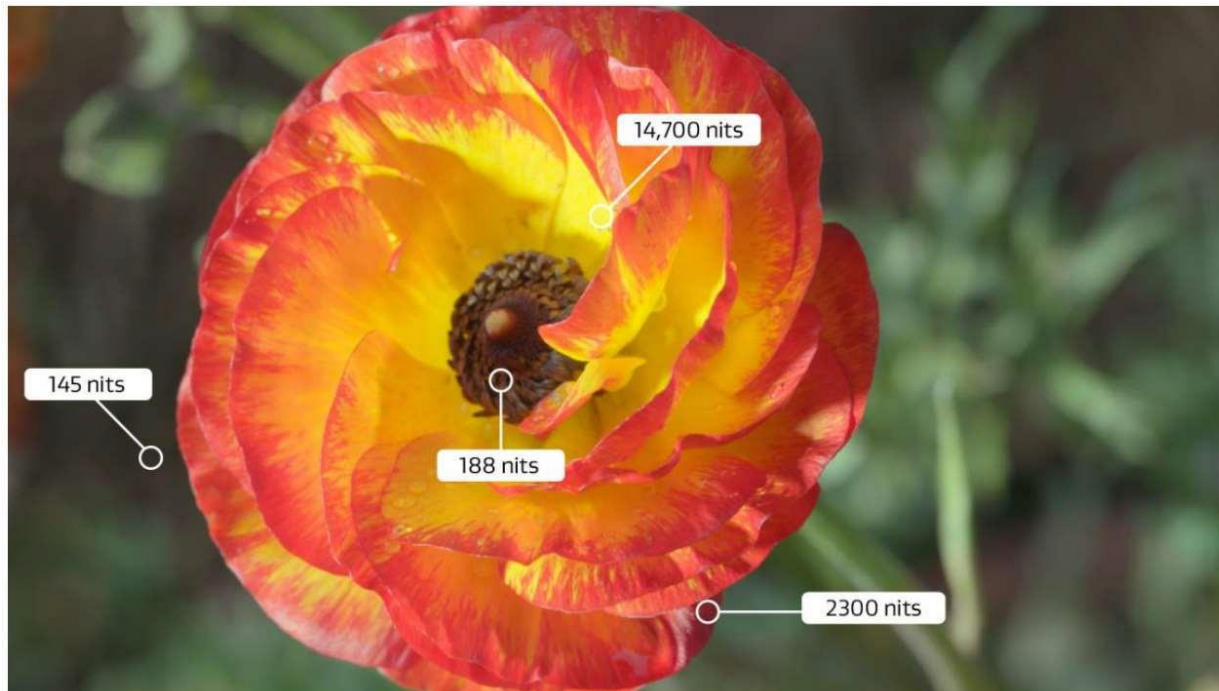
- Most digital cameras use a 10 – 14 bit A/D so their theoretical maximum dynamic range is 10-14 f-stops.
- Higher precision A/D converter does not necessarily mean greater dynamic range. Total dynamic range is usually limited by noise levels.

Measuring Luminance



Nit (Candela/m²)

- 1 nit (cd/m²) approximately equal to the amount of light from a candle dispersed over a square meter



The real world is high dynamic range

Object	Approximate Luminance (nits)
Sun	1,600,000,000
Arc lamp	150,000,000
Maximum visual tolerance	50,000
Cloud (sunny day)	35,000
2020 UHD-TV	800-1000
Typical computer screen	100-300
White paper under lamp	50
Night sky	0.001
Threshold of vision	0.000003



500,000
nits



200,000
nits



40
nits



2,000 nits









HDR image examples



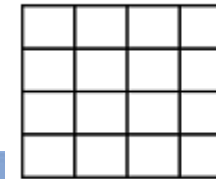
Moving to HDR

- **UHD Alliance: with mission to create (Jan. 2015 at CES)**
 - *New standard to support video technologies including **4K**, Higher resolution, **high dynamic range**, **wider color gamut**, and **3D audio***

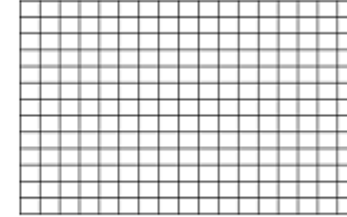


Improving Image Quality

More pixels: SD – HD –UHD (4k, 8K)



Low resolution



High resolution

SDTV
HDTV
UHDTV



UHD vs. HD. vs SD resolution comparison

LEDTVReviews.com

Increase spatial resolution

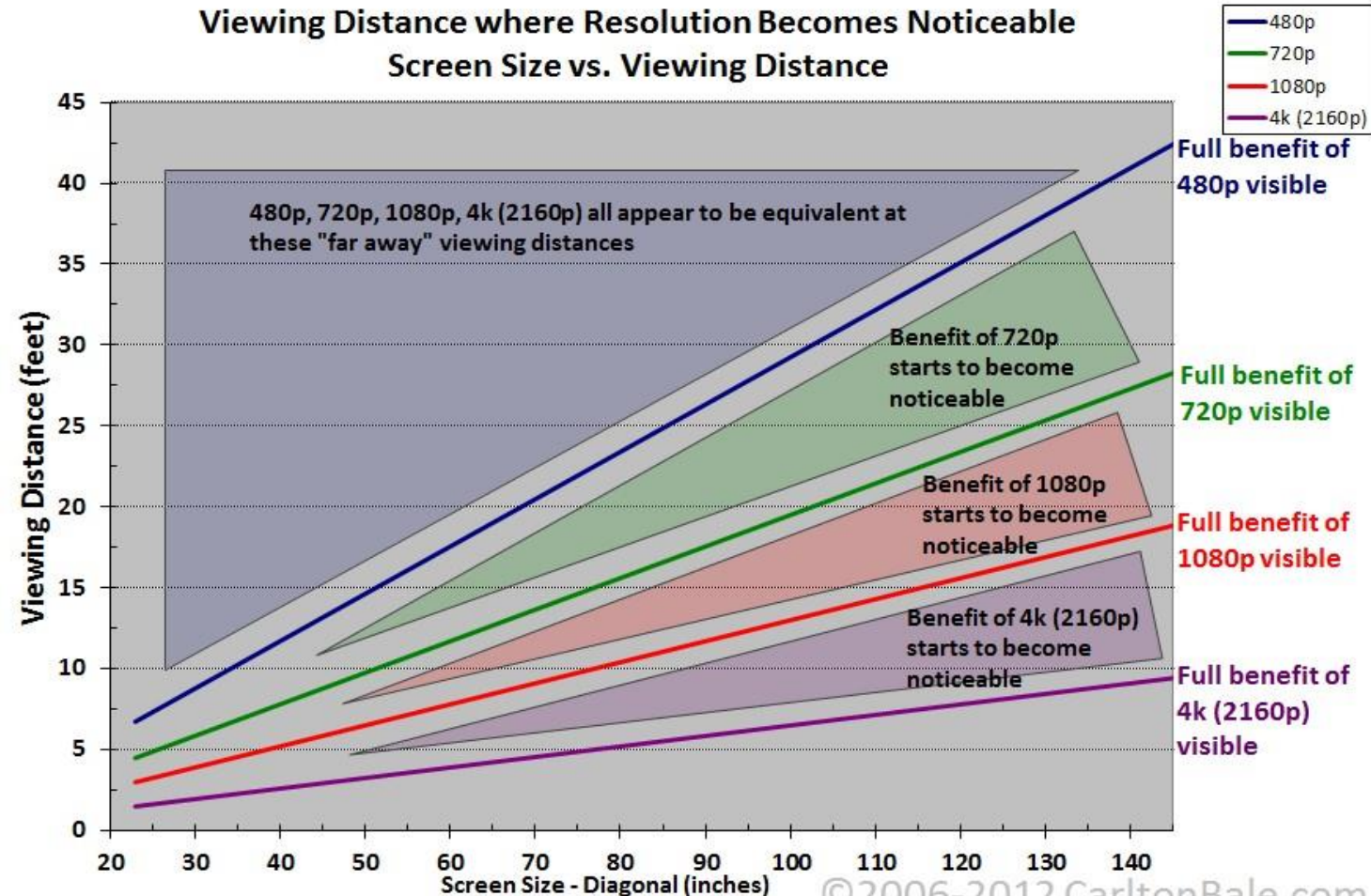
Moving to HDR

**Dramatic better
visual quality**

- **(Netflix) HDR is more important for video quality than 4K**
 - “We kind of ran out of more pixels to add,” Neil Hunt

Resolution is only apparent
when you are close

**Users can notice HDR from
any distance and screen size,
at once**

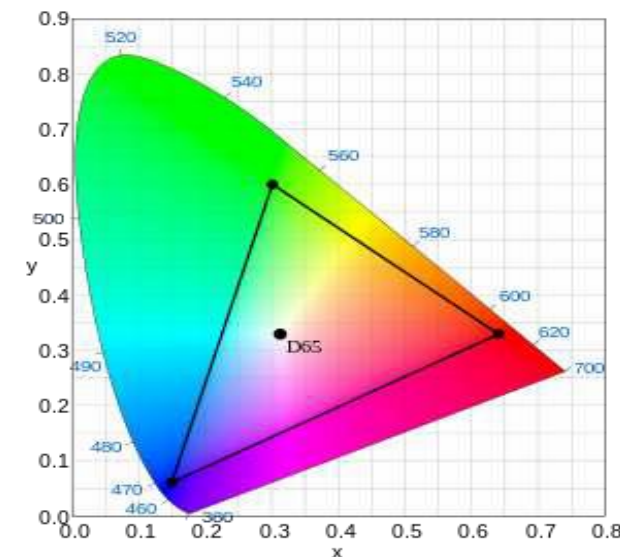


High Dynamic Range

- Need around 10.000 nits to satisfy 90% viewer at ideal range



Current Color Space (Color Gamut)
is not enough

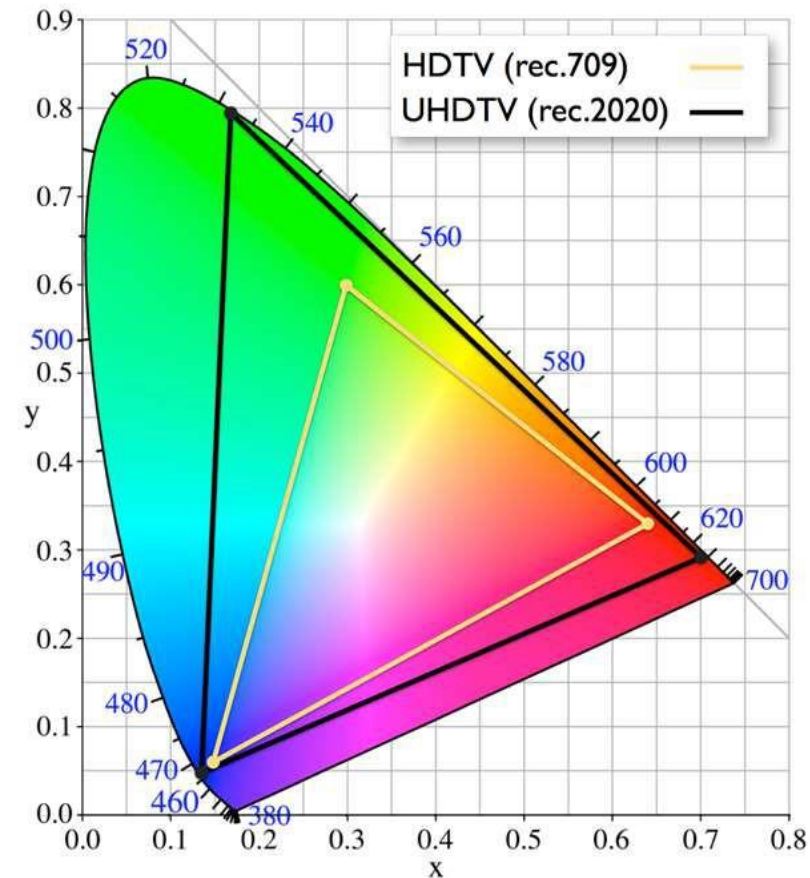


Wider Color Gamut

- Current color gamut is limited
 - Current HDTV (BT.709) – 1990
 - Created for CRT, and HDTV
 - Around 100 nits.
 - 8 bpp (bits per pixel)

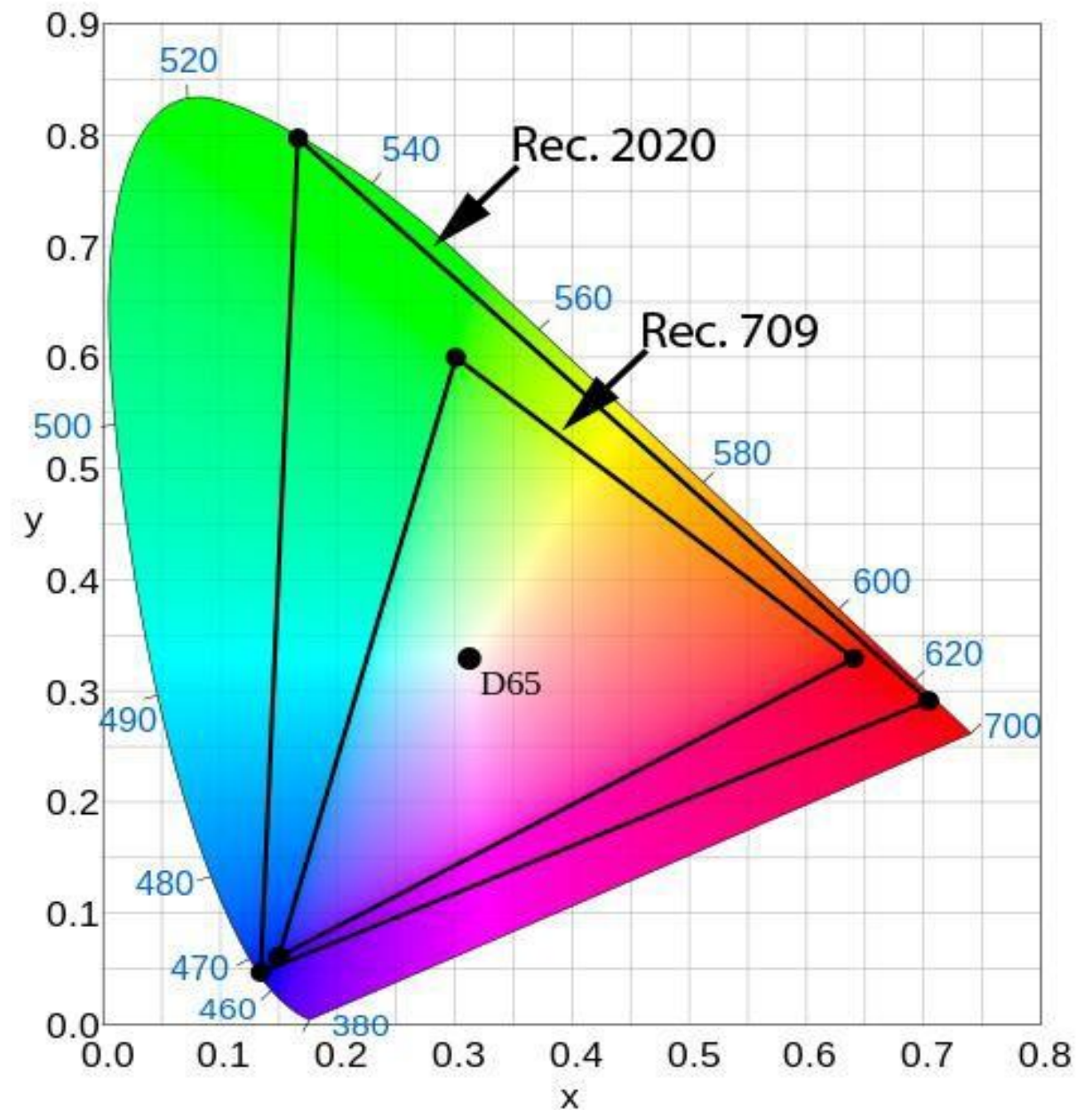
Today we have better display technology: LCD, OLED ...

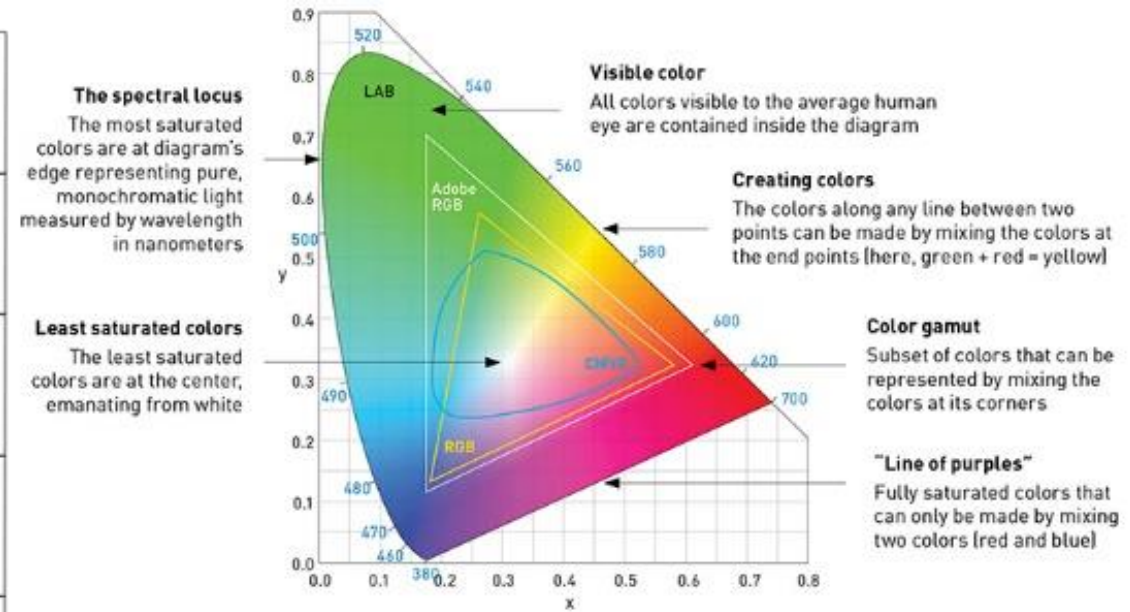
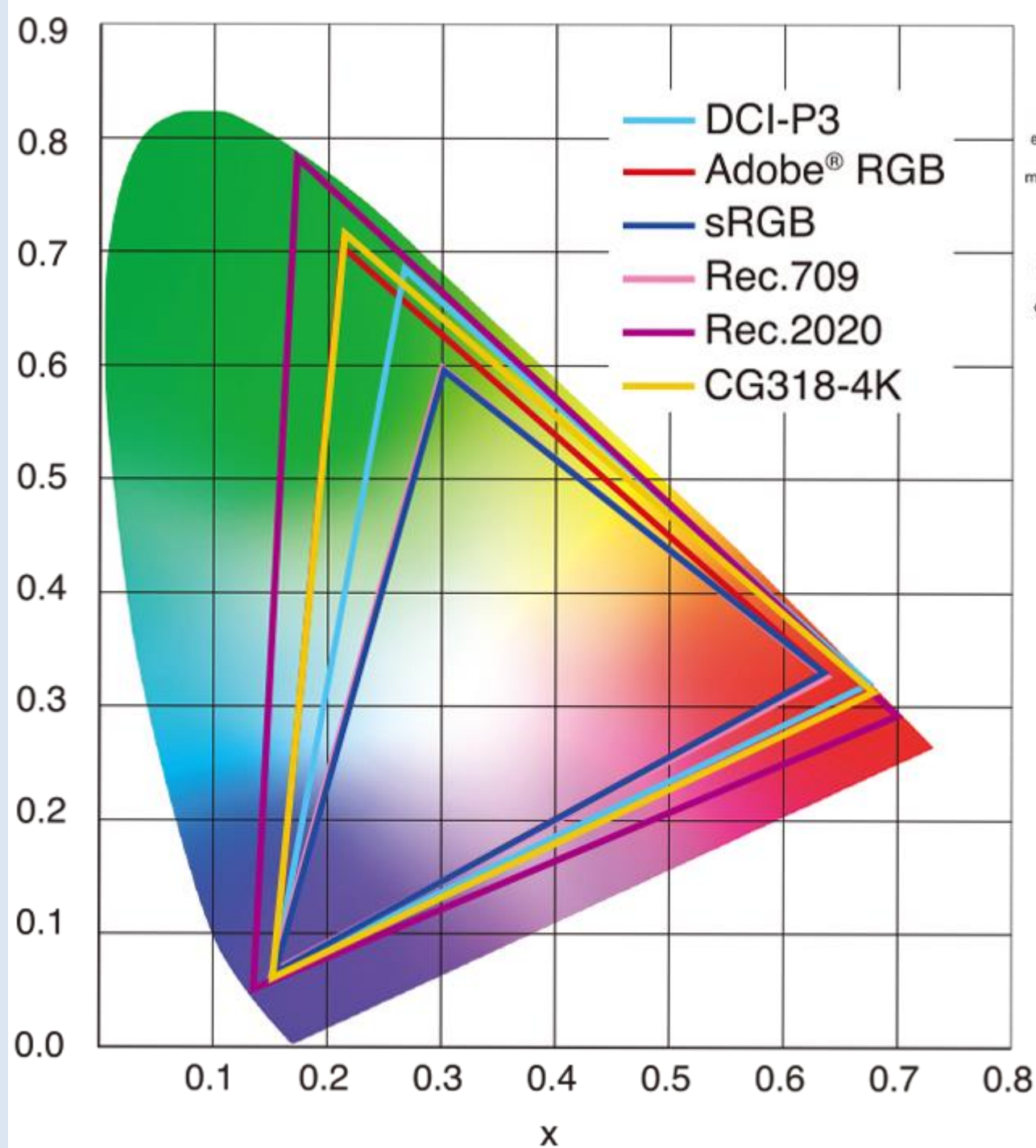
More Colors: REC. 2020 for UHDTV 10, 12 bpp.



UHDTV vs HDTV Color Gamuts

- Most PCs and HDTVs use 709 gamut
- Majority of next generation content uses Bt.2020 gamut
- 2015 UHD TVs are ~85% Bt.2020





Color profiles of video and monitors

**Bt 2020 content
displayed unmodified
on 709 display**



**Bt 2020 content
gamut-mapped for
709 display**



We must perform gamut mapping



Standard Dynamic Range (SDR)

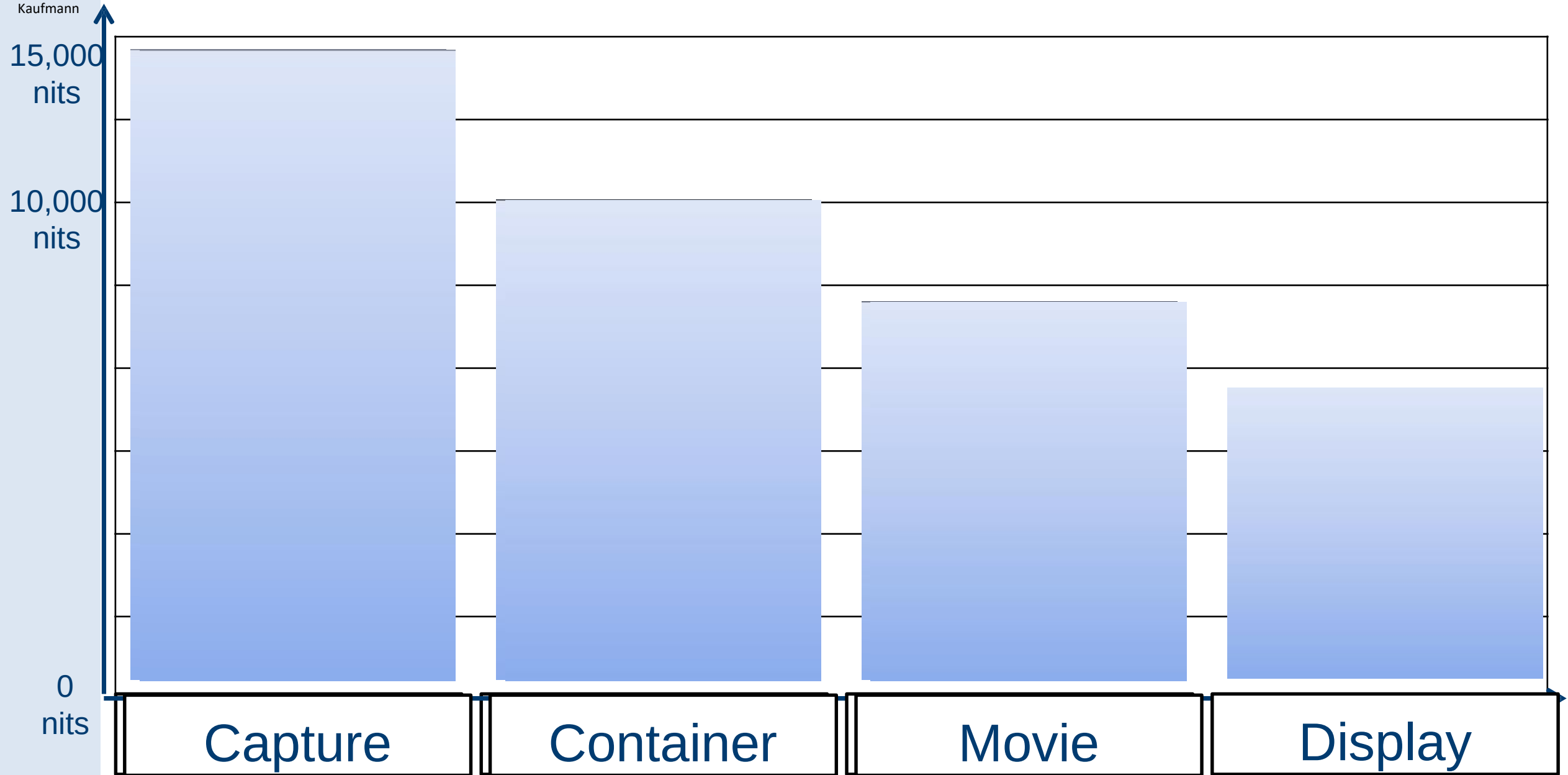
- For the past 80 years, video has been graded to appear properly on a **100 nit** display
- However, no adjustments are made to comprehend displays of differing luminance
 - 30 nit laptops in low power mode
 - 600 nit HDTVs in Vivid mode
- Similarly, no adjustments to the content are made when you surf the web and play games, switching from phones to tablets to PCs.

This is **wrong**, but has been **good enough**

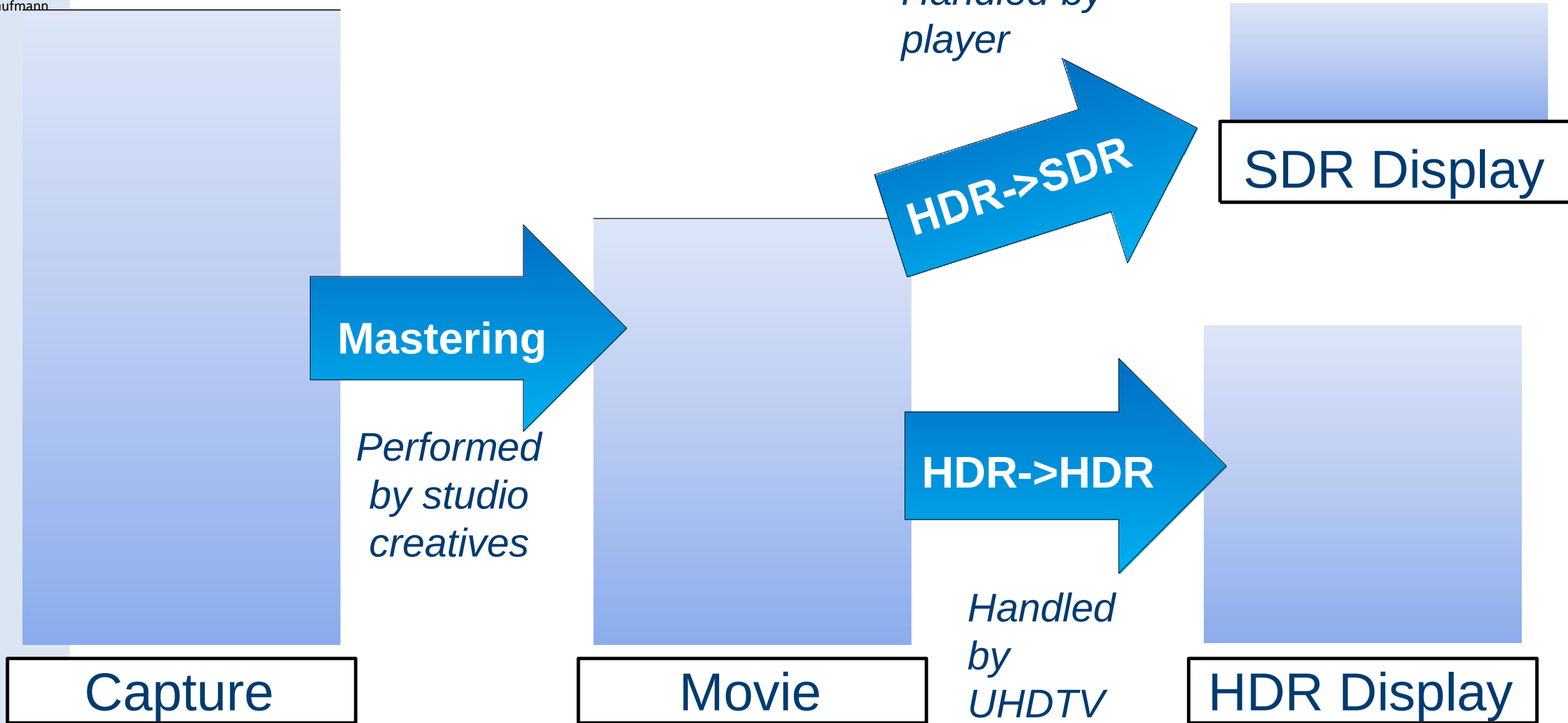
Things to ponder

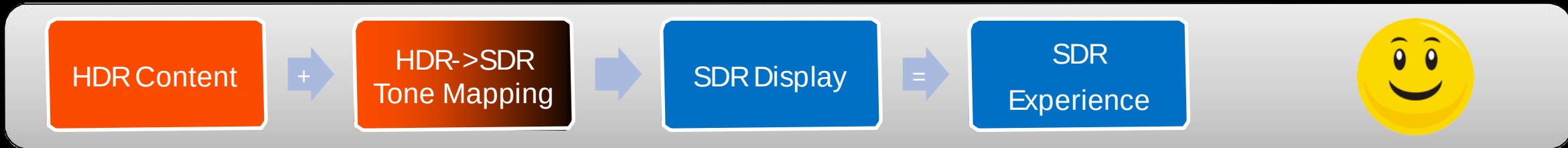
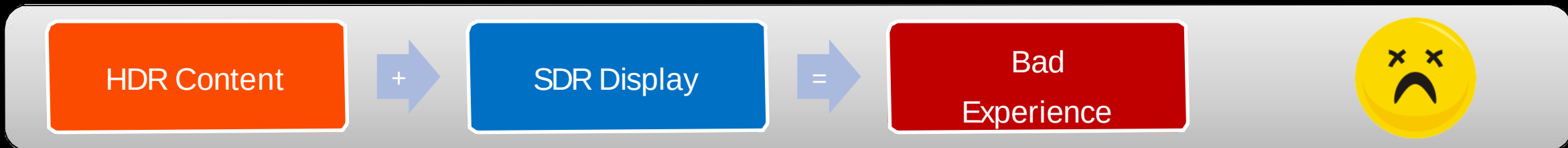
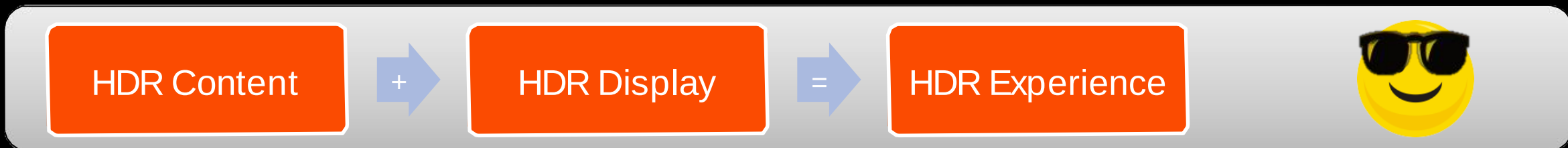
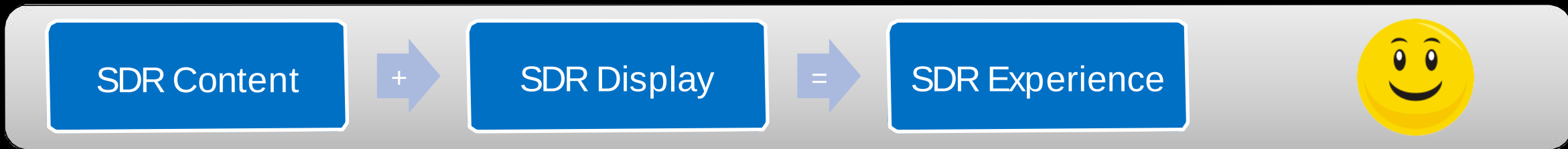
- With color, many real-world hues can be exactly reproduced on screens
- Almost every real-world situation has objects at >100 nits
- So the real world is HDR, and every SDR picture you take involves HDR- $\rightarrow$ SDR conversion
- A 100-nit object on the screen almost never corresponds to a 100-nit object in the real world

Multiple Ranges of HDR



HDR Conversions

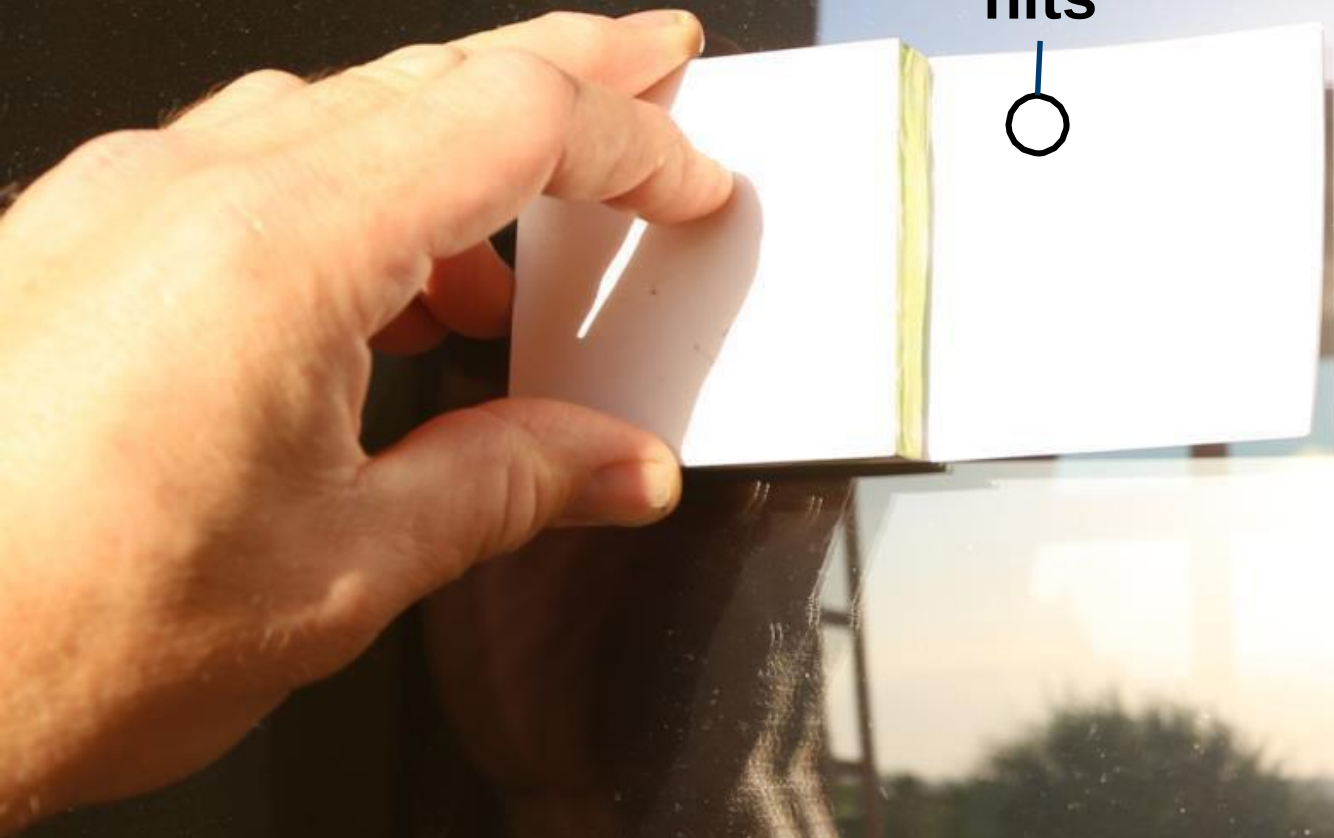




Real World

**3,000
nits**

**300,000
nits**



Mastered for SDR

**100
nits**



**Coded
value=
0xFF**

**100
nits**



**Coded
value=
0xFF**

Mastered for HDR

200
nits



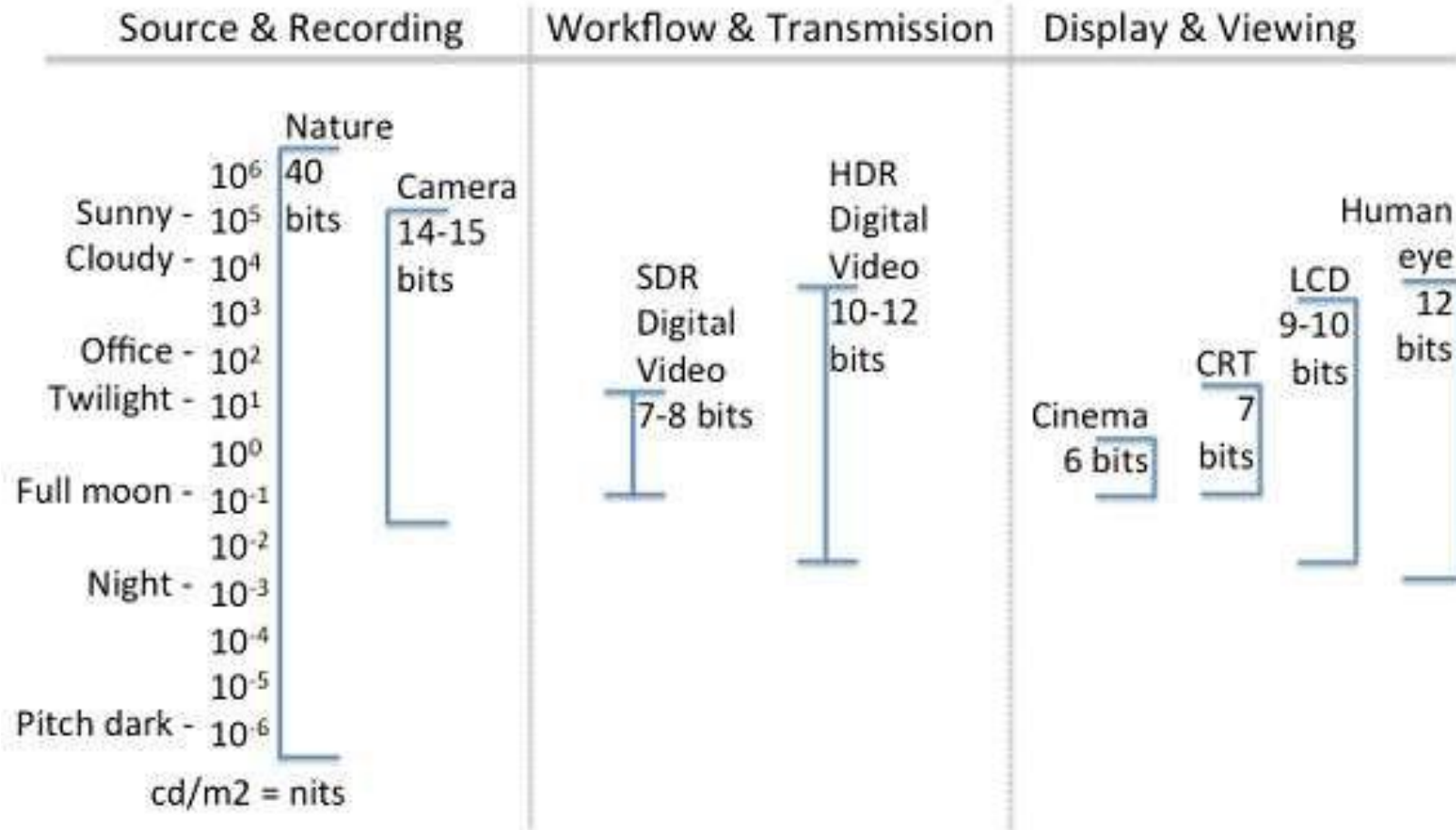
Coded
value=
0x23B

5,000
nits



Coded
value=
0x36C

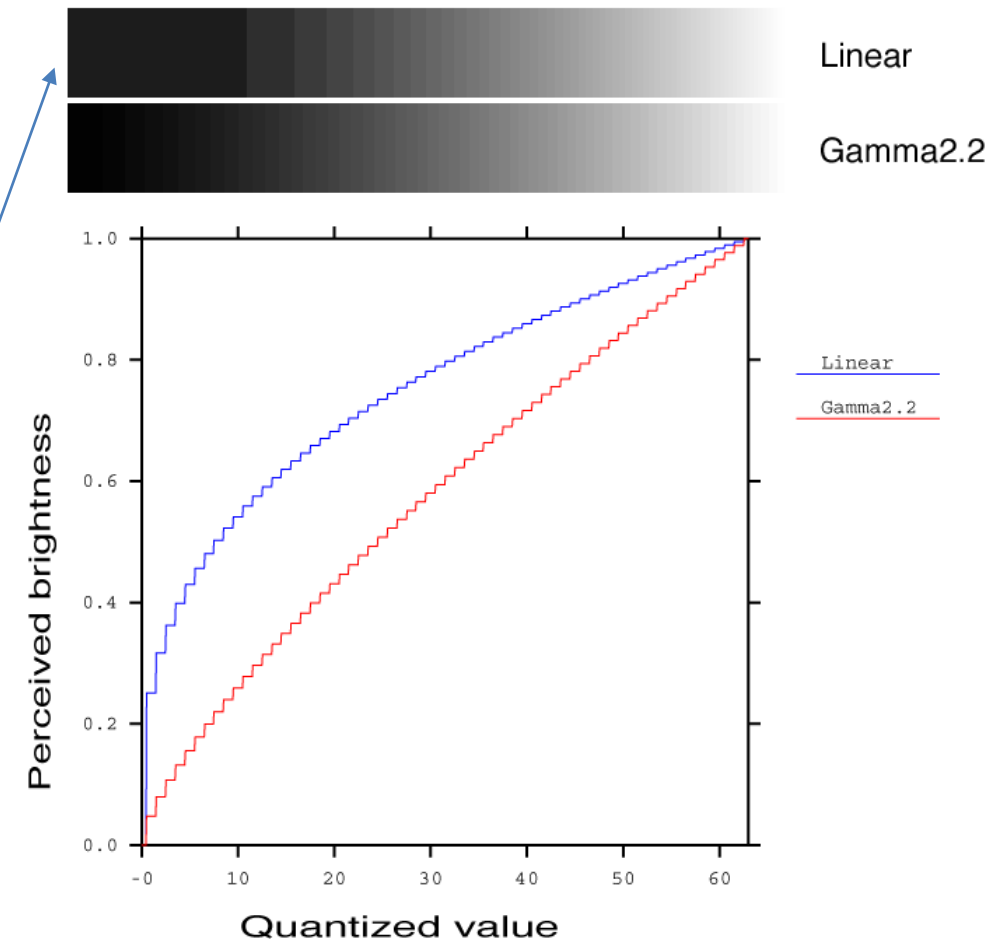
Color and Brightness



Luminance dynamic range for various technologies

Quantization: Linear vs non-linear light

- In the real world, luminance is determined by the number of photons
- However, brightness (the human perception of luminance) is non-linearly proportional to number of photons
 - The eye has a non-linear response to intensity. At most adaptation levels, we perceive brightness roughly as the cube root of intensity.
- If we apply a linear quantization of color values, we see more steps in darker regions than we do in the brighter regions
- Using a power law encoding with a g value of 2.2, we see a much more even distribution of quantization steps, though the behavior near black is still not ideal

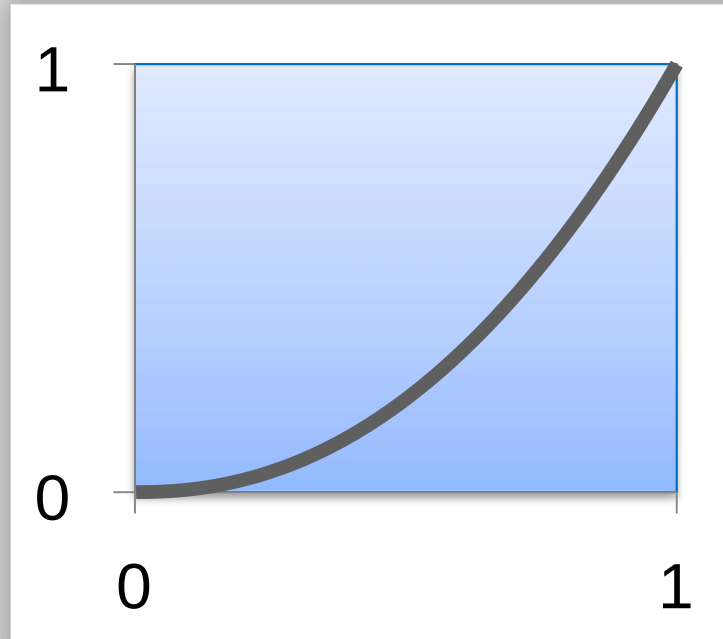
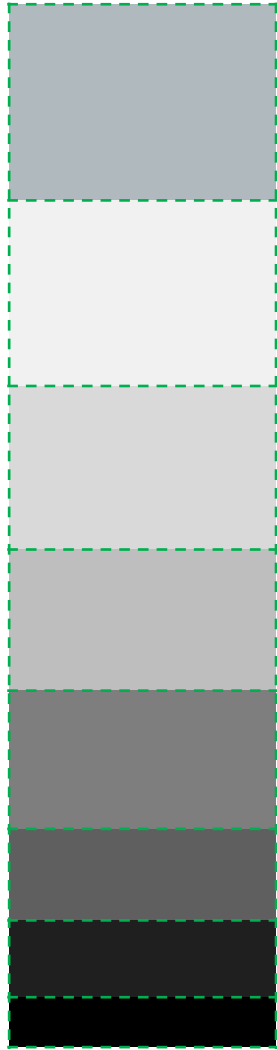


To efficiently code video signals, we apply a non-linear transform function so that we can code perceptually uniform brightness intervals

Luminance

Linear Light

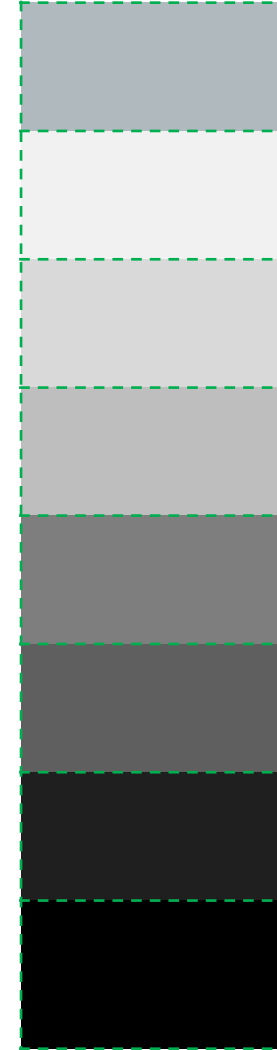
Perceptually
non-uniform



Luma

Non-Linear Light

Perceptually
Uniform



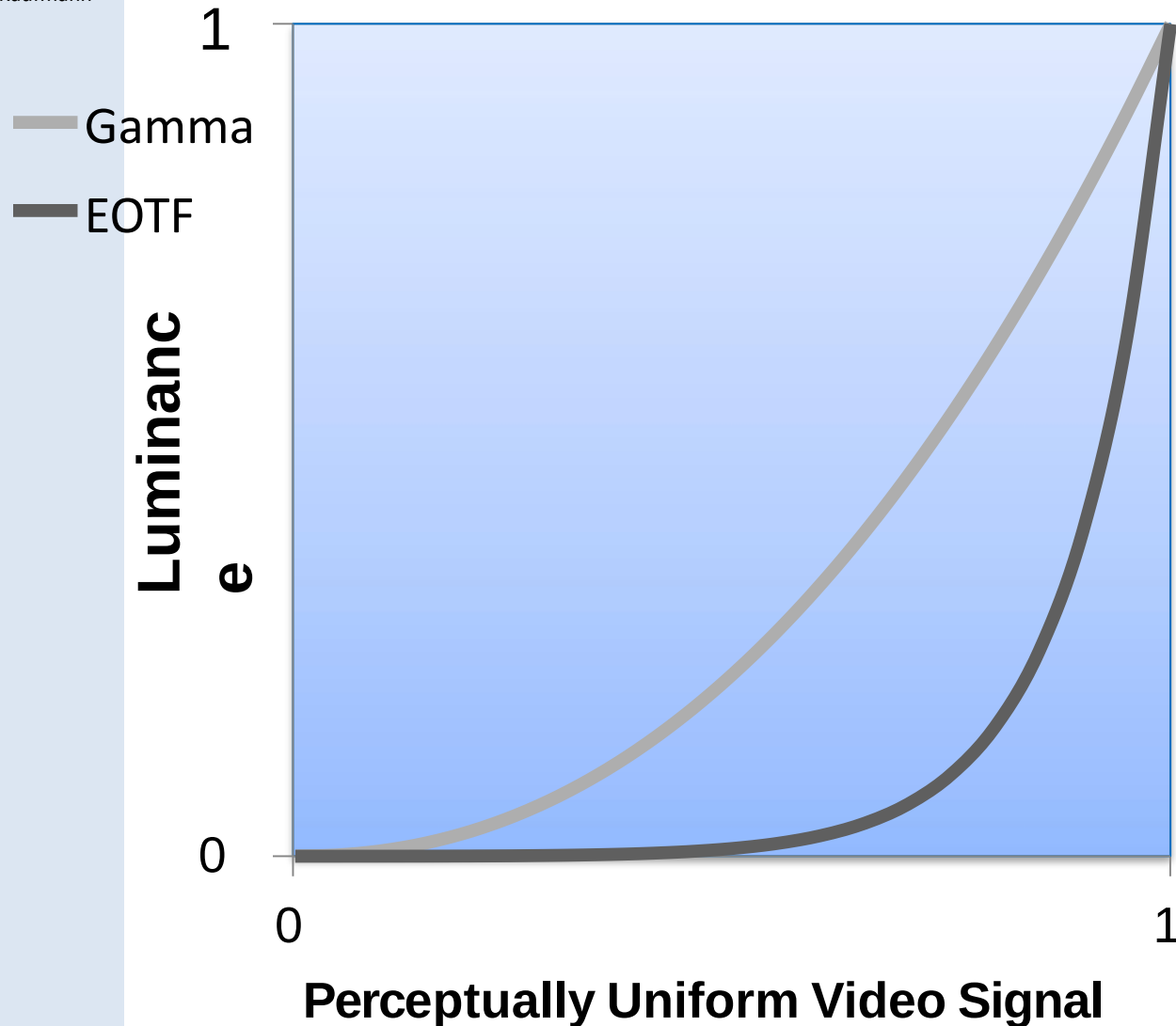
Problems

- Although gamma is a reasonable approximation of human perception in the SDR (0-100 nit) range...
- Gamma is not a good match for human perception in the 0-10,000 nit HDR
- If you use gamma for a 10,000 nit signal, you need to use two extra bits/sample to eliminate banding

Color banding



SMPTE 2084: HDR Electro-Optical Transfer Function (EOTF)



Electro-Optical Transfer Function (EOTF) is a correction function that **converts electrical voltages into brightness values**.

- If gamma curve is used = gamma correction, if more complex dependencies between voltage and brightness, an EOTF is used
- EOTF is used to adjust the brightness of displays or televisions to the human perception of brightness. Since this does not run linearly with voltage values, the display representation must be adapted.
- **Dependencies between brightness and voltage are different for various display technologies** and resolutions, such as SDTV, HDTV, UHDTV, HDR10 and Dolby Vision.
- Therefore several correction curves in addition to gamma correction: Perceptual Quantization (PQ) and Hybrid Log Gamma (HLG), which is described in the SMPTE's ST.2084 standard for video transmission of High Dynamic Range (HDR).

Hybrid Log Gamma (HLG) & Perceptual Quantization

- HLG correction is comparable to gamma correction in classic television.
- Hybrid Log Gamma (HLG) and Perceptual Quantization (PQ)
 - standardized by the International Telecommunication Union (ITU) in BT.2100
 - adapting the dynamic range of the display on the UHD displays to the dynamic range of the recording devices.
 - Why: So that display corresponds to the viewer's perception of brightness.
 - Correspond to a logarithmic function
 - The brightness range corresponds to a luminance between 100 nits and 5,000 nits.
 - HLG correction curve requires a resolution of at least 10 bits and is used with High Dynamic Range HDR10 and Dolby Vision in transmission and mastering.
 - When transmitting video in SDR technology and HDR technology simultaneously in one data stream, the image content is encoded with the HLG curve so that it can be played back on both SDR and HDR displays.

What is HDR (in this context)

- Content with a wider range of brightness and color
- Requires new monitors to experience HDR content
 - Legacy content (SDR): 8-bit color, 100 nit peak brightness, 709 gamut
 - Dynamic range of about 6 stops
 - HDR content: 10-bit color, 10,000 nit peak brightness, 2020 gamut
 - Dynamic range of about 17,6 stops

HDR is an ambiguous term

- There are several industry specs defining various flavors of HDR

Summary

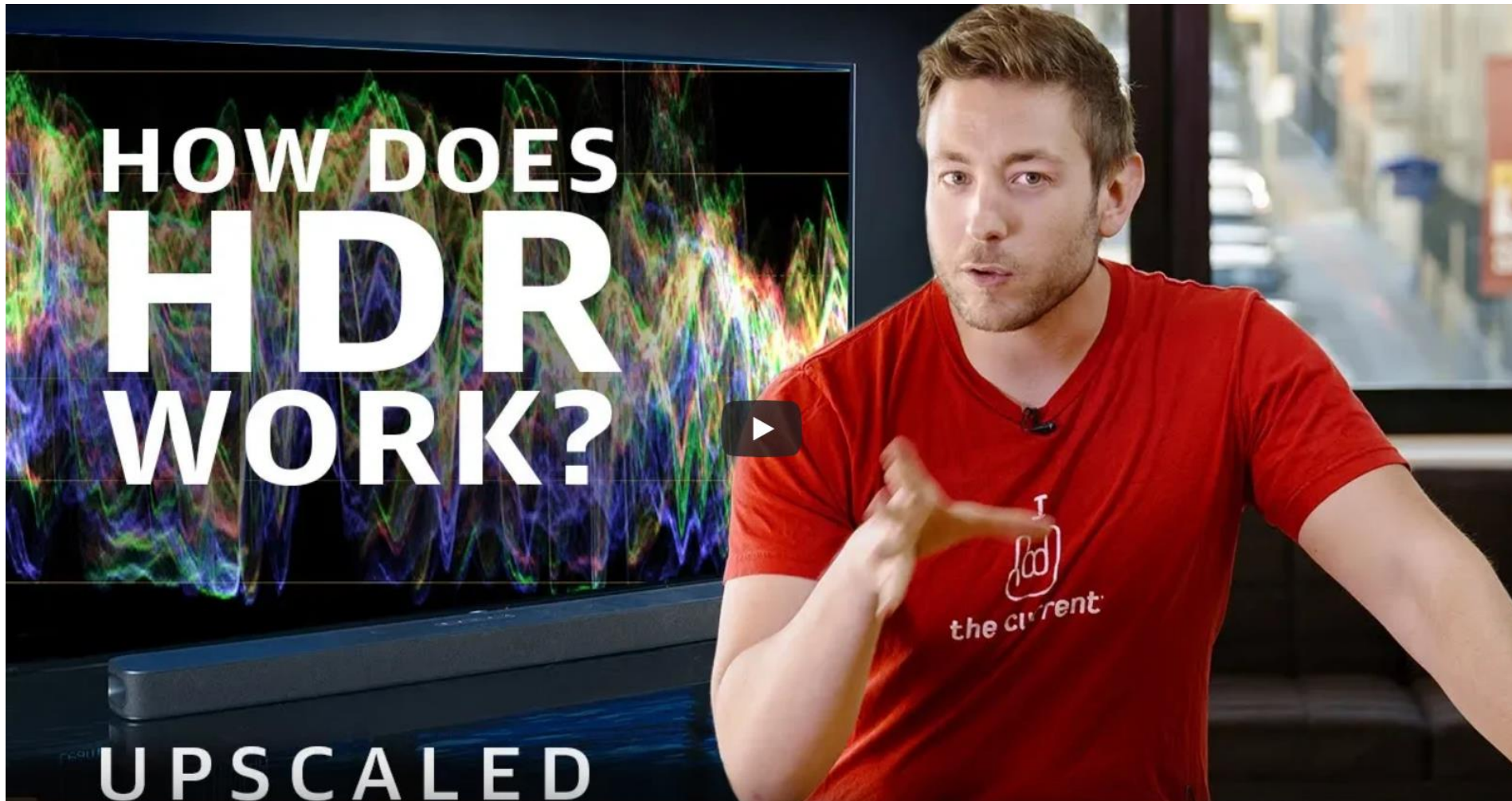
- HDR + Wide color gamut (WCG) gives us powerful new tools to create visual experiences that are clearly visible to the average viewer
- Making this all work requires lots of changes to the production pipeline and the products used to display it

HDR Encoding

- **Most formats are lossless**
- **Adobe DNG (digital negative)**
 - Specific for RAW files, avoid proprietary formats
- **RGBE**
 - 24 bits/pixels as usual, plus 8 bit of common exponent
 - Introduced by Greg Ward for Radiance (light simulation)
 - Enormous dynamic range
- **OpenEXR**
 - By Industrial Light + Magic, also standard in graphics hardware
 - 16bit per channel (48 bits per pixel) 10 mantissa, sign, 5 exponent
 - Fine quantization (because 10 bit mantissa), only 9.6 orders of magnitude
- **JPEG 2000**
 - Has a 16 bit mode, lossy

HDR Technologies Summary

Luminance
Dynamic Range
Color Space
Bit Depth



HDR formats

- Summary of all HDR encoding formats (Greg Ward):
http://www.anywhere.com/gward/hdrenc/hdr_encodings.html
- Greg's notes: <http://www.anywhere.com/gward/pickup/CIC13course.pdf>
- <http://www.openexr.com/>

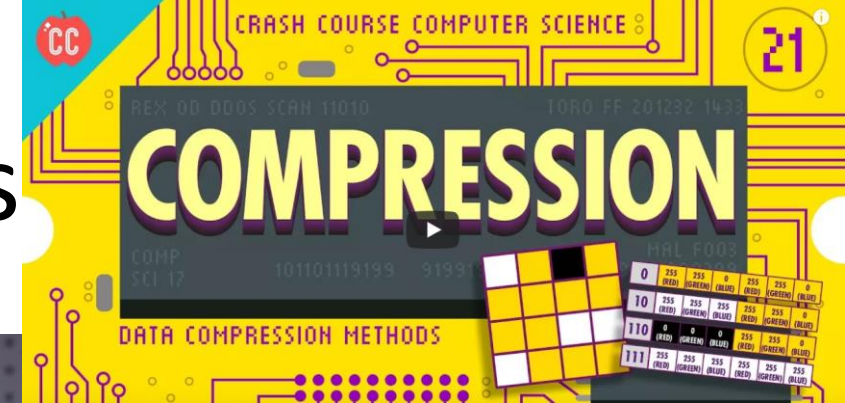
Some Standard Image Formats

- EPS — Encapsulated PostScript
- **GIF — Graphics Interchange Format**
- PICT — may contain geometric data (lines, polygons, etc.)
- **JPEG — Joint Photographic Experts Group, lossy**
- PNG – includes transparency (alpha channel)
- and many many more....

Image Compression

Recap: Compression Algorithms

<https://www.youtube.com/watch?v=OtDxDvCpPL4>



Compression Techniques - Classification

Entropy coding	run-length coding	
	Huffman coding	
	arithmetic coding	
Source coding	prediction	DPCM
		DM
	transformation	FFT
		DCT
	vector quantization	
hybrid coding	JPEG	
	MPEG	
	px64	
	DVI (RTV, PLV)	

Entropy Coding

- **lossless** encoding
- used **regardless of media's** specific **characteristics**
- data taken as simple digital sequence
- One of the main types of entropy coding
 - creates and **assigns a** unique prefix-free **code to each unique symbol** that occurs in the input.
 - then **compress data by replacing** each fixed-length **input symbol with** the corresponding variable-length prefix-free output **codeword**.
- Examples: **Run-length encoding, Huffman encoding, Arithmetic encoding**

Source Coding & Hybrid Coding

- Usually **lossy** encoding (can also be lossless)
- **takes into account the data semantics**
- degree of compression **depends on data content**
- In source coding the particular **code** to be used **is chosen to match the source** (i.e. the relative probabilities of the symbols in the source alphabet).
- Example: Discrete Cosine Transformation (DCT) as transformation technique of the spatial domain into the two-dimensional frequency domain

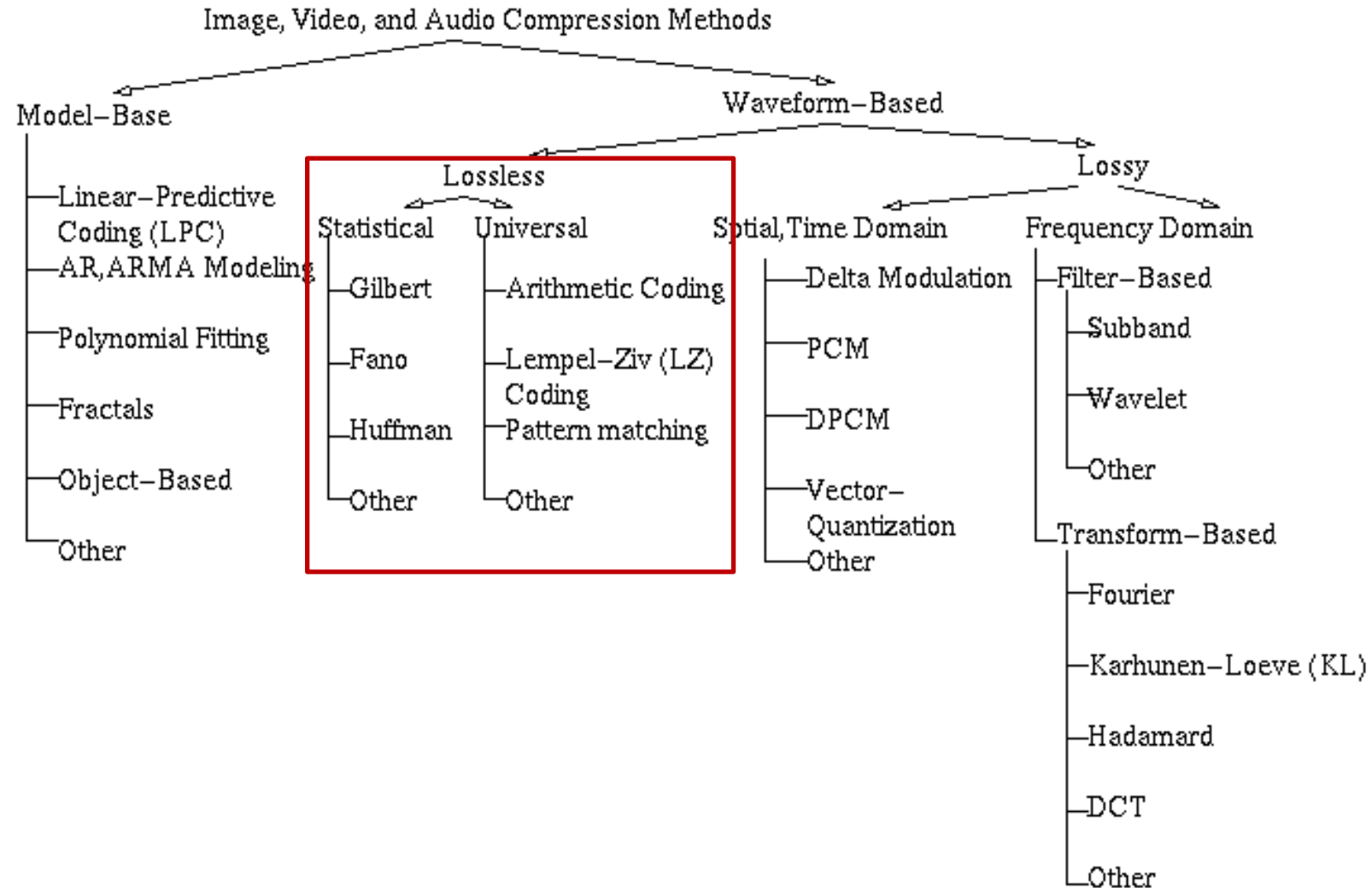
Hybrid encoding

- Used by most multimedia systems
- Combination of entropy and source encoding
- Examples: JPEG, MPEG, H.261

Decompression Requirements

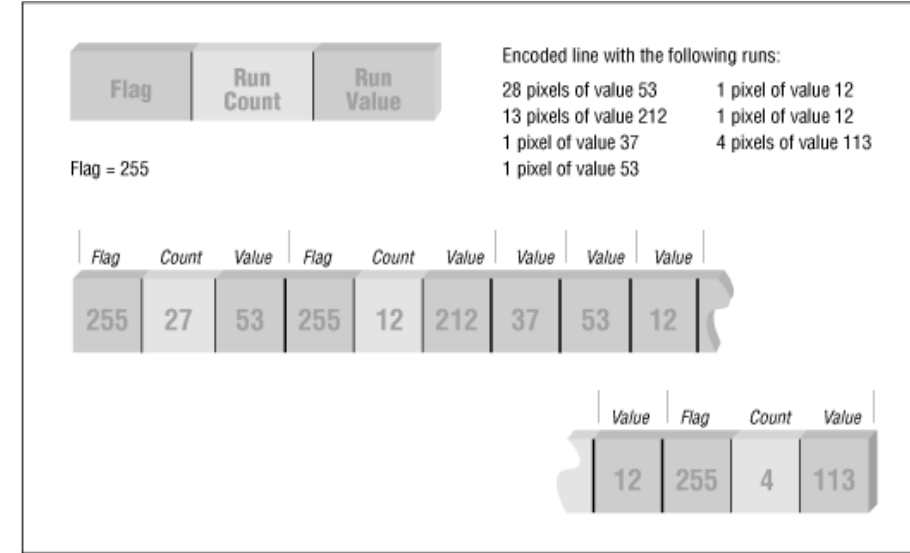
- Dialog mode applications (e.g. editing) might need **symmetric** compression
= compression and decompression take about the same time
- Retrieval mode applications (e.g. video playback) use **asymmetric** compressions
 - compression is performed once and sample time available
 - decompression is performed frequently and needs to be done fast (in case of time-dependent media)

Lossless Compression Algorithms



Run-Length Encoding

- **Entropy coding** algorithm—
content independent coding
- Replaces the sequence of the same consecutive bytes with the number of occurrences
- the number of occurrences is indicated by a *special flag (!)*
- Procedure:
 - if the same byte occurs at least 4 times count number of occurrences
 - write compressed data in the format
 - the counted byte '!' number of its occurrences



Run-length Encoding

- Example:
 - uncompressed sequence (20 Bytes):
ABCCCCCCCCCDEFFFFGGG
 - compressed sequence (13 Bytes): ABC!9DEF!4GGG
- Variations
 - Zero suppression technique (remove zeros)
 - text compression technique (also for images, video, audio)
 - diatomic encoding (represents a pair of characters as one special character)
- Is used for older image file formats e.g. Windows Bitmap, Targa or PCX.
In Windows *.rle is used

Statistical Encoding

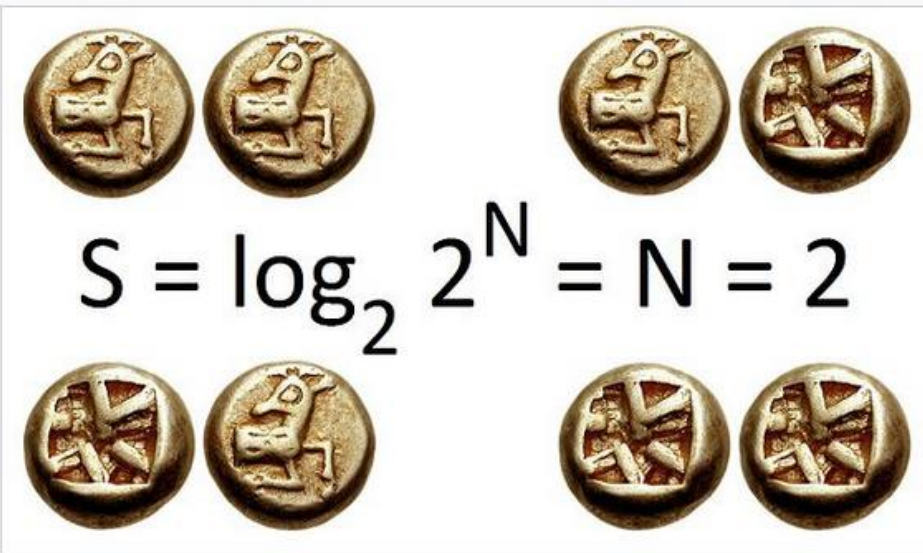
- Frequency dependent encoding—belongs to **entropy encoding**
- given is a sequence of symbols: $S_1, S_2, S_3, \dots$ and the probability of occurrence of each symbol $p(S_i)=p_i$
- Ex.: $p(A)=0.16, p(B)=0.51, p(C)=0.09, p(D)=0.13, p(E)=0.11$
first choice: encode A,B,C,D,E as 000,001,010,011,100

Q: What is the minimum number of bits per symbol?

A: The theoretical minimum average number of bits per code word is known as **entropy** (H), according to Shannon

Entropy

- Introduced by Claude Shannon in his 1948 paper "A Mathematical Theory of Communication" - sometimes called **Shannon entropy**
- Example: A biased coin with probability p of landing on heads and probability $1-p$ of landing on tails. The maximum surprise is for $p = 1/2$, when there is no reason to expect one outcome over another, and in this case a coin flip has an entropy of **one bit**. The minimum surprise is when $p = 0$ or $p = 1$, when the event is known and the entropy is zero bits.
- Shannon's experiments with human predictors show an information rate between 0.6 and 1.3 bits per character in English.
- Shannon's entropy **measures the information contained in a message** as opposed to the portion of the message that is determined (or predictable). Examples of the latter include redundancy in language structure or statistical properties relating to the occurrence frequencies of letter or word pairs, triplets etc.



Two bits of entropy: In the case of two fair coin tosses, the information entropy in bits is the base-2 logarithm of the number of possible outcomes; with two coins there are four possible outcomes, and two bits of entropy. Generally, information entropy is the average amount of information conveyed by an event, when considering all possible outcomes.

$$H(X) = - \sum_{i=1}^n P(x_i) \log P(x_i)$$

Huffman Encoding

- Statistical encoding, depends on **occurrence frequency of single characters or sequences** of data bytes
- Characters stored with their probabilities
- Length (number of bits) of the coded characters differs
- Shortest code assigned to the most frequently occurring character
- To determine Code, it is useful to construct a **binary tree**
 - leaves are characters to be encoded
 - nodes carry **occurrence probabilities** of the characters belonging to the subtree

Huffman Encoding — Algorithm

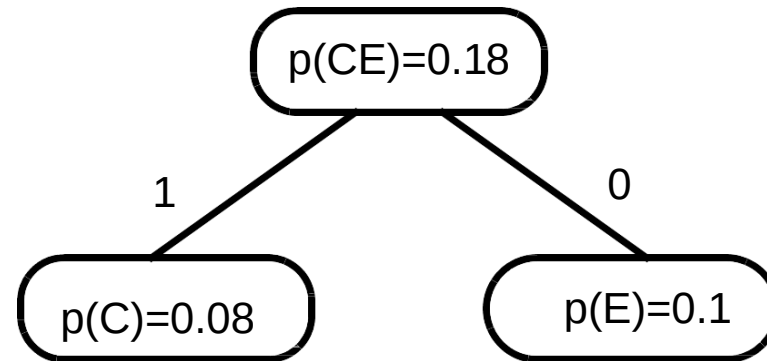
build a Huffman Tree:

$p(A)=0.17$, $p(B)=0.51$, $p(C)=0.08$, $p(D)=0.14$, $p(E)=0.1$

```
build ordered list of symbols (increasing probability)
do while list contains at least 2 elements
    construct tree using the first two elements in list
    add parent node for the union of these elements and compute probability
    mark edges by '0' and '1'
    delete the first 2 elements in list; insert parent into list
end
```


Huffman Encoding—Example

step 1: $p(C)=0.08$, $p(E)=0.1$, $p(D)=0.14$, $p(A)=0.17$, $p(B)=0.51$

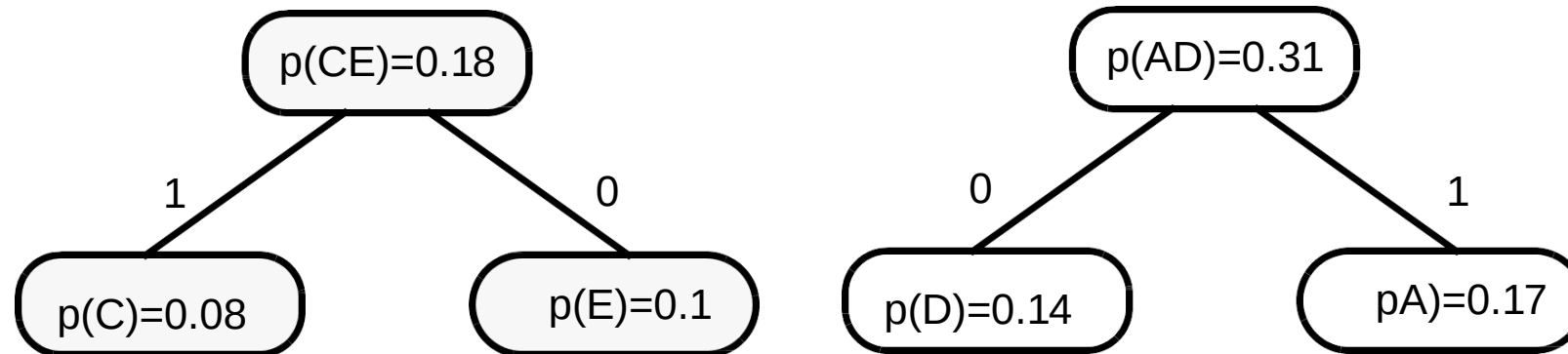


Huffman Encoding—Example

step 2: $p(D)=0.14$, $p(A)=0.17$, $p(CE)=0.18$, $p(B)=0.51$

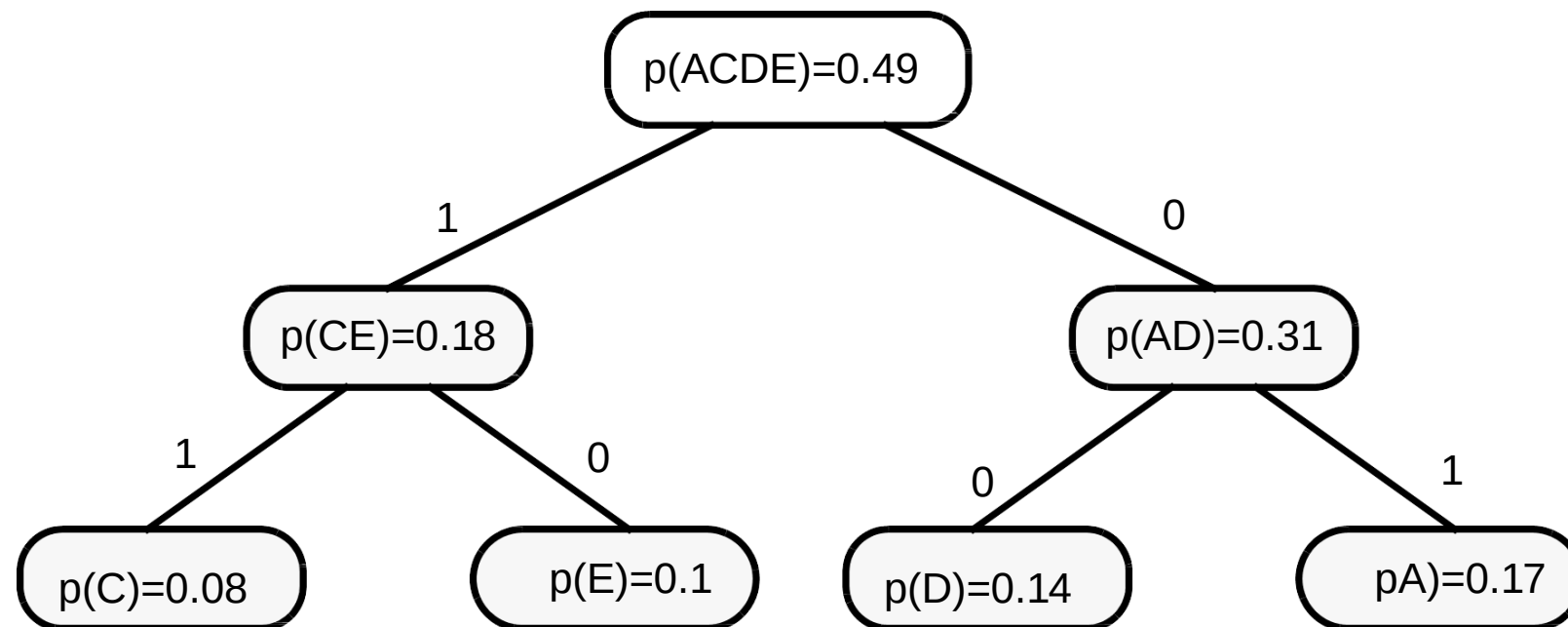
```

build ordered list of symbols (increasing
probability)
do while list contains at least 2 elements
    construct tree using the first two
    elements in list
    add parent node for the union of these
    elements and compute probability
    mark edges by '0' and '1'
    delete the first 2 elements in list;
    insert parent into list
end
    
```



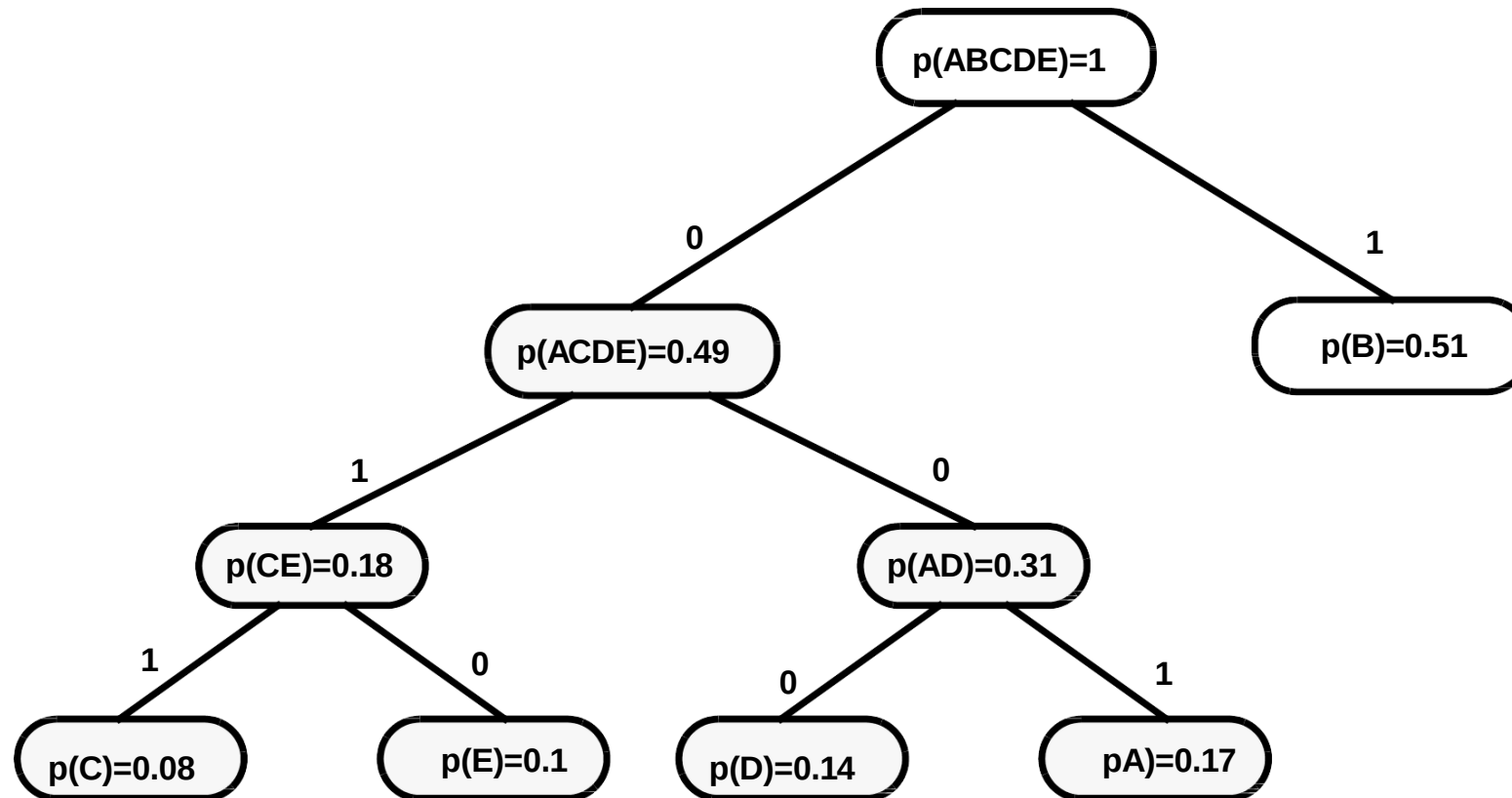
Huffman Encoding—Example

step 3: $p(CE)=0.18$, $p(AD)=0.31$, $p(B)=0.51$



Huffman Encoding—Example

last step: $p(ACDE)=0.49$, $p(B)=0.51$



Huffman Encoding—Example

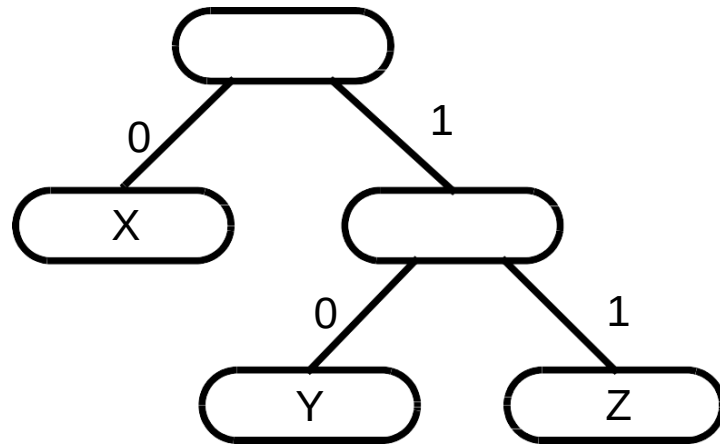
encode symbols according to tree

A	001
B	1
C	011
D	000
E	010

$p(C)=0.08$, $p(E)=0.1$, $p(D)=0.14$, $p(A)=0.17$, $p(B)=0.51$

Huffman Decoding—Example

given: 000101011 and Huffman Tree

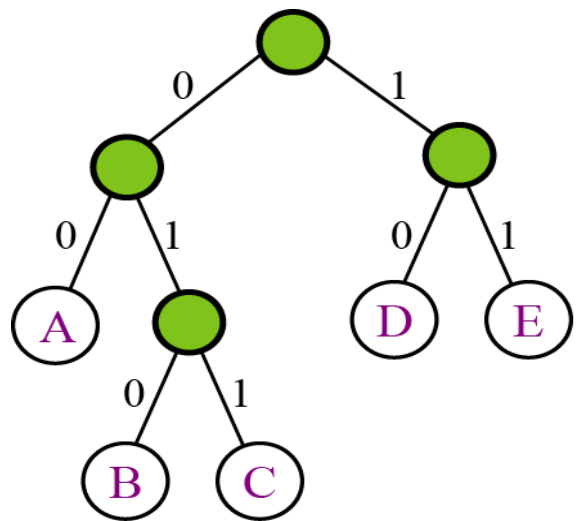


reconstruct message from bit stream using
finite state machine

Solution: XXXYYZ

Huffman Encoding

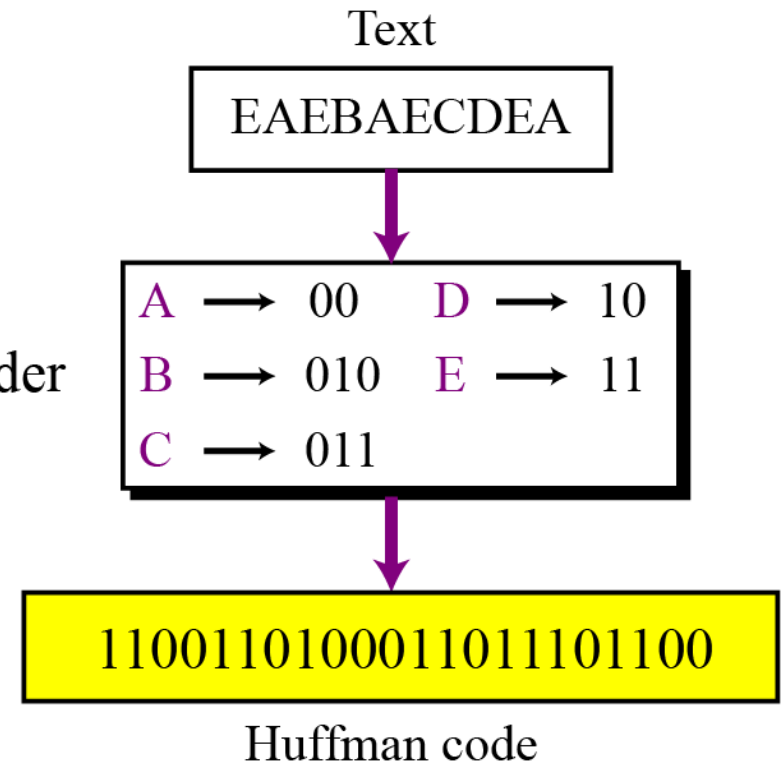
- Character code found by starting at the root and following the branches that lead to that character.
- The code itself is the bit value of each branch on the path, taken in sequence.



A: 00	D: 10
B: 010	E: 11
C: 011	

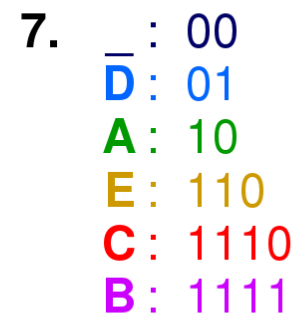
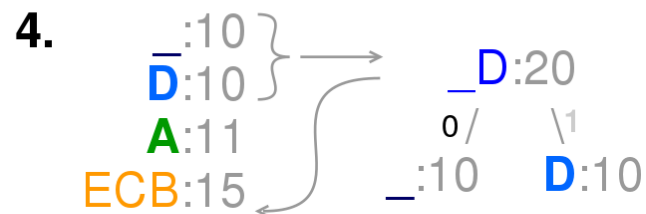
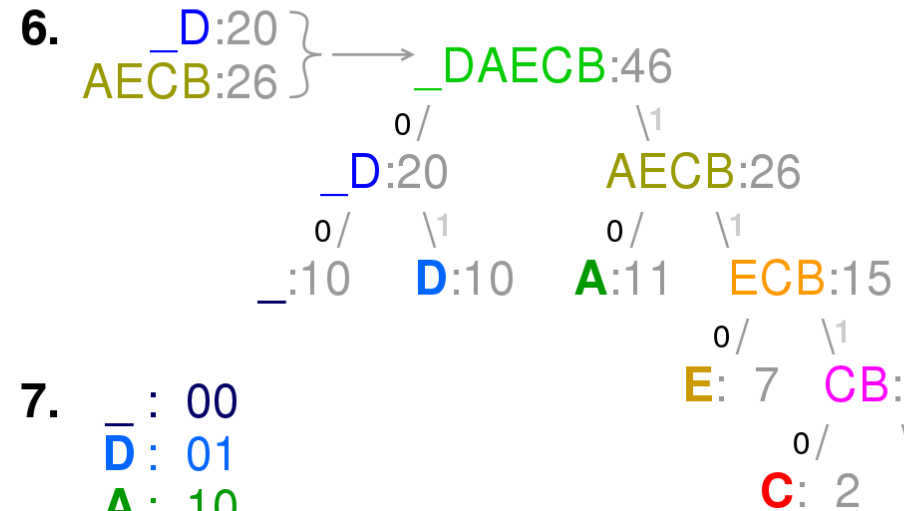
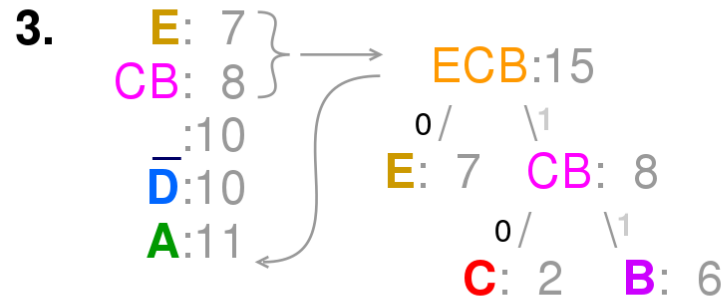
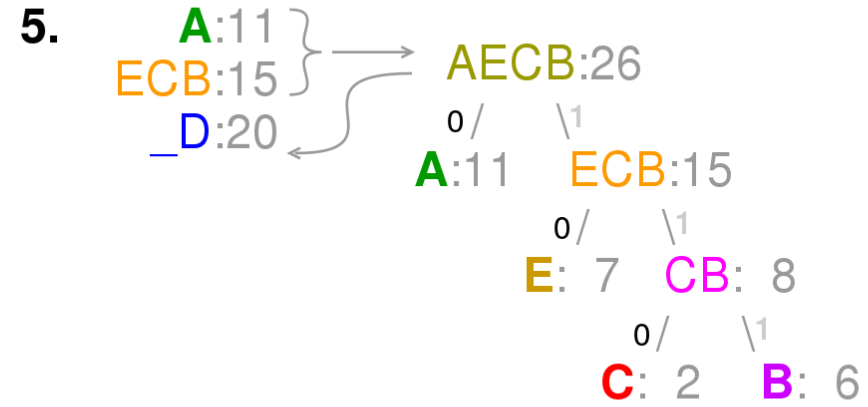
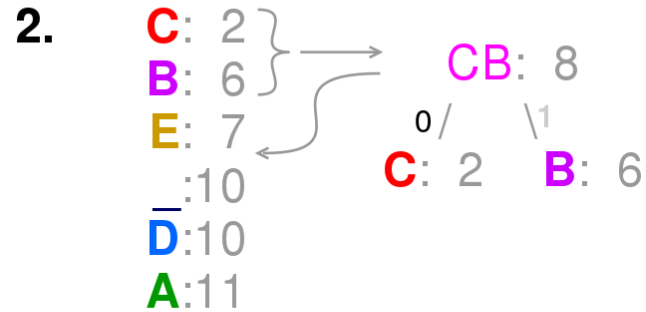
Code

Encoder



- Decoding: reverse process

1. "A_DEAD_DAD_CEDD_A_BAD_BABE_A_BEADED_ABACA_BED"



8. "1000011101001000110010011110110011100100100011111001001111101111110
0010001111110100111001001011111011101000111111001"

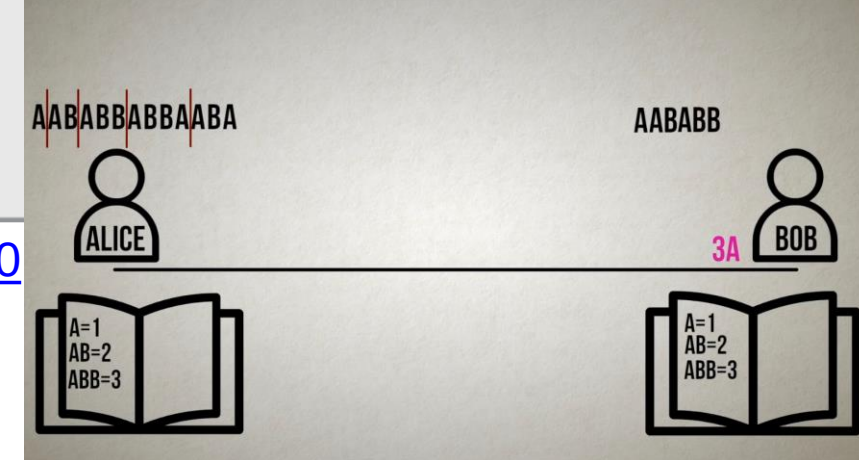
Application of Huffman Coding for Video

- If the image information can be transformed into a bit stream, Huffman table can be used to compress data without loss
- Simple way to generate a bit stream is to code pixels individually and read them line by line
- For video: Huffman table can be used for a single sequence of images
 - for a set of scenes
 - for an entire film clip
- need Huffman table for encoding and decoding
- Is **used for MP3** encoding in the final step!

Lempel Ziv (LZ) Dictionary-based Encoding

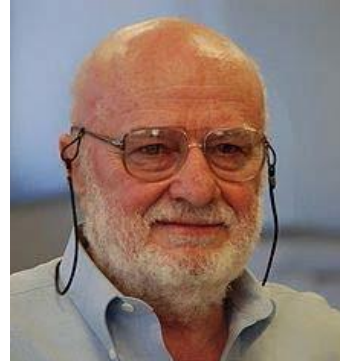
<https://www.youtube.com/watch?v=RV5aUr8sZD0>

- Dictionary is a table of strings
 - Sender and receiver have a copy of dictionary
 - Previously-encountered strings are substituted by their index in dictionary
- Compression - two concurrent events:
 - Building an indexed dictionary
 - Compressing a string of symbols.
- Algorithm extracts smallest substring not in the dictionary from remaining uncompressed string.
- Stores a copy of this substring in dictionary as a new entry and assigns it an index value.
- Compression occurs when substring (except for the last character) is replaced with the index found in the dictionary.
- Process inserts the index and the last character of the substring into the compressed string.

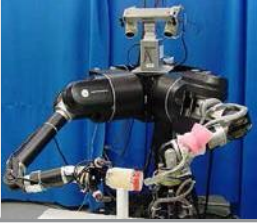


Used in GIF & ZIP

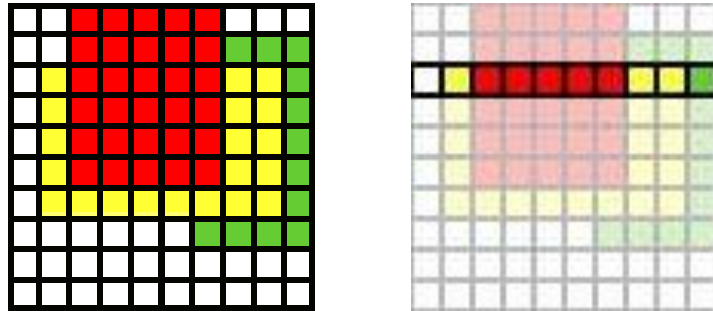
Abraham Lempel



Lossless Image Formats



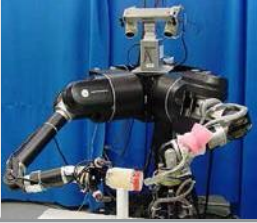
- GIF-Format (Graphics Interchange Format)
 - LZW-Compression (Lempel, Ziv, Welch): works line-wise



- 1 st line, 2nd line
- 3rd line is compressed as:
 - 1 white, 1 yellow, 5 red, 2 yellow, 1 green
- Row 4 to 6: „as row 3“
- Indexed (1-8 bit Color Lookup Table)



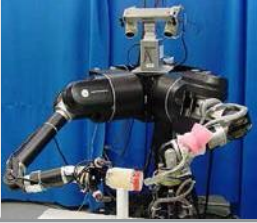
CompuServ GIF (gif)



- First standardized in 1987 by CompuServ (called GIF87a)
- Updated in 1989 to include transparency, interlacing, and animation (called GIF89a)
- Use the LZW (Lempel-Ziv Welch) algorithm for compression (not free, licenses necessary)
- A maximum of **256 colors** freely selectable out of 24 bit ($2^{24} = 16.777.216$)
- Does not work for photographs
- Suitable for small images such as icons
- Simple animations



Portable Network Graphics (png)



- Developed because of licensed LZW with gif
- Replacing GIF and TIFF (not JPEG!)
- 3 Types
 - True Color (3 x 16 bit/pixel)
 - Grayscale (1 x 8 bit/pixel)
 - Palette (256 colors)
- Compression similar to PKZIP (Phil Katz)
- The standard zip format provides approximately **62 percent** compression. (RAR results in a **71 percent** compression rate; 7-zip has **76 percent**)



Differential Encoding

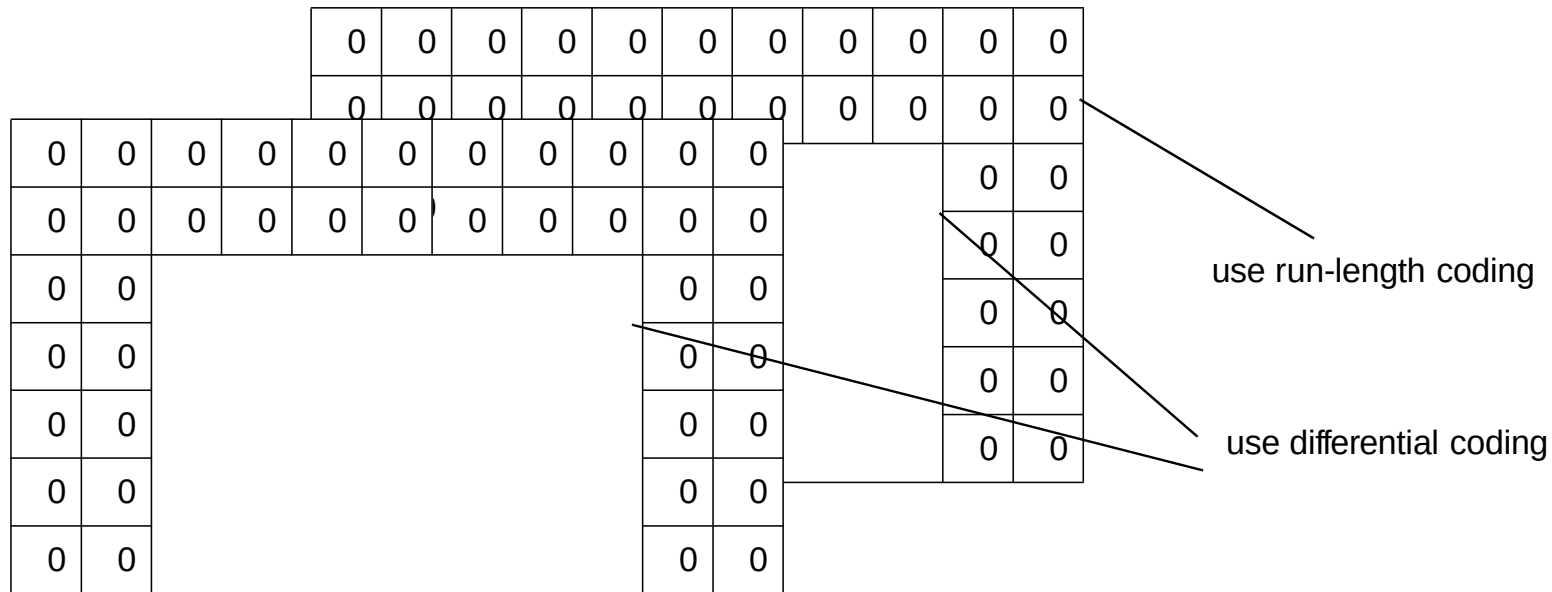
- belongs to **source coding**
- consider sequence of symbols $S_1, S_2, S_3, \dots$ where values are not zeros, but do not vary much $\rightarrow$ calculate difference from previous value
- Ex.: still images—calculation of differences between nearby pixels or pixel groups

0	0	0	0	0
0	255	94	87	100
0	0	0	0	255
0	0	0	0	0

- edges result in large values
 - similar chrominance or luminance values result in small values
- zeros can be suppressed by run-length coding

Differential Coding in Video (Examples)

- static background (videoconferencing, newscast, etc.)



- motion compensation—8x8 blocks are compared; areas similar, only shifted, e.g., to the right (*motion vector*)

Lossy Image Compression

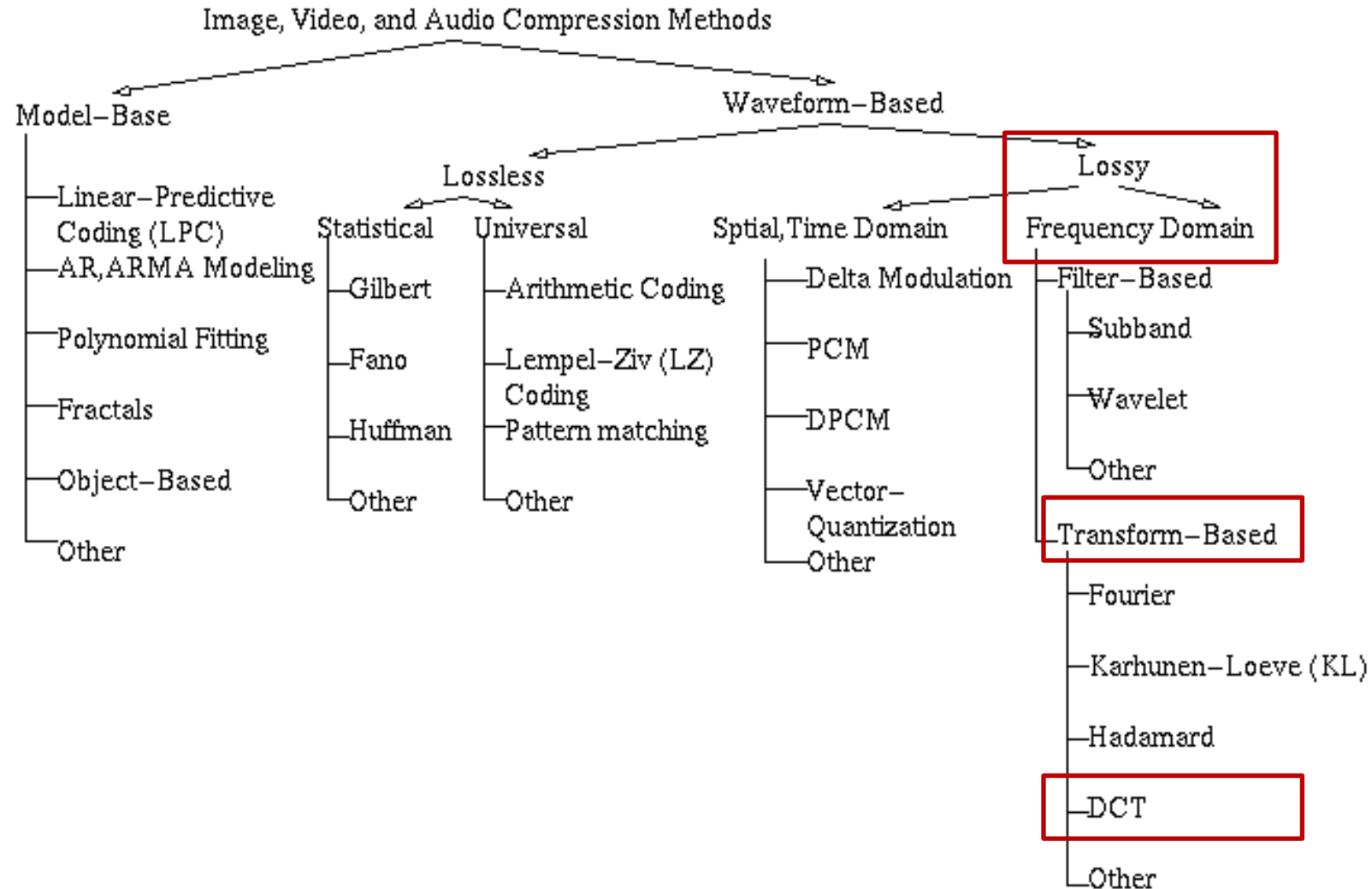
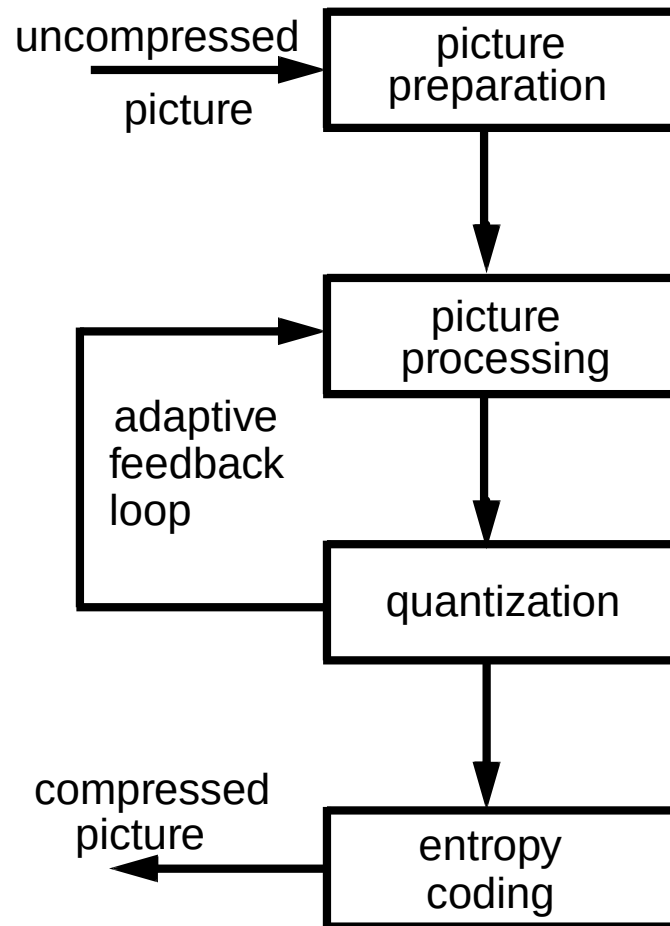


Image Compression Steps



Possible operations:

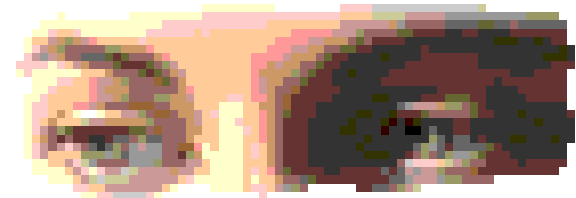
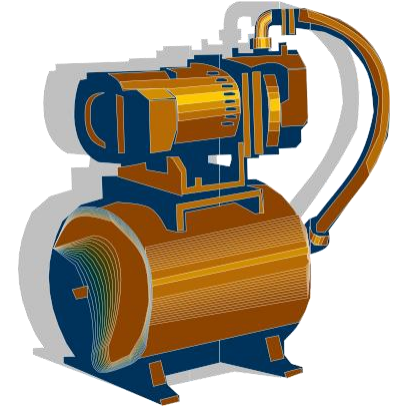
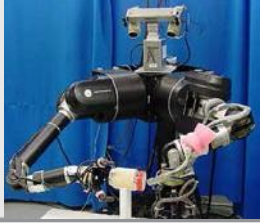
- analog-to-digital conversion
- generation of appropriate digital representation
- image division into 8 x 8 blocks
- fix number of bits/pixel

Compression Algorithm

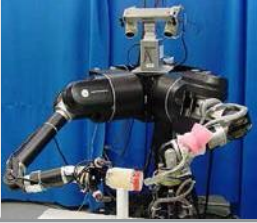
- transformation from time to frequency domain e.g., using DCT
- motion vector application for motion video
- Typically a lossy step
- mapping of real numbers to integers (reduction in precision)
example: μ -law audio—from 12 bits to 8 bit quantization
- compress a sequential digital stream without loss

Lossy Compression

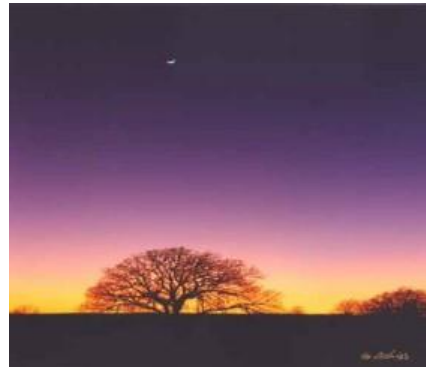
- Run-length coding works well for simple images (graphics) and computer data
- However, a large, detailed picture usually is not reduced enough
- In order to reduce information further lossy compression is needed
 - Information is permanently removed
 - The trick is to remove details that are not perceived by a human observer
 - Many of these psycho-visual coding systems take advantage of a number of aspects of the human visual system



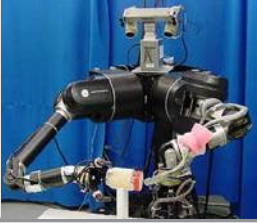
Human Visual System



- The eye is more sensitive to brightness changes than to color changes
- The eye is not able to perceive brightness above or below certain threshold values
- The eye does not perceive little brightness or color changes. The strength of this phenomenon is dependent on the color



Compression of Still Images



- Characteristics of human vision have been transferred to **lossy image compression techniques**
 - remove some image details to achieve a higher compression rate by suppressing higher frequencies
- Such a development is the JPEG format
 - Stands for "Joint Photographic Experts Group"
 - JPEG is the worldwide standard
 - JPEG compresses brightness and color information separately
 - Color information is more compressed (lower sensitivity)
- Color space for JPEG compression
 - RGB values are a combination of brightness and color
 - If the RGB values are separated into a luminance and a color component, the color component is more compressed



Joint Photographic Experts Group (JPEG)

JPEG is a joint standard of ISO and ITU-T

- In June 1987, an adaptive transformation coding technique based on DCT was adopted for JPEG
- In 1992, JPEG became a ISO international standard
- Lossy compression combined with lossless techniques
- Trade-Off between file size and quality
- JPEG typically achieves 10:1 compression with little perceptible loss in image quality

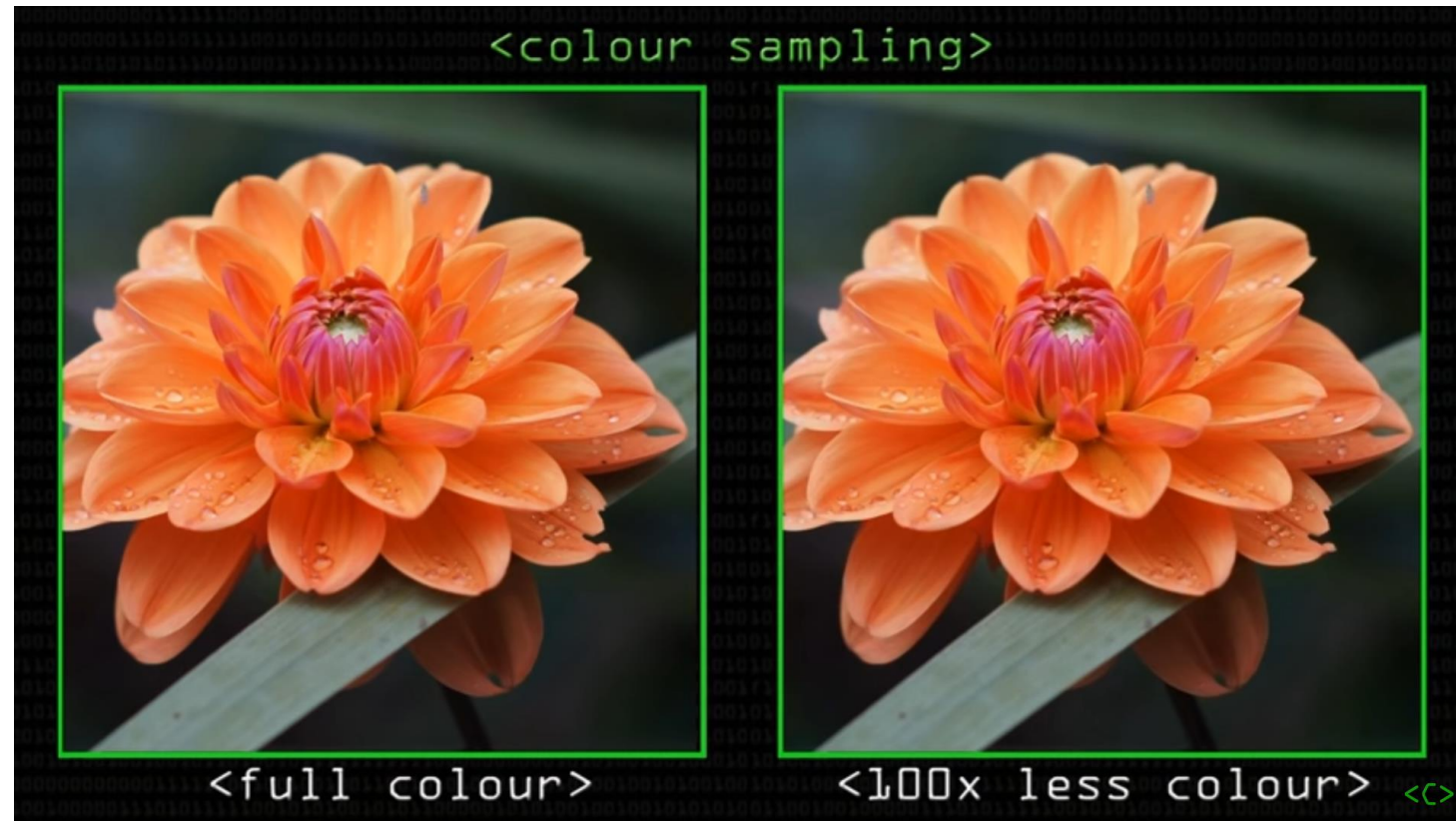
Requirements:

- to be state-of-the-art for degree of compression versus image quality,
- to be parameterizable (user can select parameters),
- to be applicable to any kind of source image, without regard to dimensions, content, image aspect ratio, pixel aspect ratio, etc.,
- to have computational requirements that are reasonable for hardware and software implementations,
- to run on as many standard platforms as possible

Joint Photographic Experts Group (JPEG)

- to support four different modes of operation:
 - **sequential encoding—encoded in same order as scanned**
 - progressive encoding—multiple pass encoding
 - lossless encoding
 - hierarchical encoding—encoded at multiple resolutions

How JPEG Works – Part 1



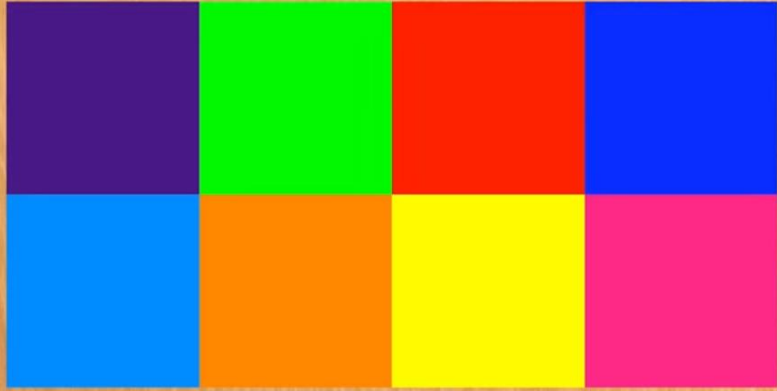
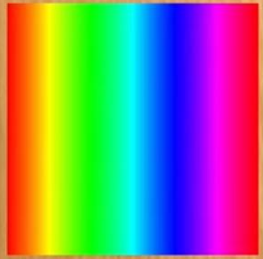
1st Step: Color Conversion YCbCr <-> RGB

- Necessary in preparation for JPEG compression
 - Color space conversion and Downsampling:
 - Separation of the color components from the luminance information
 - Insensitivity of human eye to rapid color changes allows coarser sampling of the color components
- > No loss of subjective quality
- > Significant data reduction

Chroma Subsampling

4:4:4

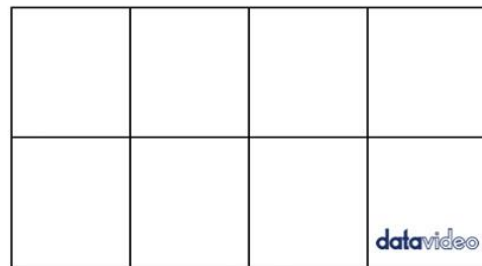
Chroma



Some visual examples
<https://www.red.com/red-101/video-chroma-subsampling>

<https://www.youtube.com/watch?v=32PPzwPjDZ8>

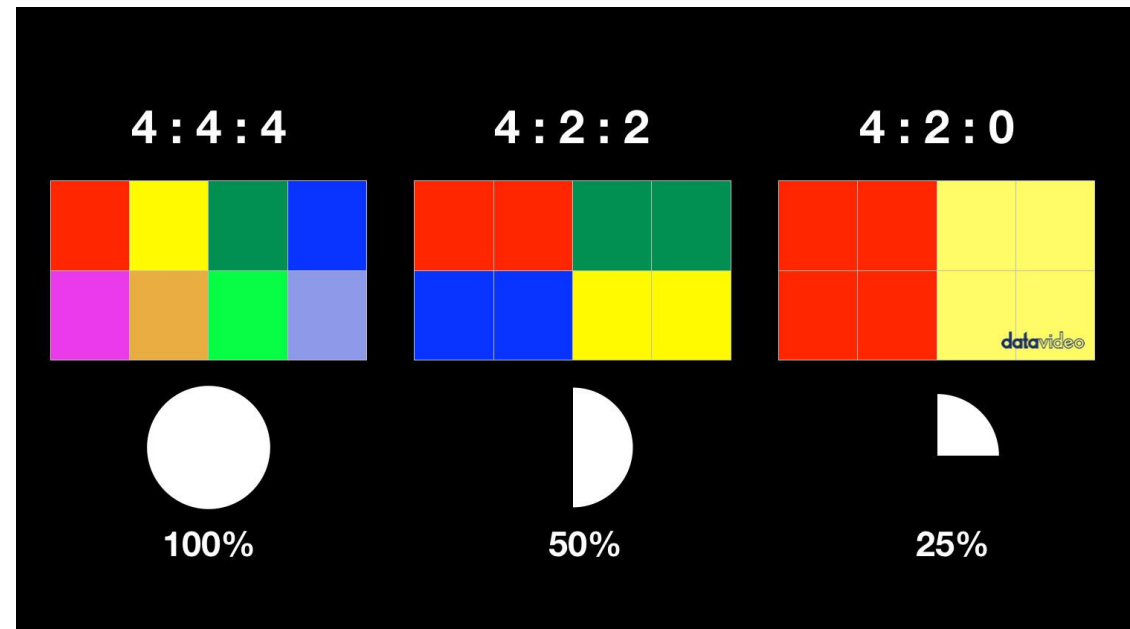
Y	Cr	Cb
The Numbers of Across Pixels 4 x 2	First Row The Numbers of Color	Second Row The Numbers of Color



Cr

Cb

Y **4** x 2



1st Step: Color Conversion YCbCr <-> RGB

- YCbCr (256 levels) can be computed directly from 8-bit RGB as follows:

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

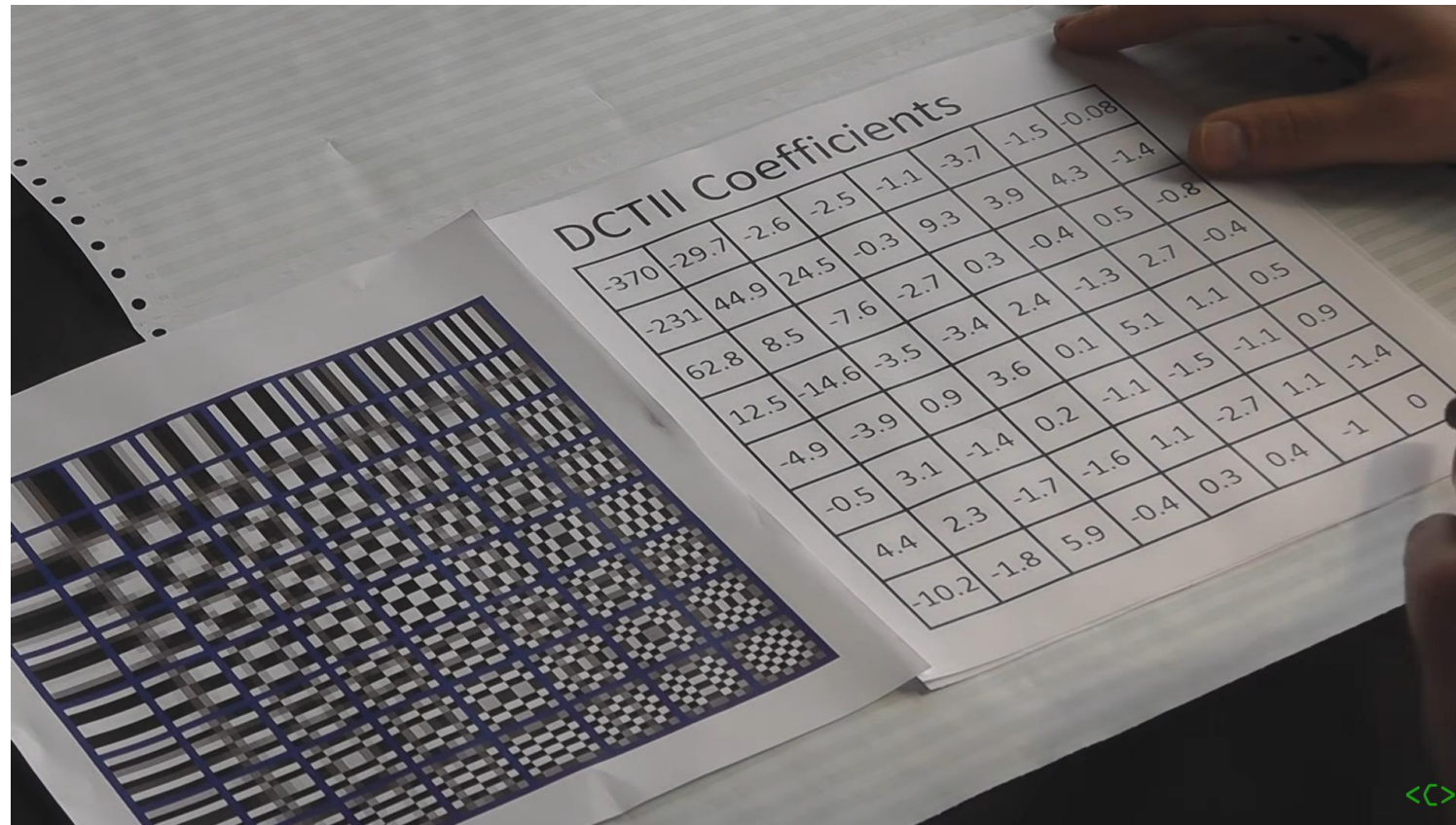
- During decoding YCbCr color space needs to be converted back to RGB:

$$R = Y + 1.402 (Cr-128)$$

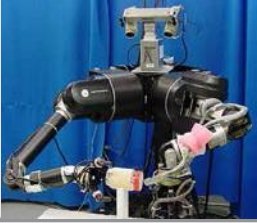
$$G = Y - 0.34414 (Cb-128) - 0.71414 (Cr-128)$$

$$B = Y + 1.772 (Cb-128)$$

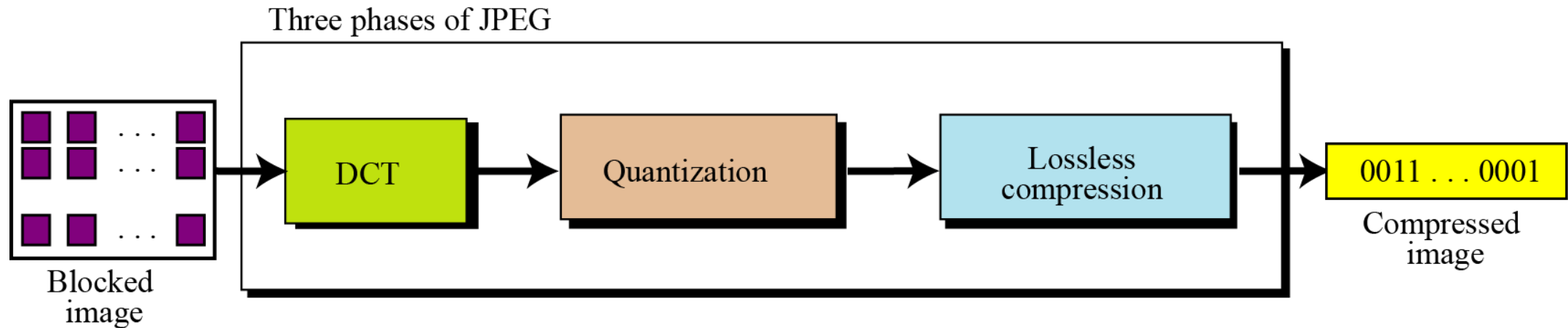
How JPEG Works – Part 2



JPEG Compression - Overview

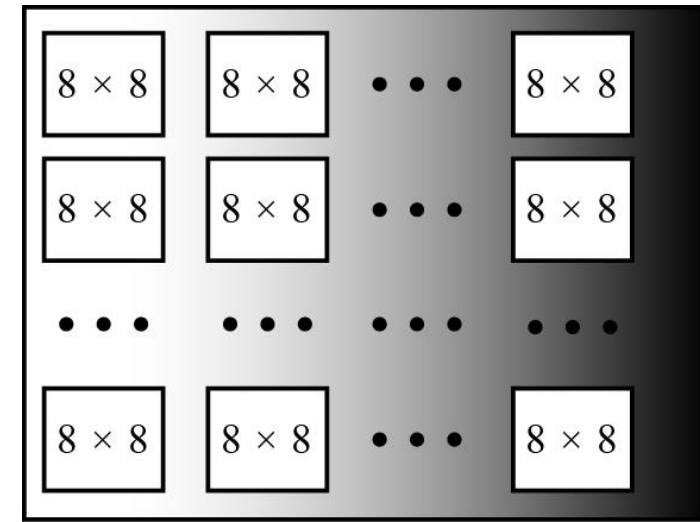


- Idea: change image into a linear (vector) set of numbers that reveals redundancies.
- Redundancies can be removed using one of the lossless compression methods



JPEG Image Preparation

- Images – each channel – divided in data units (blocks), DCT operates on blocks to decrease the number of calculations
- Lossy mode operates on 8x8 pixel blocks (4:4:4 – no subsampling), 16×8 (4:2:2), or most commonly 16×16 (4:2:0).
- Lossless mode operates on data units of 1 pixel
- In most cases data units are processed component by component and passed to image processing
- processing order of data units
 - left-to-right, top-to-bottom
 - interleaved data ordering
- Interleaved data units of different color channels are combined to
 - **Minimum Coded Units (MCU)**



Discrete Cosine Transform

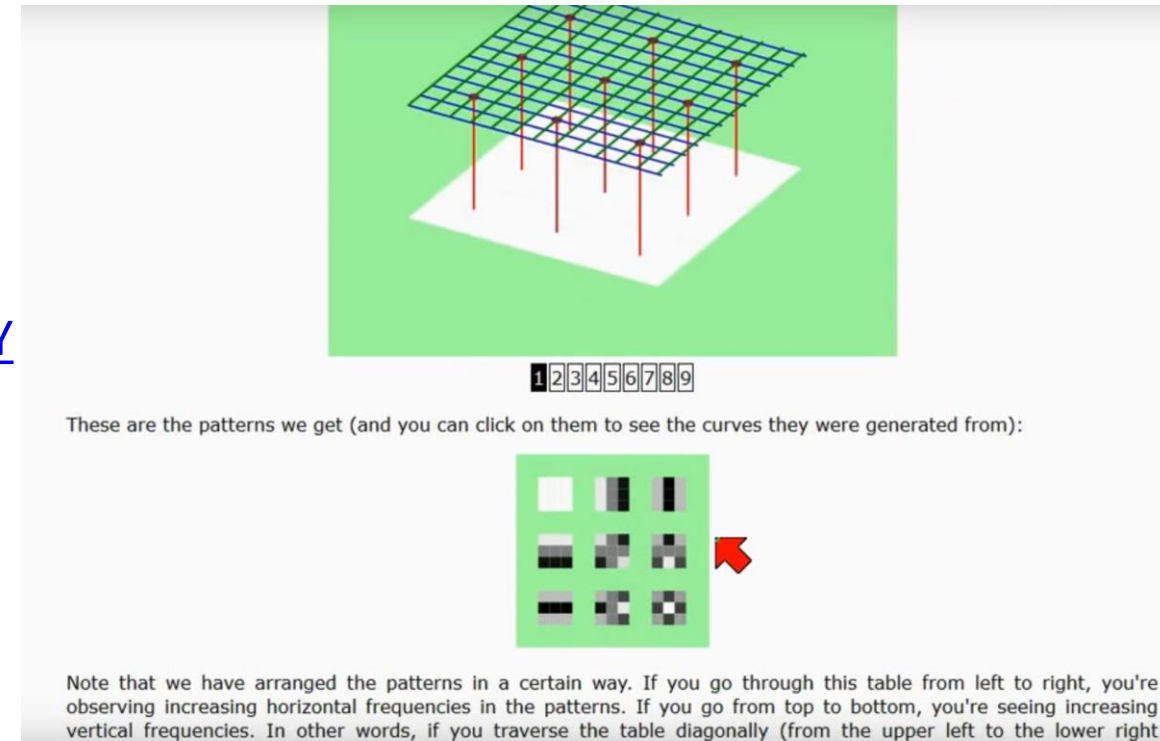
- First step: pixel values shifted into the range $[-128, 127]$
- Forward Discrete Cosine Transformation (Forward DCT) maps values from time to frequency domain

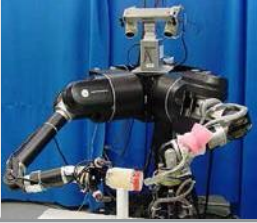
- Interactive Website

<http://weitz.de/dct/>

Explanation:

<https://www.youtube.com/watch?v=7fhHQgu2OcY>





- DCT-based codecs use a two-dimensional version of the transform.
- The 2-D DCT of an 8 x 8 block:

$$F(u, v) = \sum_{x=0}^7 \sum_{y=0}^7 \alpha(u) \alpha(v) f(x, y) \cos\left[\frac{\pi}{8} \left(x + \frac{1}{2}\right) u\right] \cos\left[\frac{\pi}{8} \left(y + \frac{1}{2}\right) v\right]$$

u is the horizontal spatial frequency, for the integers $0 \leq u < 8$

v is the vertical spatial frequency, for the integers $0 \leq v < 8$

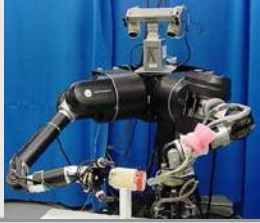
$\alpha(u) = \begin{cases} \sqrt{\frac{1}{8}}, & \text{if } u = 0 \\ \sqrt{\frac{2}{8}}, & \text{otherwise} \end{cases}$ is a normalizing scale factor to make the transformation orthonormal

$f(x, y)$ is the pixel value at coordinates (x, y) “space domain”

$F(u, v)$ is the DCT coefficient at coordinates (u, v) “frequency domain” (indicates how fast the information moves inside the rectangle)

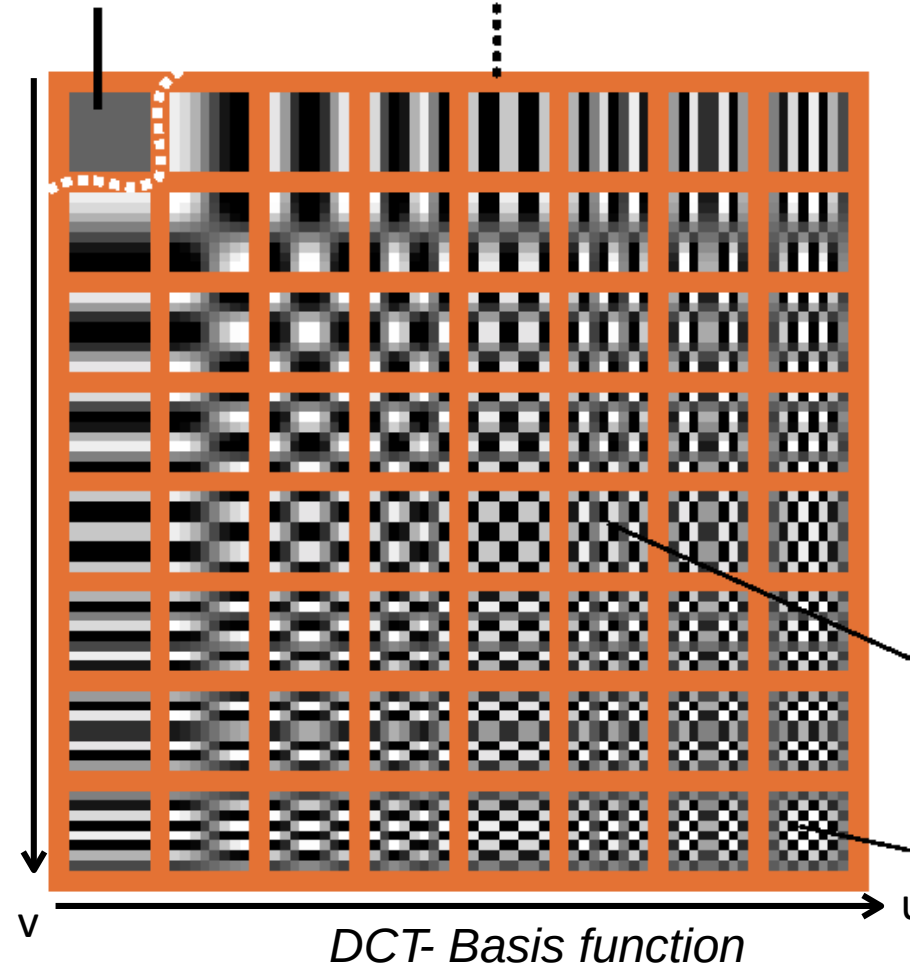
◆ Note: The DCT decomposes a signal into a series of harmonic cosine functions.

DCT Basis Functions

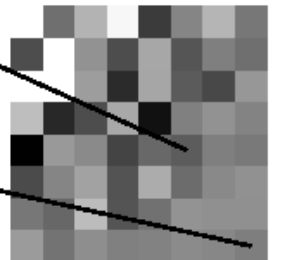
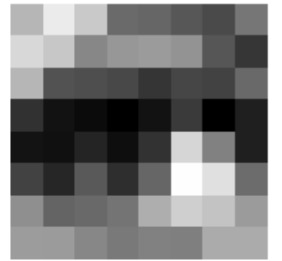


- DCT values indicate the weighting of frequency images
- The brighter the pixel, the greater the value of DCT

Constant component Alternating component



8 x 8 Imageblock



JPEG Image Processing (DCT)

- $S(u,v)$ coefficients
- $S(0,0)$ —lowest frequency in both directions is called DC coefficient
 - determines the fundamental luminance/color of the block
 - frequency = 0 in both directions
- $S(0,1), \dots, S(7,7)$ are called AC coefficients
 - frequency in one or both directions non-zero
- computing of DCT—use factoring

Discrete Cosine Transform

Humans have only a restricted capability to perceive high frequencies.
Images reduced from their high frequent elements may have a high degree of quality while being compressed quite considerably

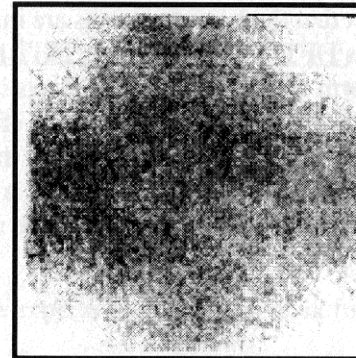
- Low frequency components of an image correspond to brief outlines of image objects
- High frequency components of an image correspond to fine structures

To decrease the high frequencies the data must be sorted by frequencies
->DCT

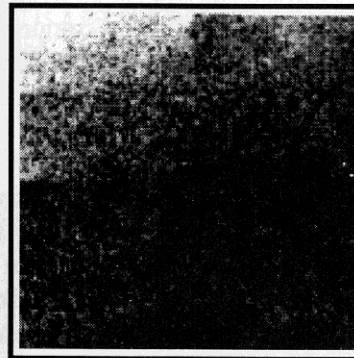
Why DCT?



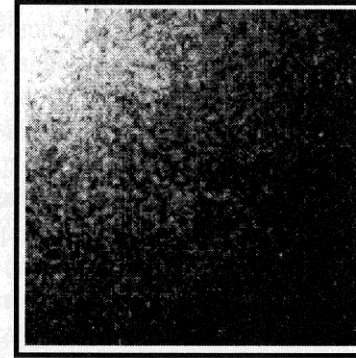
(a) Test Image



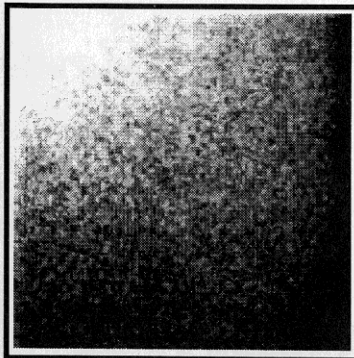
(b) Discrete Fourier transform



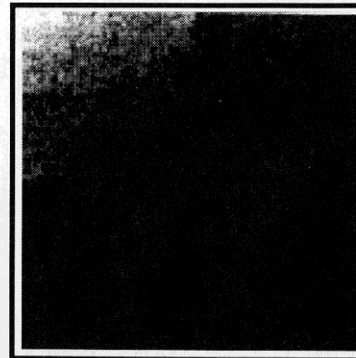
(c) Discrete Cosine transform



(d) Discrete Sine transform



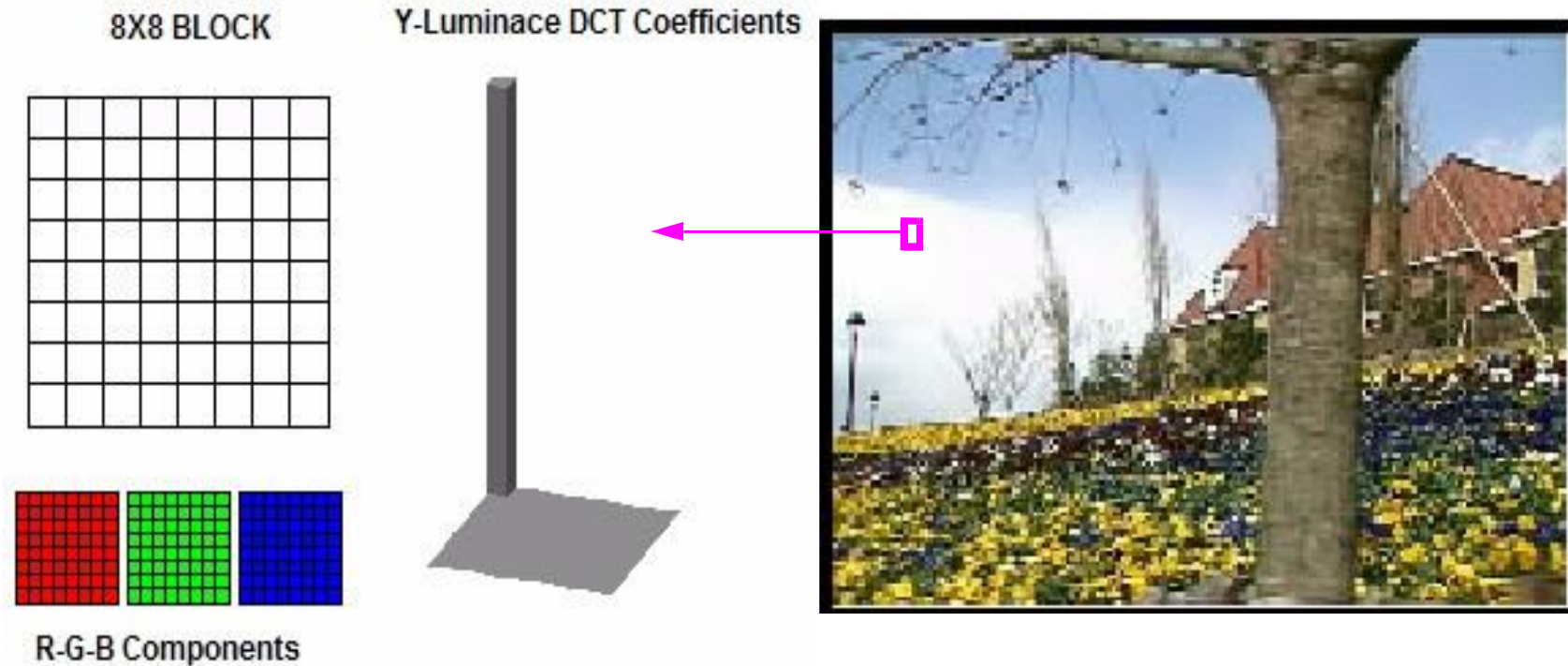
(e) Discrete Hadamard transform



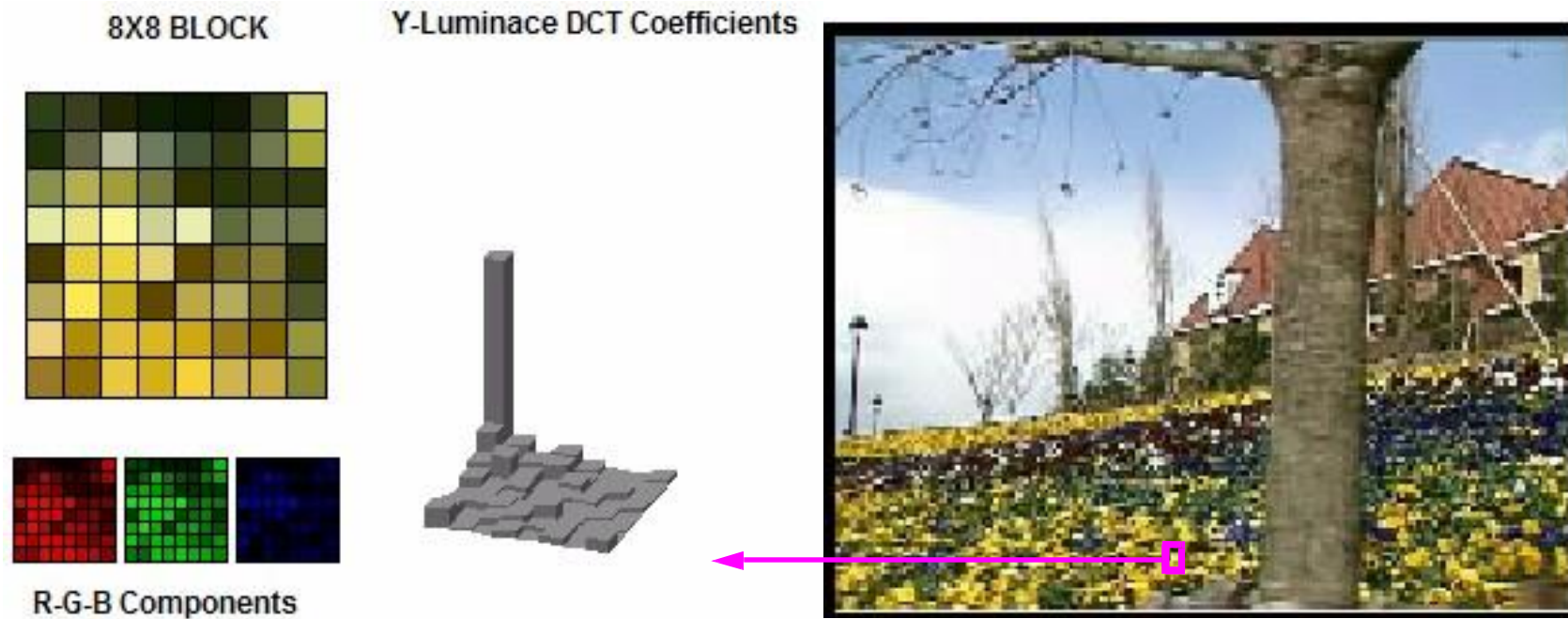
(f) Karhunen-Loeve transform

Karhunen-Loeve not used in practice for image compression because it needs to transmit the transform kernel for every block, which requires huge amount of side information.

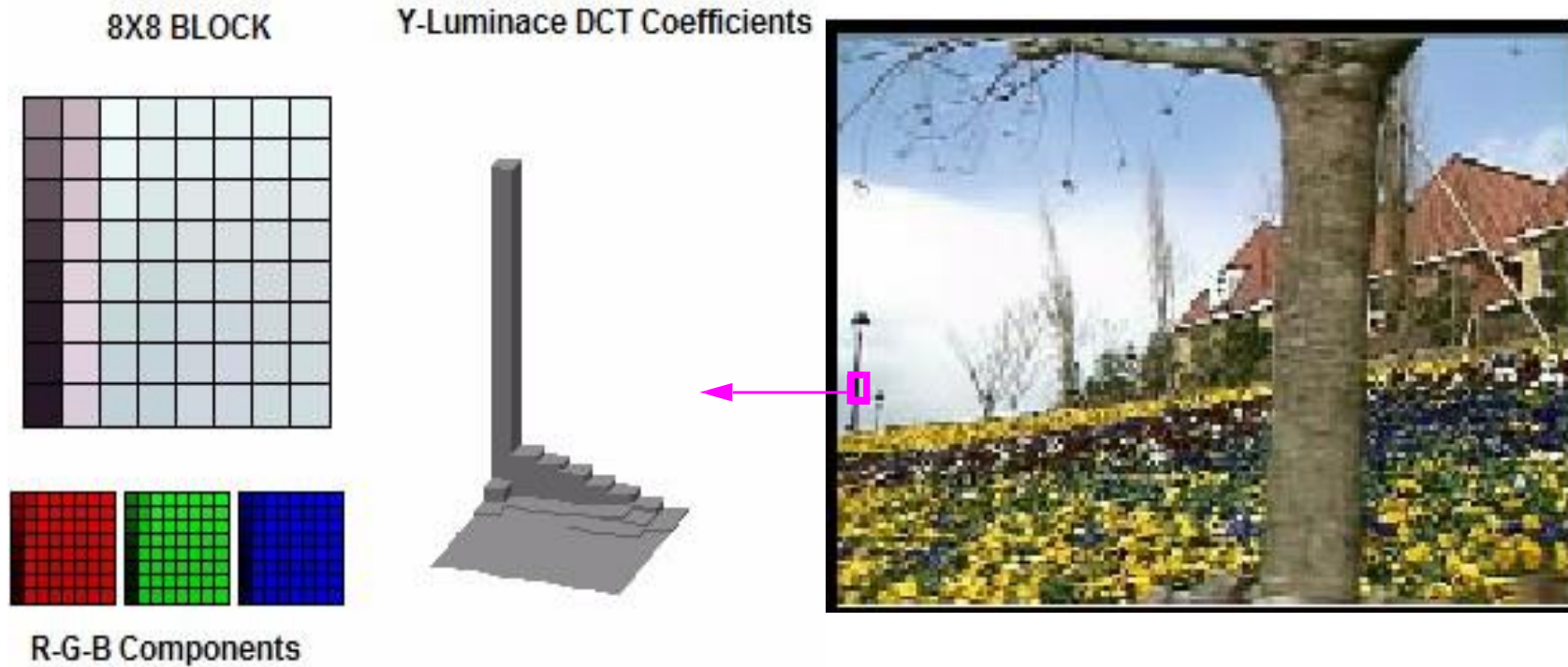
DCT-Coefficients



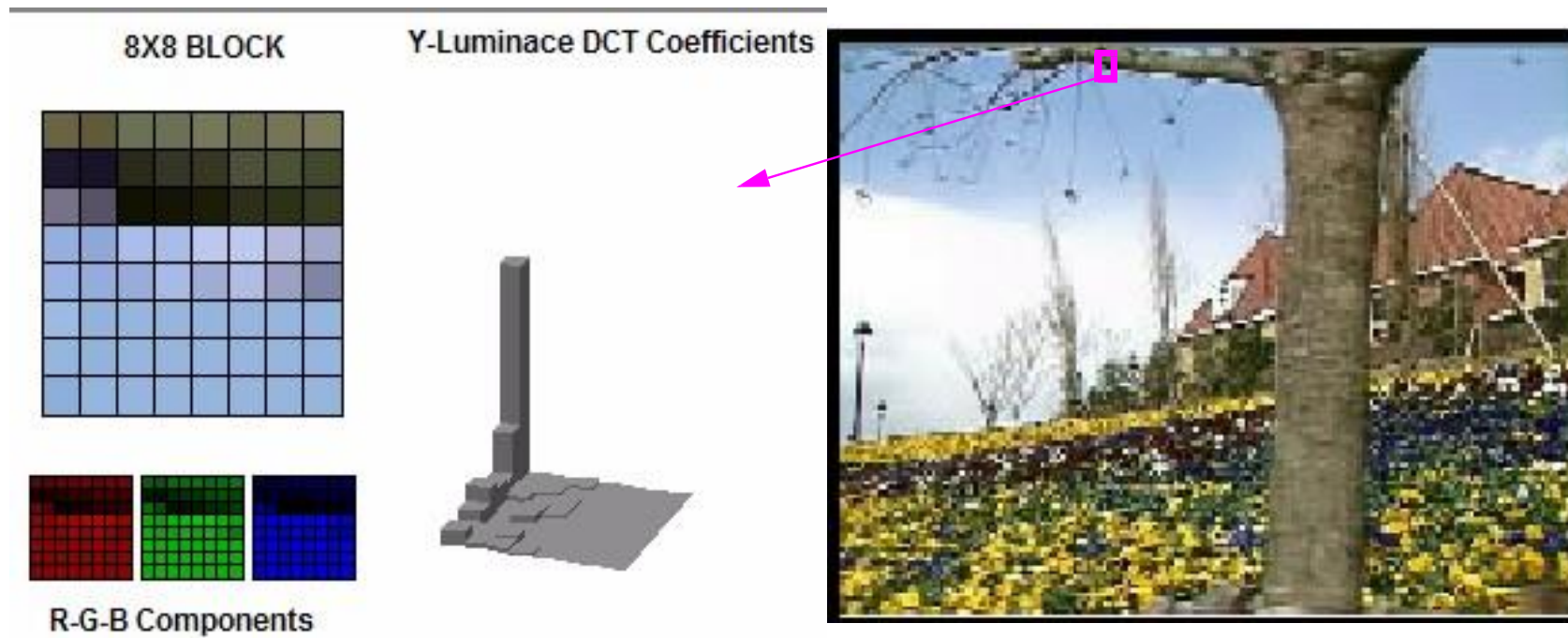
DCT-Coefficients



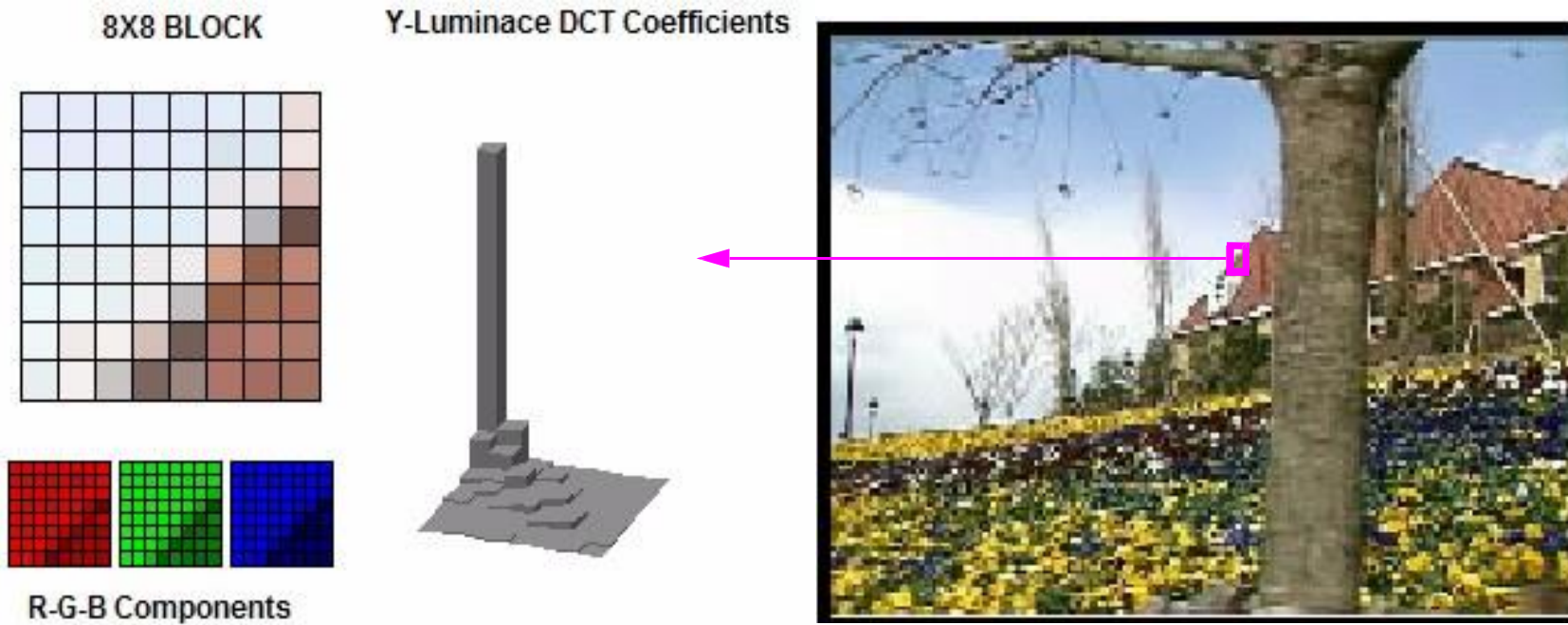
DCT-Coefficients



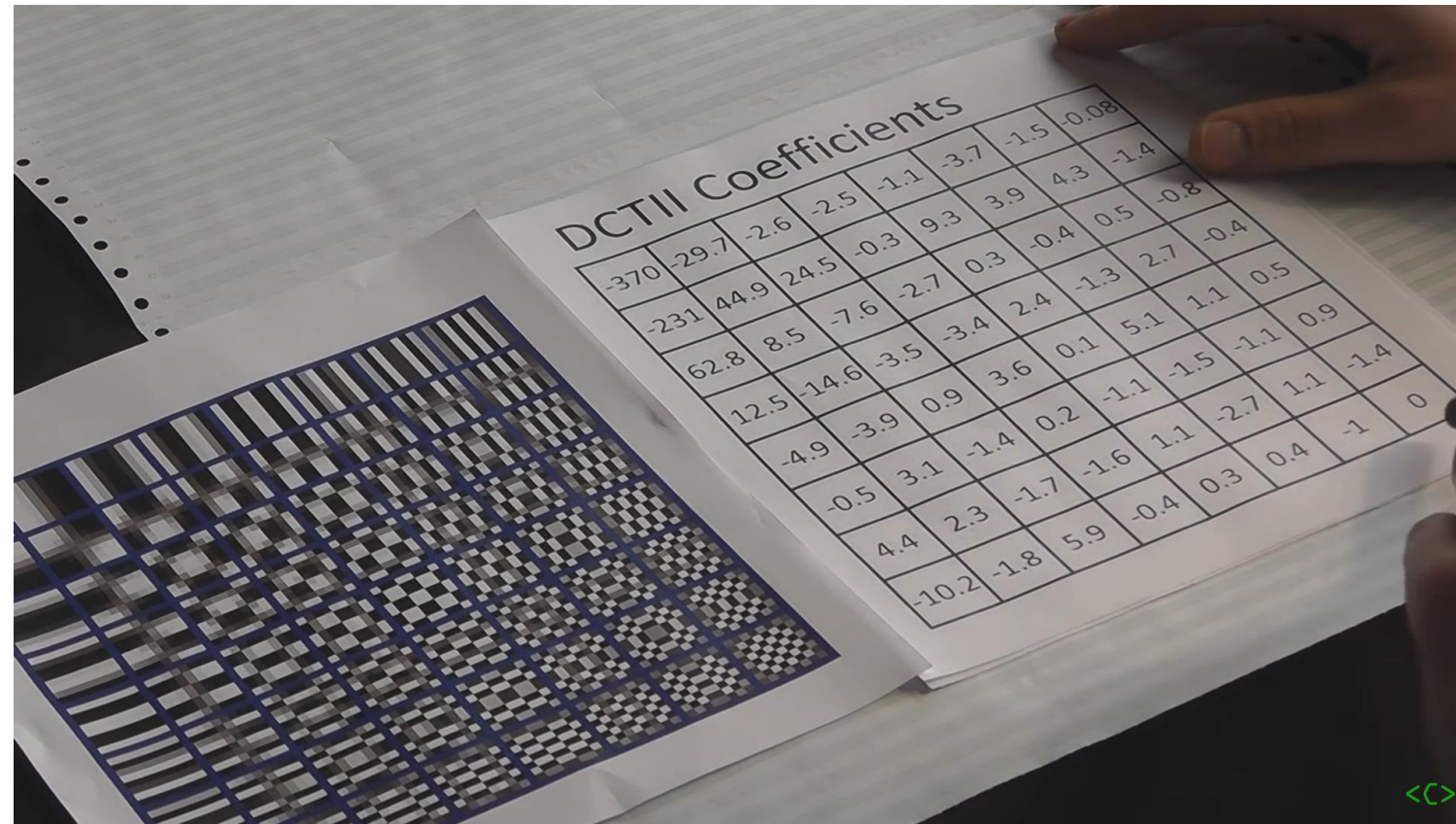
DCT-Coefficients



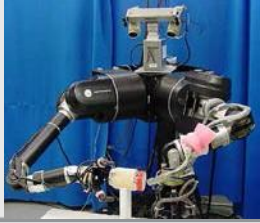
DCT-Coefficients



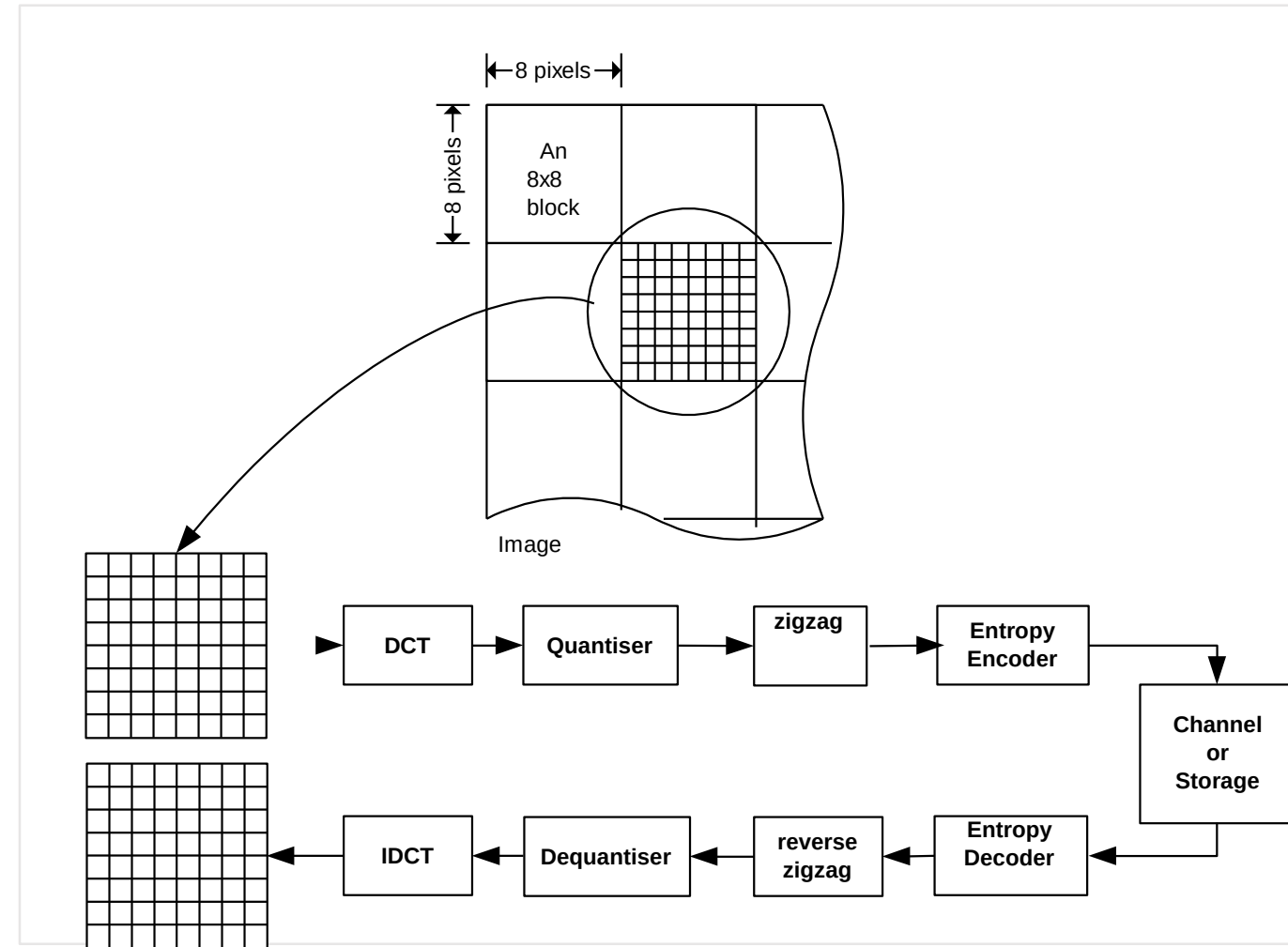
How JPEG Works – Quantization - Part 3



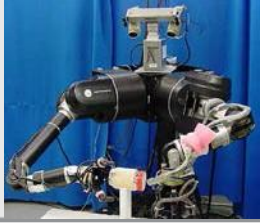
JPEG Compression



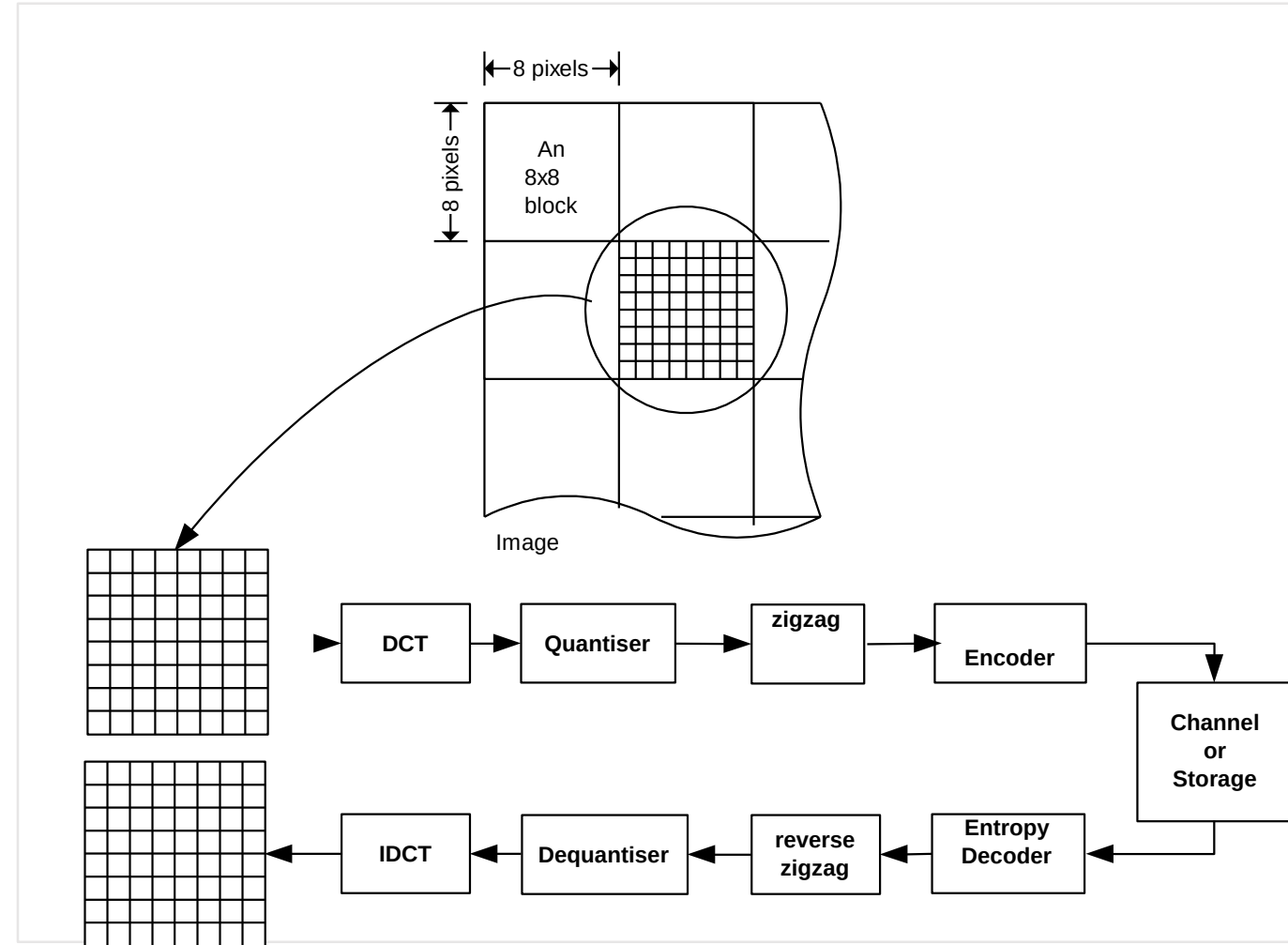
- To perform the JPEG coding, an image (in color or grey scales) is first subdivided into blocks of 8x8 pixels.
- The Discrete Cosine Transform (DCT) is then performed on each
- This generates 64 coefficients which are then quantized to reduce their magnitude.



JPEG Compression



- The coefficients are then reordered into a one-dimensional array in a zigzag manner before further entropy encoding.
- The **compression is achieved in two stages**; the first is **during quantization** and the second **during the entropy coding** process.
- JPEG decoding is the reverse process of coding.



JPEG Quantization

- Goal: to “throw out” bits (truncation)
- Quantization in JPEG = Process of removing high frequency data
- Uniform quantization: divide coefficient values $S(u,v)$ by N and round result

Q: In $S(u,v)$ how many bits should be truncated?

A: use quantization tables

- quantization tables consist of 64 elements, each value uses 8-bit: Q_{uv}
- new compressed values by using tables $Sq_{uv} = S_{uv} / Q_{uv}$
- standard defines two default tables (luminance, chroma)

JPEG Quantization

Luminance quantization table

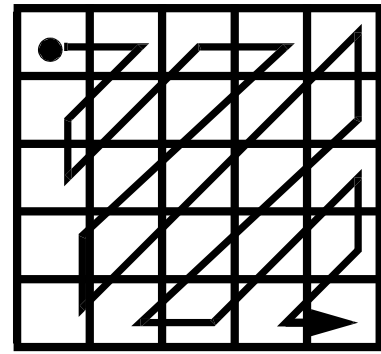
16	11	12	14	12	10	16	14
13	14	18	17	16	19	24	40
26	24	22	22	24	49	35	37
29	40	58	51	61	60	57	51
56	55	64	72	92	78	64	68
87	69	55	56	80	109	81	87
95	98	103	104	103	62	77	113
121	112	100	120	92	101	103	99

Human eye most sensitive to low frequencies (upper left corner)

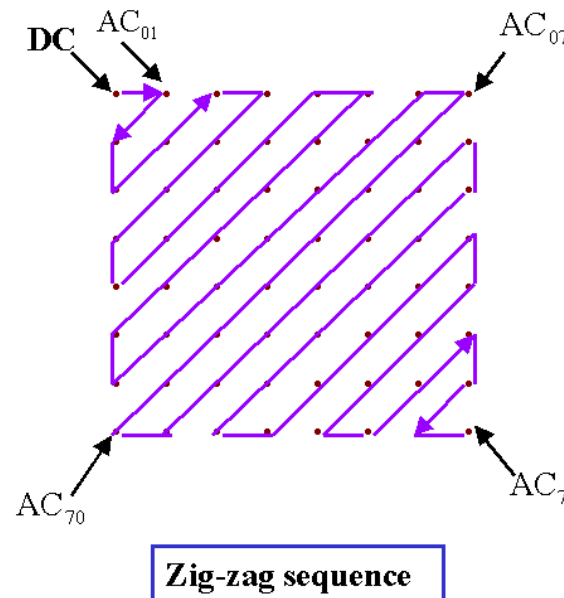
Quantization process

- Divide DCT-coefficient value by an integer number from the quantization table
- Round the result to the nearest integer
- Quantization of all DCT-coefficients results in a *lossy* transformation – some image details given by higher frequencies are cut off.
- JPEG application provides a table with 64 entries, each used for quantization of one DCT-coefficient
 - each coefficient can be adjusted separately
- A high compression factor is achievable on the expense of image quality
 - large quantization numbers: high data reduction but information loss increases
- No default values for quantization tables are specified in JPEG

JPEG Entropy Encoding



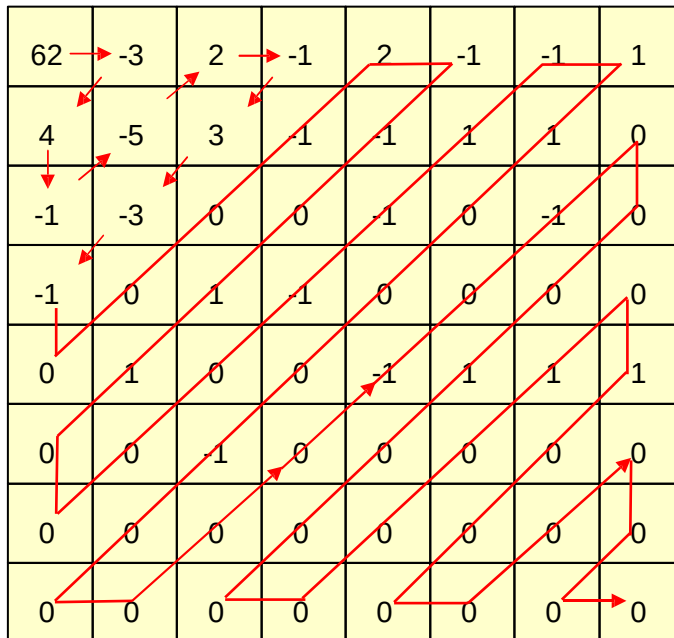
First step: map 8x8 pixel blocks into 64 element vector using zig-zag-scan



- DC coefficients processing:
 - DC coefficients determine basic color of data unit
 - DC coefficient is large, but often close to previous value
==> encode difference

Example

Zig-zag ordering



DC-coefficient: code coefficients for one block as difference to the previous one

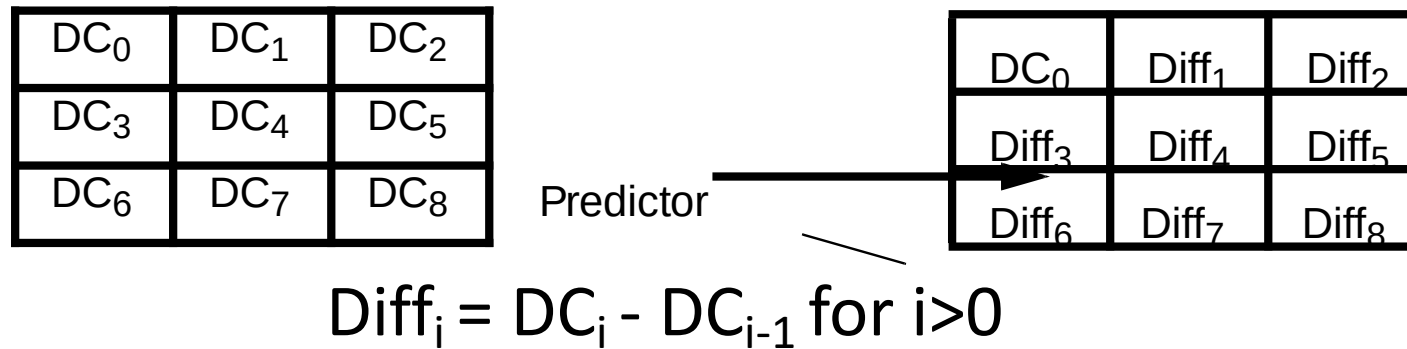
AC-coefficients: consider each block separately, order data using zig-zag sequence to achieve long sequences of zero- values:

[illegible]

Entropy encoding:

- Run-length encoding of zero values of quantized AC-coefficients
- Huffman encoding on DC- and AC-coefficients

JPEG Entropy Encoding



- AC coefficients processing:
 - processing order of the AC coefficients using zig-zag scan (coefficients with lower frequencies are encoded first) ==>
 - sequence of similar data bytes ==> efficient entropy coding
- JPEG standard specifies Huffman or Arithmetic coding, but sequential encoding mode uses only Huffman coding.

JPEG — Entropy Encoding

Algorithm:

1. apply run-length coding of AC coefficient of zero values
2. apply Huffman coding on DC and AC coefficients

(only DC coefficients explained in detail)

JPEG — Entropy Encoding

- DC coefficients—Huffman coding
 - categorize DC values into DC code tables
 - difference magnitude categories for DC coefficients (12 categories)

Diff Values	SSSS (number of bits needed to encode Diff)
0	0 bit
-1, 1	1 bit
-3, ..., 3	2 bits
-7, ..., 7	3 bits
.	
.	
.	
-2047, ..., 2047	11 bits

JPEG — Entropy Encoding

- DC coefficients — Huffman encoding
 - handle SSSS as Huffman symbol, get $p(0)$, $p(1)$, ... $p(11)$ and create Huffman tree ==> Huffman code for SSSS
 - for each category, an additional bits field is appended to the code word to uniquely identify which difference in that category actually occurred.
 - send: (Huffman codeword, actual value)

Example: If SSSS=2 has the Huffman code 001 and Diff=-3 ==>

send 00100

Huffman code for 2 -3 in two's complement is 00 because 3=>11

Details about JPEG Entropy Coding in

<https://cnx.org/contents/IJvrSfU5@3.3:eMc9DKWD@5/JPEG-Entropy-Coding>

JPEG Image Processing (DCT) - Decoding

- Inverse Discrete Cosine Transformation (I-DCT) maps DCT coefficients to sampled values

$$s_{xy} = \frac{1}{4} \sum_{u=0}^7 \sum_{v=0}^7 c_u c_v s_{vu} \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

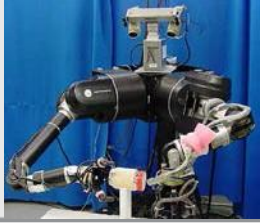
with $c_u, c_v = 1/\sqrt{2}$ for $u,v=0$; otherwise $c_u, c_v = 1$

- DCT and I-DCT cannot be calculated in full precision
==> lossy compression
- JPEG does not define precision parameters, therefore various implementations exist
- many AC coefficients with small values (around zero)

Comments

- Applications do not have to include both an encoder and decoder if the compression process agrees on a common table
- The encoded data stream has a fixed *Interchange Format*
 - encoded image data
 - chosen parameters
 - tables of the coding process
- In regular mode, the interchange format includes all of the information necessary for decoding without any previous knowledge of the coding process

DCT Examples



- DCT sorts values from the lowest to the highest frequency
- In image blocks it is likely that the energy is concentrated in low frequencies
 - Regions with smooth colors or little details (= low spatial frequencies) have high values
 - Regions with different colors and detail (= high spatial frequencies), most values are almost zero.



8x8 Block (Luminance)

134	142	145	131	114	122	131	130
129	143	134	130	135	144	134	118
123	117	118	111	97	109	130	143
129	116	112	116	120	126	130	118
118	127	141	138	138	148	141	125
125	129	119	127	143	149	145	136
131	126	128	142	141	135	126	116
131	140	146	154	133	118	124	124

Pixelvalues

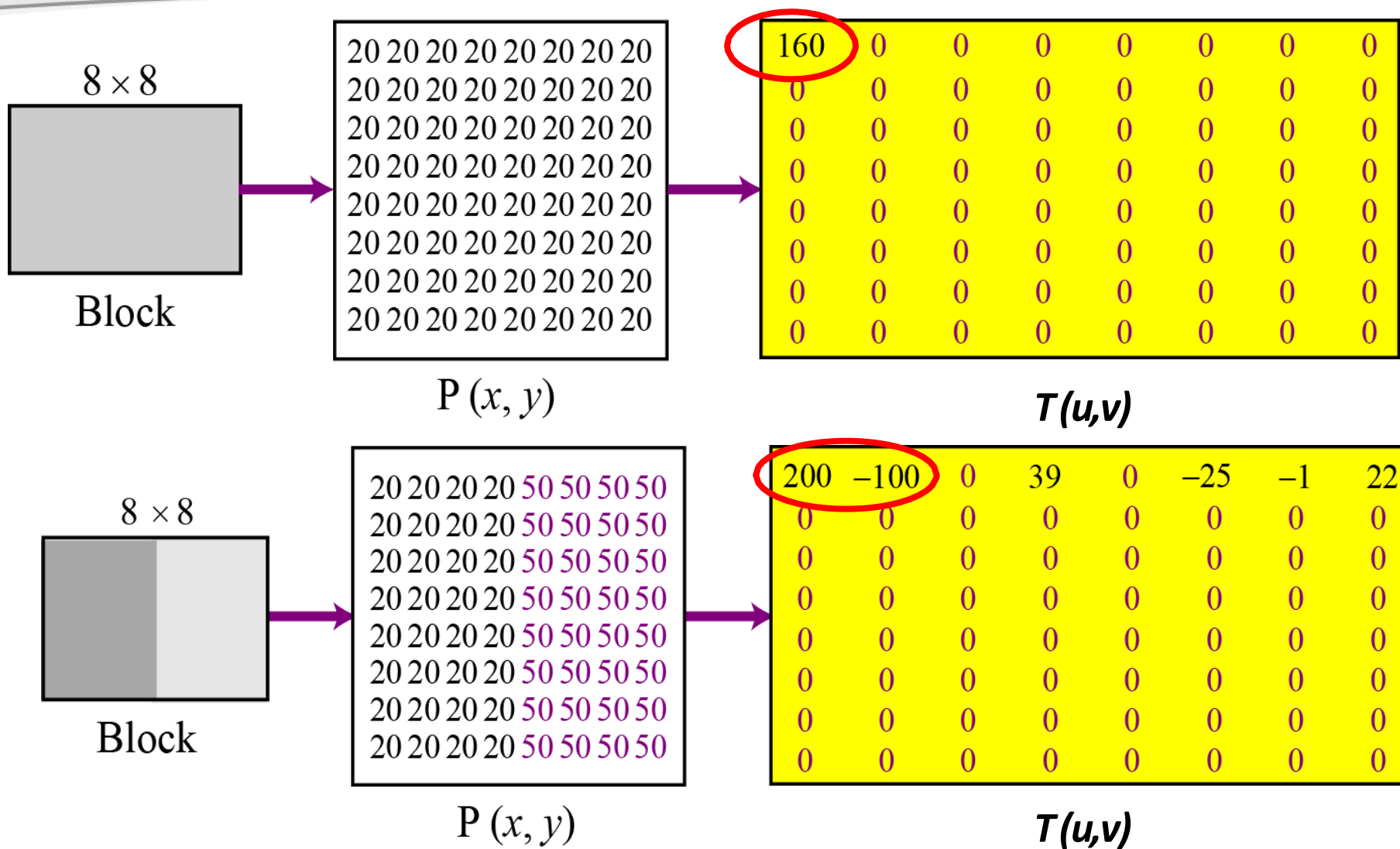
Low frequencies

1037	-1	-6	1	-12	8	-4	-4
-16	1	28	-6	-14	0	4	0
19	32	-7	-19	2	-1	-4	-3
29	-9	-14	13	-10	-6	1	0
4	14	-6	-13	-2	7	1	2
-26	2	16	2	11	6	1	1
-10	-11	27	-18	4	1	0	0
-2	1	1	-19	-1	6	6	0

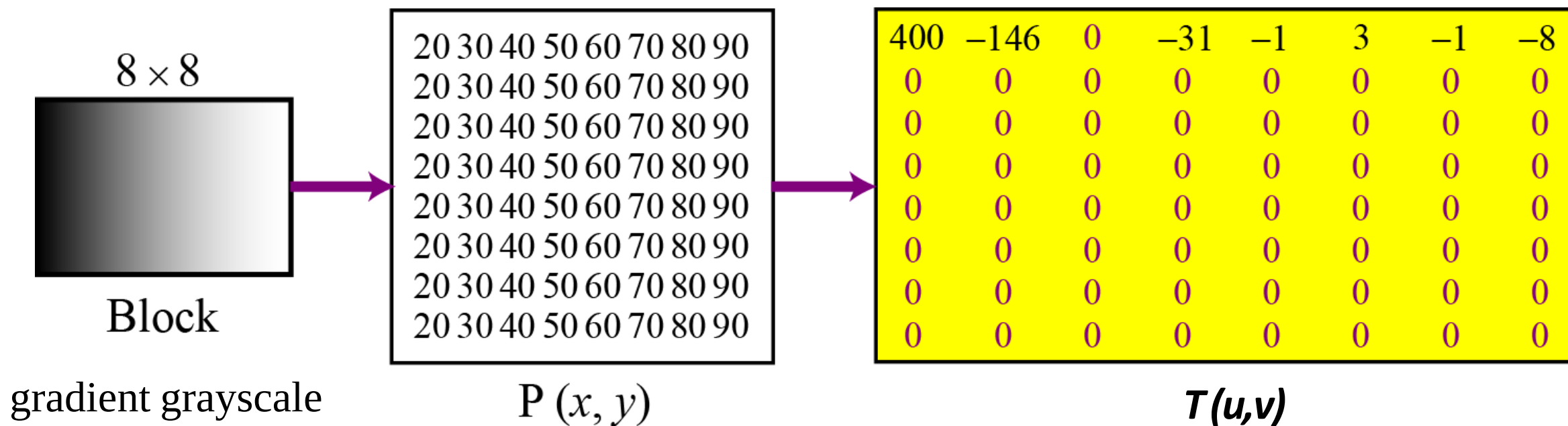
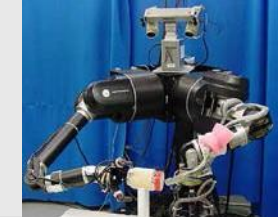
High frequencies

DCT- Values

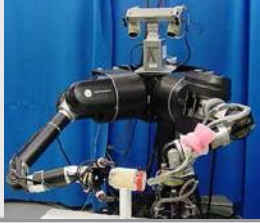
DCT Examples



DCT Examples

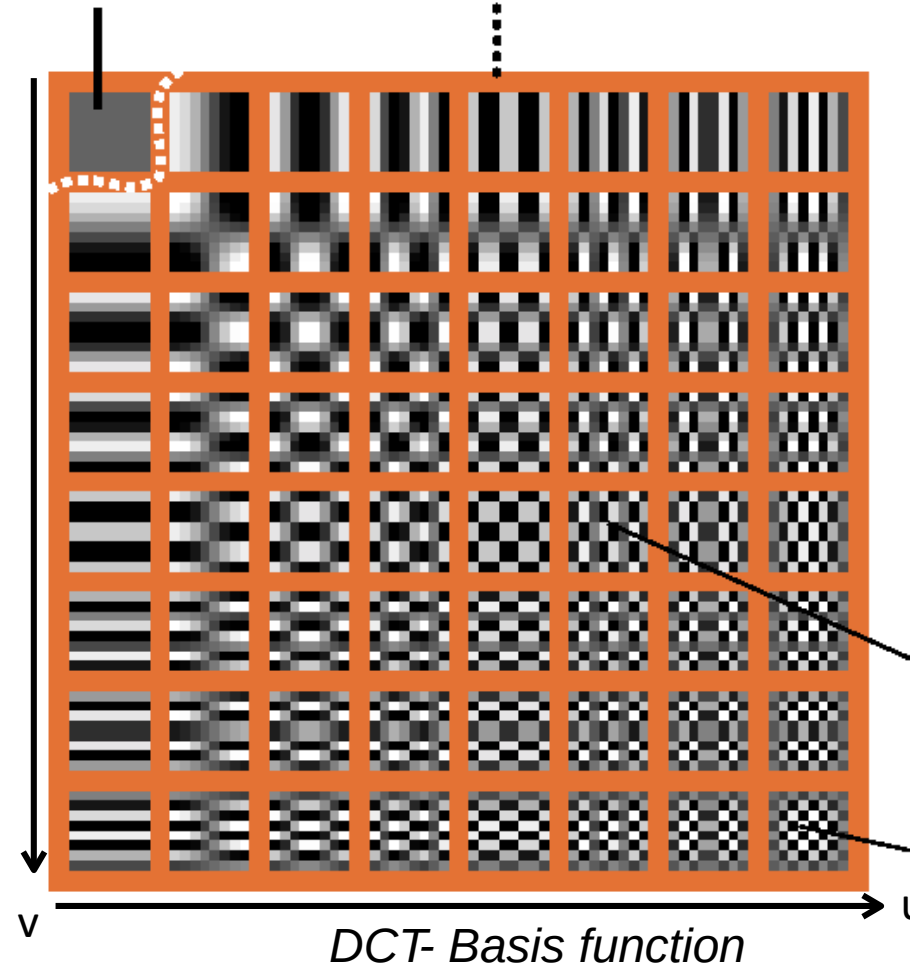


DCT Basis Functions

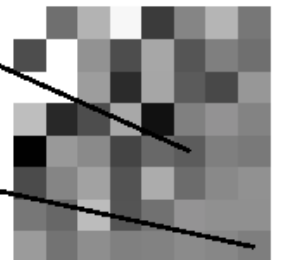
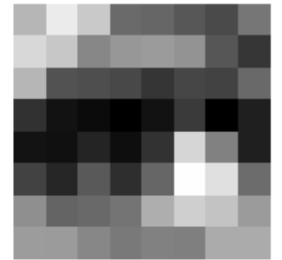


- DCT values indicate the weighting of frequency images
- The brighter the pixel, the greater the value of DCT

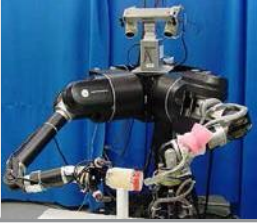
Constant component Alternating component



8 x 8 Imageblock



Quantization



- Compression effect: DCT values are approximated (quantized) to make them smaller and to repeat themselves
 - Each DCT value is divided by a quantization factor and then rounded
 - The larger the quantization factor, the smaller the values to be stored

134	142	145	131	114	122	131	130
129	143	134	130	135	144	134	118
123	117	118	111	97	109	130	143
129	116	112	116	120	126	130	118
118	127	141	138	138	148	141	125
125	129	119	127	143	149	145	136
131	126	128	142	141	135	126	116
131	140	146	154	133	118	124	124

Pixelvalues

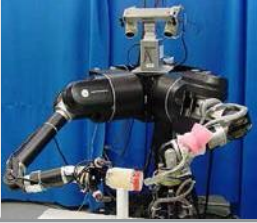
1037	-1	-6	1	-12	8	-4	-4
-16	1	28	-6	-14	0	4	0
19	32	-7	-19	2	-1	-4	-3
29	-9	-14	13	-10	-6	1	0
4	14	-6	-13	-2	7	1	2
-26	2	16	2	11	6	1	1
-10	-11	27	-18	4	1	0	0
-2	1	1	-19	-1	6	6	0

DCT- Values

130	0	0	0	-1	0	0	0
-2	0	3	0	-1	0	0	0
2	3	0	-1	0	0	0	0
3	-1	-1	1	0	0	0	0
0	1	0	-1	0	0	0	0
-2	0	2	0	0	0	0	0
-1	-1	2	-1	0	0	0	0
0	0	0	-1	0	0	0	0

Quantized DCT- Values

Quantization Matrix



- Quantization factor for each DCT value is defined using a quantization matrix
 - Disturbances in low frequency parts of the image are perceived strongly
 - Disturbances in high frequency parts of the image are less noticeable
 - Quantization matrices in JPEG standard ensures that the DCT values for low frequencies are stored more accurately than for high frequencies
 - Quantization table can be freely selected in the compression

8	16	19	22	26	27	29	34
16	16	22	24	27	29	34	37
19	22	26	27	29	34	34	38
22	22	26	27	29	34	37	40
22	26	27	29	32	35	40	48
26	27	29	32	35	40	48	58
26	27	29	34	38	46	56	69
27	29	35	38	46	56	69	83

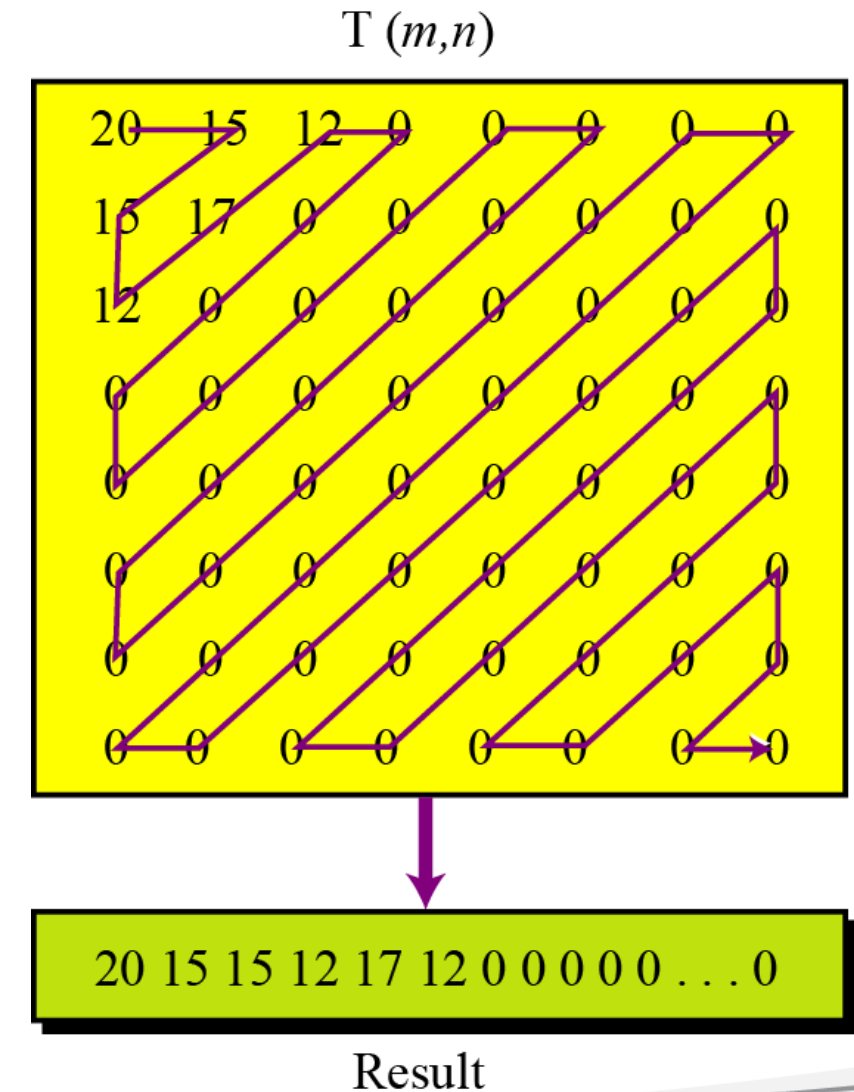
Quantization Matrix

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

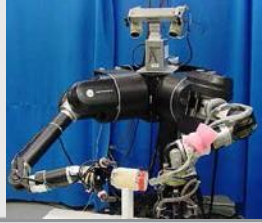
Quantization Matrix

Compression

- After quantization values are read from the table, and redundant 0s are removed.
- To cluster 0s together process reads the table diagonally in a zigzag fashion because if image does not have fine changes bottom right corner of table is all 0s.
- JPEG uses run-length encoding at the compression phase to compress the bit pattern resulting from the zigzag linearization.



Quantization



Coding

134	142	145	131	114	122	131	130
129	143	134	130	135	144	134	118
123	117	118	111	97	109	130	143
129	116	112	116	120	126	130	118
118	127	141	138	138	148	141	125
125	129	119	127	143	149	145	136
131	126	128	142	141	135	126	116
131	140	146	154	133	118	124	124

Pixelvalues

1037	-1	-6	1	-12	8	-4	-4
-16	1	28	-6	-14	0	4	0
19	32	-7	-19	2	-1	-4	-3
29	-9	-14	13	-10	-6	1	0
4	14	-6	-13	-2	7	1	2
-26	2	16	2	11	6	1	1
-10	-11	27	-18	4	1	0	0
-2	1	1	-19	-1	6	6	0

DCT- Values

130	0	0	0	-1	0	0	0
-2	0	3	0	-1	0	0	0
2	3	0	-1	0	0	0	0
3	-1	-1	1	0	0	0	0
0	1	0	-1	0	0	0	0
-2	0	2	0	0	0	0	0
-1	-1	2	-1	0	0	0	0
0	0	0	-1	0	0	0	0

Quantized DCT- Values

Decoding

1040	0	0	0	-9	0	0	0
-12	0	24	0	-10	0	0	0
14	24	0	-10	0	0	0	0
24	-8	-9	10	0	0	0	0
0	9	0	-10	0	0	0	0
-19	0	10	0	0	0	0	0
-9	-10	21	-12	0	0	0	0
0	0	0	-14	0	0	0	0

Reconstructed DCT- Values

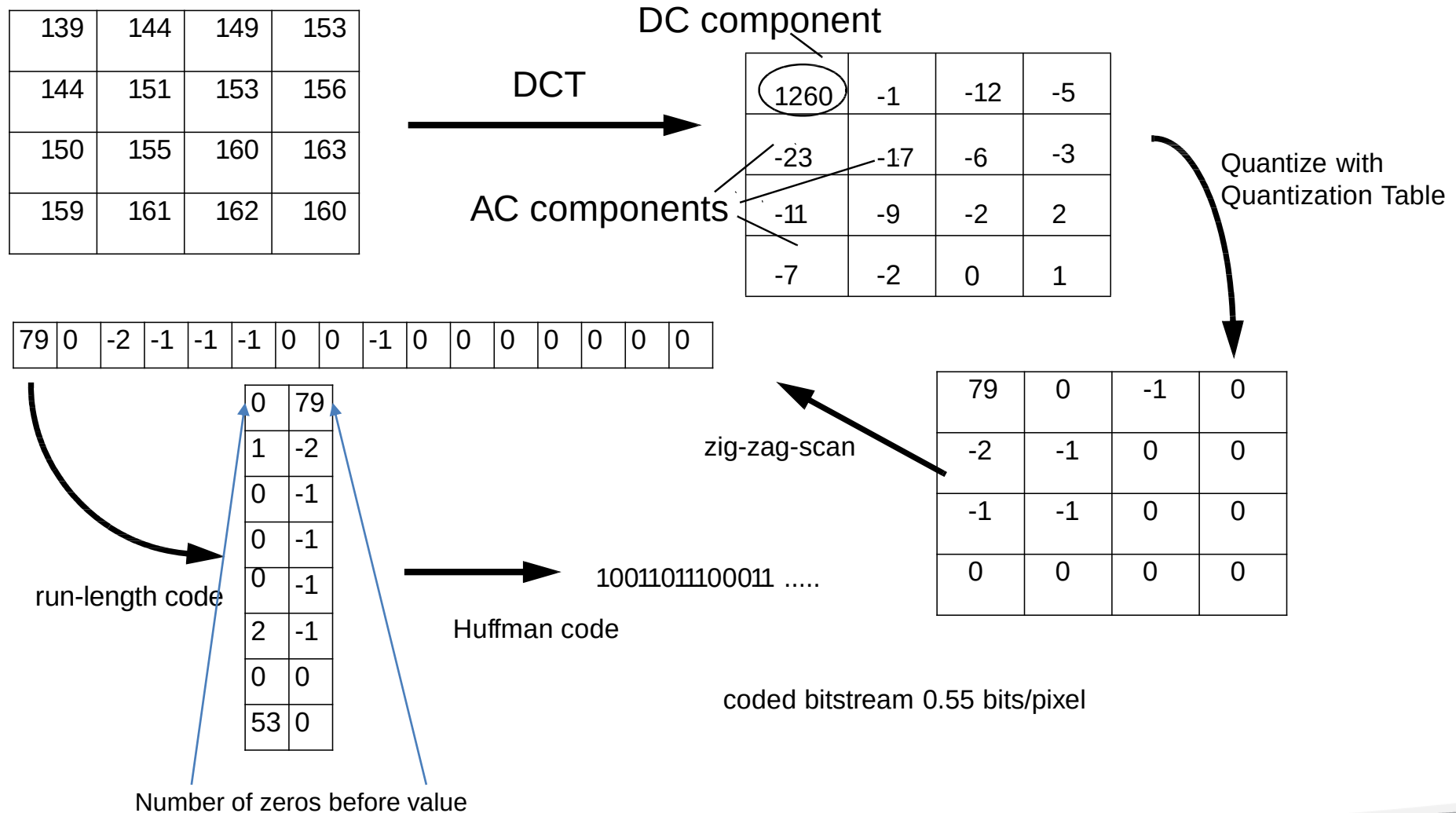
136	141	138	125	119	125	132	134
136	137	133	130	134	139	133	121
121	125	122	112	107	117	130	138
125	123	119	117	121	128	127	122
123	129	135	139	140	139	136	131
129	125	124	130	139	144	141	136
129	130	134	139	140	135	127	120
132	138	144	141	131	122	122	127

Reconstructed Pixels (IDCT)

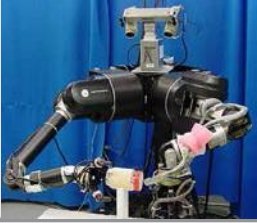
-2	1	7	6	-5	-3	-1	-4
-7	6	1	0	1	5	1	-3
2	-8	-4	-1	-10	-8	0	5
4	-7	-7	-1	-1	-2	3	-4
-5	-2	6	-1	-2	9	5	-6
-4	4	-5	-3	4	5	4	0
2	-4	-6	3	1	0	-1	-4
1	2	2	13	2	-4	1	-3

Errors (Difference)

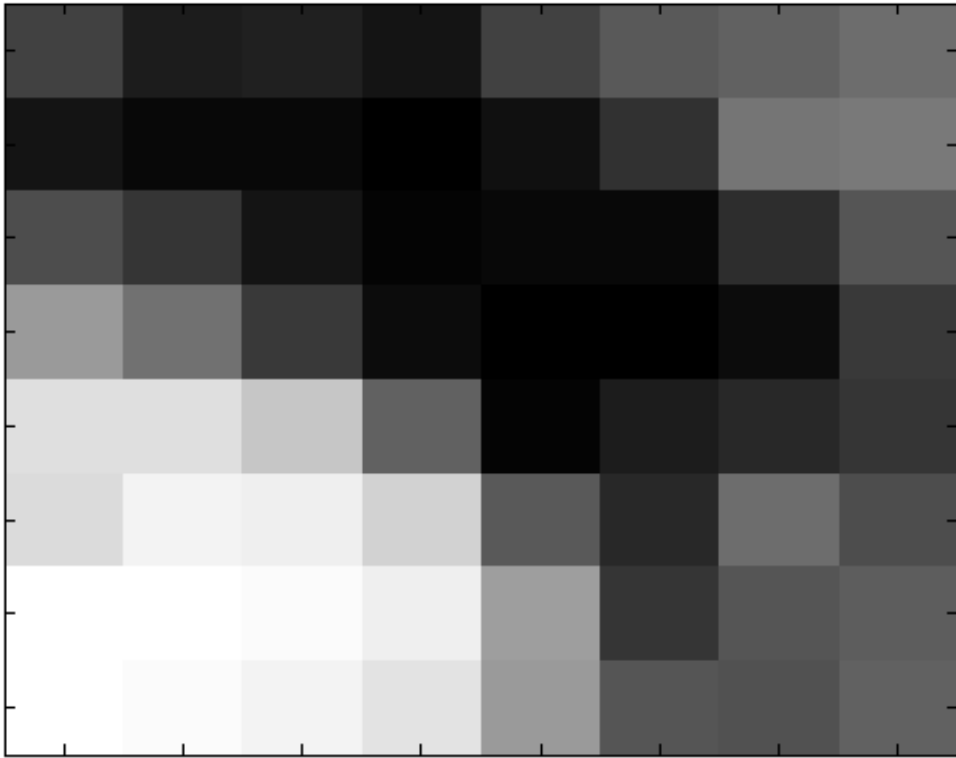
JPEG Block Encoding (Example)



Can You Tell the Difference?



Original



Compressed

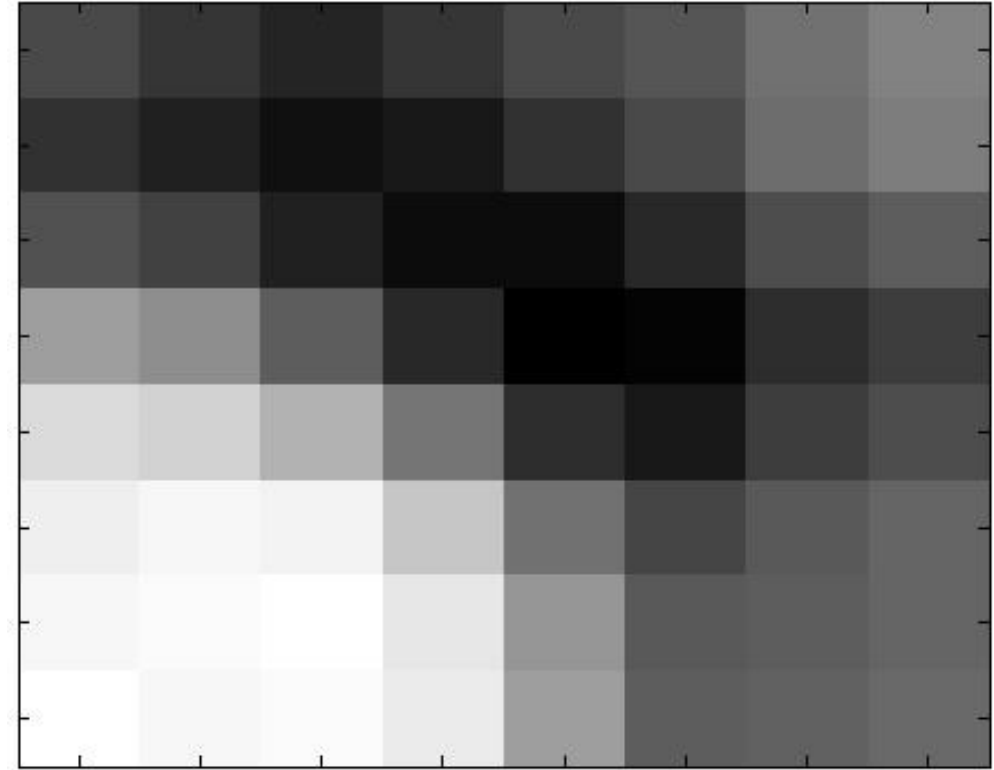
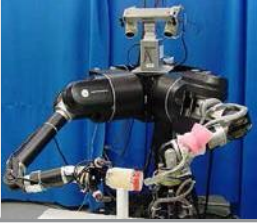


Image Compression



Original



Compressed



VisJPEG-Applet



Originalbild:

☒ skalieren ☐ scrollen

Haus (192x144)

Eigenes Bild laden...

Bild speichern...

Neu berechnen

JPEG komprimiert:



☒ Normal ☐ Differenz

Gesamtkompression:

6%

Info

Farbraum DCT Quantisierung Zick-Zack-Scan DPCM + RLE Entropie-Kodierung

Schritt 1: Farbraumkonvertierung

Zurück Info Weiter

Rotkanal:



Y-Kanal:



Grünkanal:



Cr-Kanal:



Blaukanal:



Cb-Kanal:



Cr/Cb Subsampling:

2x2

Skalierungsqualität:

mittel

Datenreduktion in diesem Schritt:

50%

Problems with JPEG

- Behavior in sharp transitions (e.g. fonts)
- Emergence of the 8x8 blocks at high compression rates.
- The greater the compression (quantization), the more artefacts occur (block formation)
- JPEG is made for natural images and not for artificial images (computer graphics)!



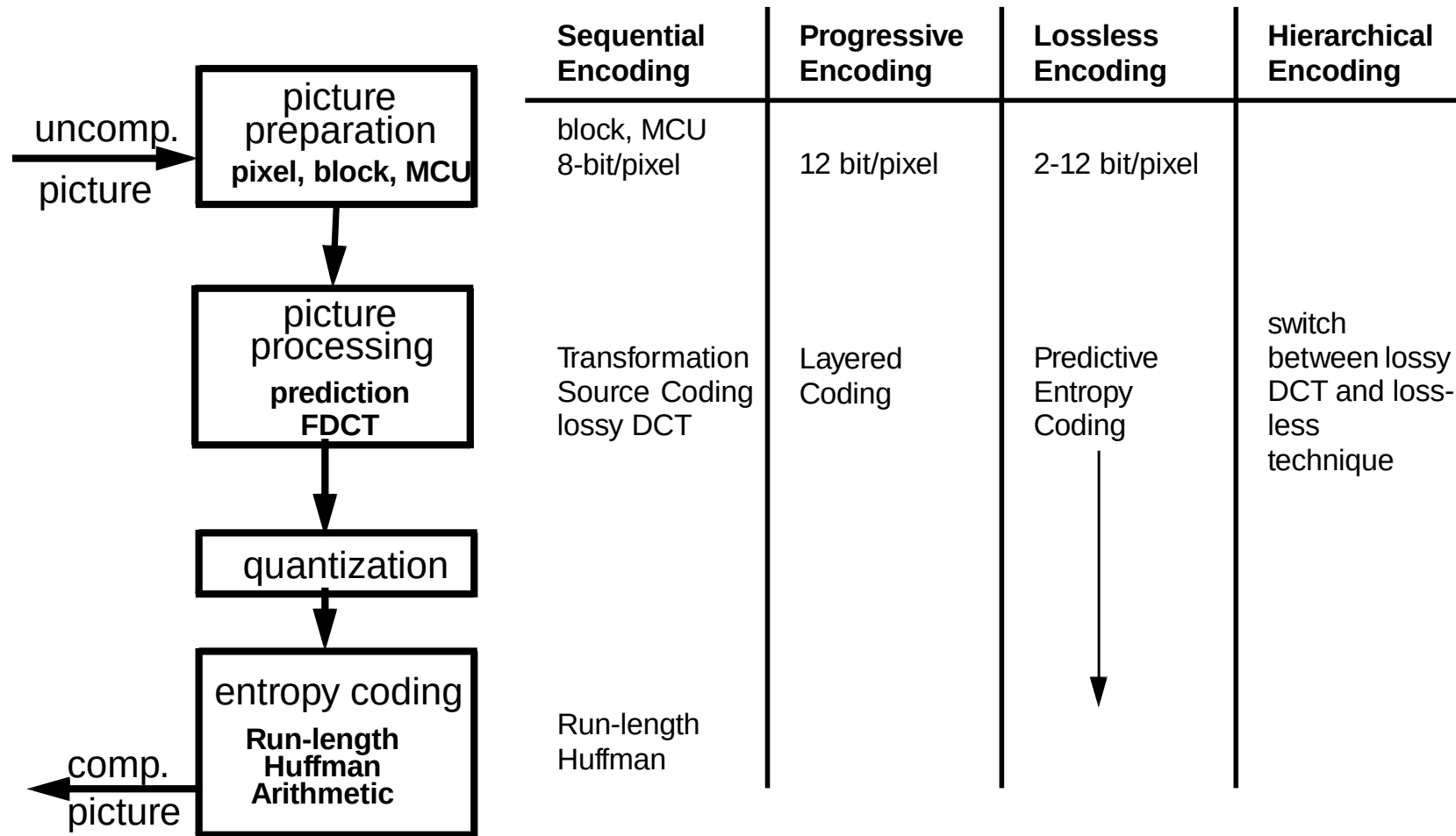
<https://www.youtube.com/watch?v=yBX8GFqt6GA&list=PLzH6n4zXuckoAod3z31QEST1ZaizBuNHh&index=4>

Variations & Additions to JPEG

Additional Material (not for Exam!)

Slides by Prof. Dr. Otto Spaniol, Dirk Thißen
RWTH Aachen

JPEG Processing Steps

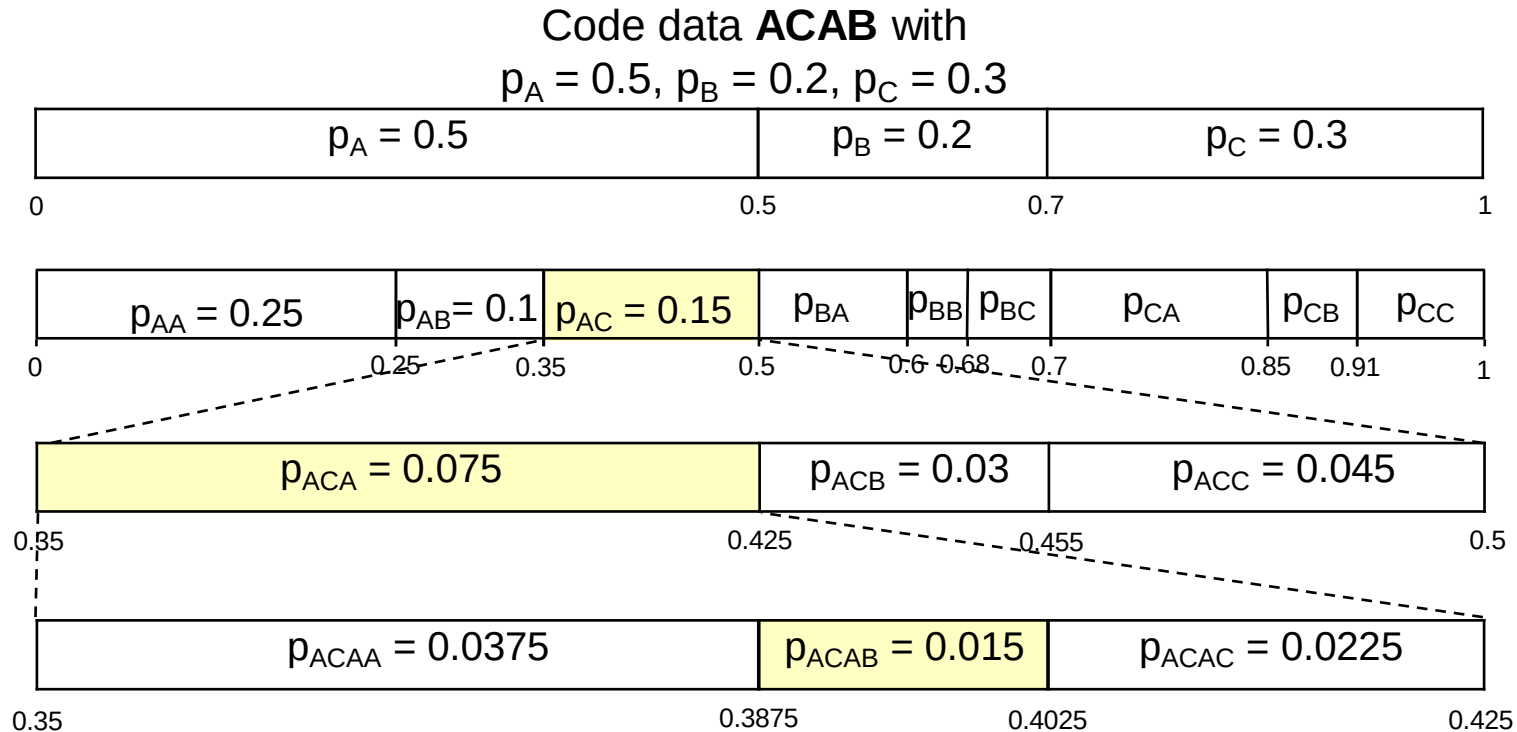


Alternative to Huffman: Arithmetic Coding

Characteristics:

- Achieves optimality (coding rate) as the Huffman coding
- Difference to Huffman: the entire data stream has an assigned probability, which consists of the probabilities of the contained characters. Coding a character takes place with consideration of all previous characters.
- The data are coded as an interval of real numbers between 0 and 1. Each value within the interval can be used as code word.
- The minimum length of the code is determined by the assigned probability.
- Disadvantage: the data stream can be decoded only as a whole.

Arithmetic Coding: Example



ACAB can be coded by each binary number from the interval $[0.3875, 0.4025)$, rounded up to $-\log_2(p_{ACAB}) = 6.06$ i.e. 7 bit, e.g. **0.0110010**

Variants of Image Compression

JPEG is not a single format, but it can be chosen from a number of modes:

Lossy sequential DCT-based mode (baseline process)

- Presented before, but not the only method

Expanded lossy DCT-based mode

- Enhancement to the baseline process by adding progressive encoding

Lossless mode

- Low compression ratio → „perfect“ reconstruction of original image
- No DCT, but differential encoding

Hierarchical mode

- Accommodates images of different resolutions
- Selects its algorithms from the three other modes

Variants: Expanded Lossy DCT-based Mode

With *sequential encoding* as in the baseline process the whole image is coded and decoded in a single run. An alternative to sequential encoding is *progressive encoding*, done in the entropy encoding step.

Two alternatives for progressive encoding are possible:

- *Spectral selection*

At first, coefficients of low frequencies are passed to entropy encoding, coefficients of higher frequencies are processed in successive runs

- *Successive approximation*

All coefficients are transferred in one run, but most-significant bits are encoded prior to less-significant bits.

12 possible coding alternatives in the expanded mode:

- Using sequential encoding, spectral selection, or successive approximation (3 variants)
- Using Huffman or Arithmetic encoding (2 variants)
- Using 8 or 12 bits for representing the samples (2 variants)

Most popular mode: sequential display mode with 8 bits/sample and Huffman encoding

Expanded Lossy DCT-based Mode (Example)

Sequential encoding: image is coded and decoded in a single run



Step 1



Step 2



Step 3

Progressive encoding: image is coded and decoded in refining steps



Step 1



Step 2



Step 3

Variants: Lossless Mode

Lossless mode uses *differential encoding*

(Differential encoding is also known as *prediction* or *relative encoding*)

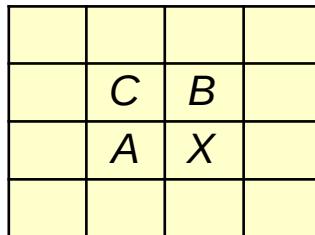
- Sequence of characters whose values are different from zero, but which do not differ much.
 - Calculate only the difference wrt. the previous value (used also for DC-coefficients)

Differential encoding for still images:

- Avoid using DCT/quantization
- Instead: calculation of differences between nearby pixels or pixel groups
 - Edges are represented by large values
 - Areas with similar luminance and chrominance are represented by small values
 - Homogenous area is represented by a large number of zeros
 - further compression with run-length encoding is possible as for DCT

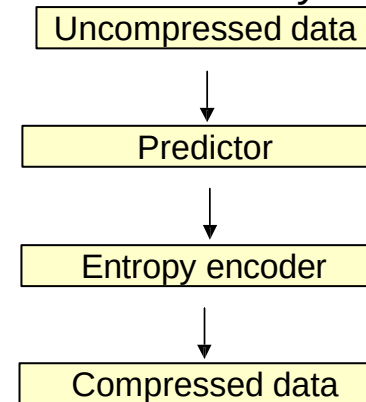
Variants: Lossless Mode

- Uses data units of single pixels for image preparation
- Any precision between 2 and 16 bits/pixel can be used
- Image processing and quantization use a predictive technique instead of transformation encoding
- 8 predictors are specified for each pixel X by means of a combination of the already known adjacent samples A , B , and C



The actual predictor should give the best approximation of x by the already known values A, B, C

predictor	predicted values X
0	no prediction
1	A
2	B
3	C
4	$A+B-C$
5	$A+(B-C)/2$
6	$B+(A-C)/2$
7	$(A+B)/2$



The number of the chosen predictor and the difference of the prediction to the actual value are passed to entropy encoding (Huffman or Arithmetic Encoding)

Example:

(4,0): X is exactly given by $A+B-C$
 (7,1): X is $(A+B)/2+1$

Variants: Hierarchical Mode

The *Hierarchical mode* uses either the lossy DCT-based algorithms or the lossless compression technique

The idea: encoding of an image at different resolutions

Algorithm:

- Image is initially sampled at a low resolution
- Subsequently, the resolution is raised and the compressed image is subtracted from the previous result
- The process is repeated until the full resolution of the image is obtained in a compressed form

Disadvantage:

- Requires substantially more storage capacity

Advantage:

- Compressed image is immediately available at different resolutions
→ *scaling becomes cheap*